Chemical Mixtures and Accelerated Biological Aging in CKD: NHANES Analysis

Reproducible NHANES analysis By January

2026-05-09

Contents

Study Objective 1

Configuration 1

Packages 3

Helper Functions 3

Data Acquisition and Cycle Availability 8

Variable Construction: PhenoAge, CKD, Covariates 10

Exposure Processing and LOD Handling 22

Analytic Sample and Survey Design 25

Mixture Modeling 29

Sample Flow (PRISMA-style) 45

PFAS Exposure Distributions 47

Tables 50

Figures 69

Cautions and Limitations 75

Reproducibility 77

Study Objective

Primary question: are environmental chemical mixtures associated with accelerated biological aging (PhenoAge acceleration), and is this association stronger among individuals with CKD?

Configuration

```
config <- list(
  cycles = c("2005-2006", "2007-2008", "2009-2010", "2011-2012", "2013-2014", "2015-2016", "2017-2018")
```

```r
  exposure_family = "pfas", # pfas, metals, both
  q = 4,
  min_age = 20,
  ckd_def = "egfr_or_acr", # egfr_or_acr, egfr_only, acr_only
  use_residualized_phenoage = TRUE,   # PhenoAgeAccel is always residualized; option reserved for futur
  # Primary analysis uses fixed 4-PFAS set with complete-case; strict (qgcomp-compatible subset) is sen
  require_complete_exposure_set = FALSE,  # Legacy: primary is always fixed-set complete-case; no longe
  min_nonmissing_exposure_fraction = 0.6,
  min_nonmissing_exposures_floor = 2,
  include_clinical_covariates = TRUE,    # Not wired to primary adjustment set; reserved for future sen
  # Weight is resolved from audit; PFAS workflow prioritizes environmental subsample WTS* weights.
  weight_type = "auto",  # Auto-resolve from audit in the exposure-complete sample (PFAS excludes WTINT
  # Multiple imputation (MICE) for missing covariates.
  use_mice = TRUE,        # If TRUE, Model B + sensitivities + supplementary use MI-pooled estimates (R
  mice_m = 24,            # Number of imputations: m >= % missing on most-missing covariate (alcohol3 ~
  mice_maxit = 20,        # Iterations per chain.
  mice_seed = 20260227,   # Reproducibility seed for mice.
  verbose = TRUE
)
config
```

```
## $cycles
## [1] "2005-2006" "2007-2008" "2009-2010" "2011-2012" "2013-2014" "2015-2016"
## [7] "2017-2018"
##
## $exposure_family
## [1] "pfas"
##
## $q
## [1] 4
##
## $min_age
## [1] 20
##
## $ckd_def
## [1] "egfr_or_acr"
##
## $use_residualized_phenoage
## [1] TRUE
##
## $require_complete_exposure_set
## [1] FALSE
##
## $min_nonmissing_exposure_fraction
## [1] 0.6
##
## $min_nonmissing_exposures_floor
## [1] 2
##
## $include_clinical_covariates
## [1] TRUE
##
## $weight_type
## [1] "auto"
```

```
##
## $use_mice
## [1] TRUE
##
## $mice_m
## [1] 24
##
## $mice_maxit
## [1] 20
##
## $mice_seed
## [1] 20260227
##
## $verbose
## [1] TRUE
```

Packages

```r
required_pkgs <- c(
  "nhanesA", "dplyr", "tidyr", "purrr", "stringr", "survey", "splines",
  "ggplot2", "broom", "knitr", "qgcomp", "forcats", "tibble",
  "mice", "mitools", "patchwork", "hexbin"
)
invisible(lapply(required_pkgs, function(p) {
  if (!requireNamespace(p, quietly = TRUE)) stop("Package not installed: ", p)
}))

library(nhanesA)
library(dplyr)
library(tidyr)
library(purrr)
library(stringr)
library(survey)
options(survey.lonely.psu = "adjust")
library(splines)
library(ggplot2)
library(broom)
library(knitr)
library(qgcomp)
library(forcats)
library(tibble)
library(mice)
library(mitools)
library(patchwork)
```

Helper Functions

```r
cycle_to_suffix <- function(cycle) {
  map <- c(
    "1999-2000" = "A", "2001-2002" = "B", "2003-2004" = "C", "2005-2006" = "D",
    "2007-2008" = "E", "2009-2010" = "F", "2011-2012" = "G", "2013-2014" = "H",
```

```r
    "2015-2016" = "I", "2017-2018" = "J", "2019-2020" = "K"
  )
  out <- unname(map[cycle])
  if (is.na(out)) stop("Cycle not mapped: ", cycle)
  out
}

safe_nhanes <- function(table_name, verbose = TRUE) {
  out <- tryCatch(nhanesA::nhanes(table_name), error = function(e) NULL)
  if (is.null(out) && verbose) message("Unavailable table: ", table_name)
  out
}

resolve_table <- function(cycle, candidates, domain = NULL, verbose = TRUE) {
  suf <- cycle_to_suffix(cycle)
  to_try <- paste0(candidates, "_", suf)
  for (tb in to_try) {
    dat <- safe_nhanes(tb, verbose = FALSE)
    if (!is.null(dat)) {
      if (verbose) message("Using table ", tb, " for cycle ", cycle)
      return(list(code = candidates[1], table = tb, data = dat))
    }
  }
  NULL
}

coalesce_num <- function(df, vars) {
  vars <- vars[vars %in% names(df)]
  if (length(vars) == 0) return(rep(NA_real_, nrow(df)))
  v <- lapply(df[vars], function(x) suppressWarnings(as.numeric(as.character(x))))
  dplyr::coalesce(!!!v)
}

parse_yes_no <- function(x) {
  x_num <- suppressWarnings(as.numeric(as.character(x)))
  x_chr <- toupper(trimws(as.character(x)))
  case_when(
    x_num == 1 ~ 1,
    x_num == 2 ~ 0,
    str_detect(x_chr, "^YES$") ~ 1,
    str_detect(x_chr, "^NO$") ~ 0,
    TRUE ~ NA_real_
  )
}

parse_smq040 <- function(x) {
  x_chr <- toupper(trimws(as.character(x)))
  x_num <- suppressWarnings(as.numeric(as.character(x)))
  case_when(
    x_num %in% c(1, 2) ~ 1,
    x_num == 3 ~ 0,
    str_detect(x_chr, "EVERY DAY|SOME DAYS") ~ 1,
    str_detect(x_chr, "^NOT AT ALL$") ~ 0,
```

```r
    TRUE ~ NA_real_
  )
}

# SMQ020 = "Smoked at least 100 cigarettes in entire life?" (the gate question for SMQ040).
# 1 = Yes (lifetime smoker), 2 = No (never smoker), 7/9 = Refused/Don't know -> NA.
parse_smq020 <- function(x) {
  x_chr <- toupper(trimws(as.character(x)))
  x_num <- suppressWarnings(as.numeric(as.character(x)))
  case_when(
    x_num == 1 ~ 1,
    x_num == 2 ~ 0,
    str_detect(x_chr, "^YES$") ~ 1,
    str_detect(x_chr, "^NO$")  ~ 0,
    TRUE ~ NA_real_
  )
}

# Lifetime "ever drinker" indicator harmonized across NHANES cycles.
#   ALQ101 (1999-2016) = ">=12 alcoholic drinks in any one year ever?"  (1=Yes / 2=No)
#   ALQ110 (1999-2016) = ">=12 drinks in entire life?"  asked when ALQ101=No (1=Yes / 2=No)
#   ALQ111 (2017+)     = "Ever had a drink of any kind of alcohol?"     (1=Yes / 2=No)
# Returns 1 if ANY indicator is Yes, 0 if all available indicators are No, NA otherwise
# (refused / don't know / not asked).
parse_alq_ever <- function(alq101, alq110, alq111) {
  e101 <- parse_yes_no(alq101)
  e110 <- parse_yes_no(alq110)
  e111 <- parse_yes_no(alq111)
  case_when(
    !is.na(e111) & e111 == 1 ~ 1,
    !is.na(e101) & e101 == 1 ~ 1,
    !is.na(e110) & e110 == 1 ~ 1,
    !is.na(e111) & e111 == 0 ~ 0,
    !is.na(e101) & e101 == 0 & (is.na(e110) | e110 == 0) ~ 0,
    TRUE ~ NA_real_
  )
}

# "Current drinker" (drank in past 12 months) harmonized across cycles.
#   ALQ121 (2013-2018) = "Past 12 mo, how often drink alcoholic beverages?"
#                         0 = Never in last year, 1-10 = various frequencies, 77/99 = Refused/DK.
#   ALQ120Q (1999-2016) = "How often drink alcoholic beverages?" (numeric # of times)
#                         in the unit specified by ALQ120U (week/month/year).
#                         0 = Never; 1-365 = numeric answer; 777/999 = Refused/DK.
# Returns 1 if any-frequency drinker, 0 if explicit "never in past year", NA otherwise.
parse_alq_current_yr <- function(alq121, alq120q) {
  c121 <- suppressWarnings(as.numeric(as.character(alq121)))
  c120 <- suppressWarnings(as.numeric(as.character(alq120q)))
  case_when(
    !is.na(c121) & c121 %in% 1:10 ~ 1,
    !is.na(c121) & c121 == 0 ~ 0,
    !is.na(c120) & c120 >= 1 & c120 <= 365 ~ 1,
    !is.na(c120) & c120 == 0 ~ 0,
```

```r
    TRUE ~ NA_real_
  )
}

parse_phq_item <- function(x) {
  x_chr <- toupper(trimws(as.character(x)))
  x_num <- suppressWarnings(as.numeric(as.character(x)))
  case_when(
    x_num == 1 ~ 0, x_num == 2 ~ 1, x_num == 3 ~ 2, x_num == 4 ~ 3,
    x_num %in% 0:3 ~ x_num,
    str_detect(x_chr, "^NOT AT ALL$") ~ 0,
    str_detect(x_chr, "^SEVERAL DAYS$") ~ 1,
    str_detect(x_chr, "^MORE THAN HALF THE DAYS$") ~ 2,
    str_detect(x_chr, "^NEARLY EVERY DAY$") ~ 3,
    TRUE ~ NA_real_
  )
}

yesno01 <- function(x) {
  xc <- toupper(trimws(as.character(x)))
  xn <- suppressWarnings(as.numeric(as.character(x)))
  case_when(
    xn == 1 ~ 1,
    xn == 2 ~ 0,
    xc %in% c("YES", "1") ~ 1,
    xc %in% c("NO", "2") ~ 0,
    TRUE ~ NA_real_
  )
}

compute_egfr_2021 <- function(scr_mgdl, age, sex_raw) {
  sex_chr <- toupper(trimws(as.character(sex_raw)))
  sex_num <- suppressWarnings(as.numeric(as.character(sex_raw)))
  male <- case_when(
    sex_num == 1 ~ TRUE,
    sex_num == 2 ~ FALSE,
    sex_chr == "MALE" ~ TRUE,
    sex_chr == "FEMALE" ~ FALSE,
    TRUE ~ NA
  )
  kappa <- ifelse(male, 0.9, 0.7)
  alpha <- ifelse(male, -0.302, -0.241)
  sex_mult <- ifelse(male, 1, 1.012)
  142 * pmin(scr_mgdl / kappa, 1)^alpha * pmax(scr_mgdl / kappa, 1)^(-1.200) * (0.9938^age) * sex_mult
}

compute_acr <- function(ua, ucr, acr_direct = NULL) {
  calc <- ifelse(!is.na(ua) & !is.na(ucr) & ucr > 0, (ua / ucr) * 100, NA_real_)
  dplyr::coalesce(acr_direct, calc)
}

compute_phenoage_components <- function(df, include_scr = TRUE, include_alb = TRUE) {
  # Levine et al. (2018) coefficients expect: albumin g/L, creatinine µmol/L, glucose mmol/L.
```

```r
  # NHANES provides: albumin g/dL, creatinine mg/dL, glucose often mg/dL (we standardize to mg/dL in pi
  alb_gL   <- df$alb * 10
  scr_umol <- df$scr * 88.4
  glu_mmol <- df$glu / 18.018
  alb_term <- if (include_alb) (-0.0336 * alb_gL) else 0
  scr_term <- if (include_scr) ( 0.0095 * scr_umol) else 0
  glu_term <- 0.1953 * glu_mmol

  xb <- -19.907 +
    alb_term +
    scr_term +
    glu_term +
    0.0954 * log(pmax(df$crp, 1e-6)) -
    0.0120 * df$lym +
    0.0268 * df$mcv +
    0.3306 * df$rdw +
    0.00188 * df$alp +
    0.0554 * df$wbc +
    0.0804 * df$age

  xb <- pmin(pmax(xb, -50), 50)
  gamma_const <- (exp(120 * 0.0076927) - 1) / 0.0076927
  hazard <- exp(xb) * gamma_const
  mort <- -expm1(-hazard)
  mort <- pmin(pmax(mort, 1e-12), 1 - 1e-12)
  pheno <- 141.50 + log(-0.00553 * log1p(-mort)) / 0.090165

  list(xb = xb, mort = mort, phenoage = pheno)
}

safe_phenoage <- function(df) {
  compute_phenoage_components(df, include_scr = TRUE, include_alb = TRUE)$phenoage
}

safe_qgcomp_extract <- function(fit_obj, model_label, n_obs) {
  if (is.null(fit_obj)) {
    return(tibble(model = model_label, beta = NA_real_, conf.low = NA_real_, conf.high = NA_real_, p.val
  }
  cf <- coef(fit_obj)
  psi_name <- grep("^psi", names(cf), value = TRUE)[1]
  if (is.na(psi_name)) psi_name <- grep("psi", names(cf), value = TRUE)[1]
  if (is.na(psi_name)) {
    message("qgcomp terms for [", model_label, "]: no psi found in ", paste(names(cf), collapse = ", "))
    return(tibble(model = model_label, beta = NA_real_, conf.low = NA_real_, conf.high = NA_real_, p.val
  }
  beta <- unname(cf[psi_name])
  vc <- tryCatch(vcov(fit_obj), error = function(e) NULL)
  if (!is.null(vc) && psi_name %in% rownames(vc) && psi_name %in% colnames(vc)) {
    se <- sqrt(vc[psi_name, psi_name])
    lo <- beta - 1.96 * se
    hi <- beta + 1.96 * se
    p_val <- 2 * pnorm(-abs(beta / se))
  } else {
```

```r
    lo <- hi <- p_val <- NA_real_
  }
  tibble(model = model_label, beta = beta, conf.low = lo, conf.high = hi, p.value = p_val, N = n_obs)
}

resolve_weight_var <- function(df, weight_type = "auto", exposure_family = "pfas") {
  cols <- names(df)
  if (weight_type != "auto" && weight_type != "MEC") {
    if (!(weight_type %in% cols)) stop("Requested weight variable not found: ", weight_type)
    return(list(var = weight_type, note = paste("User-specified weight:", weight_type)))
  }
  if ("WTMEC2YR" %in% cols) return(list(var = "WTMEC2YR", note = "Generic fallback: using MEC weight WT
  stop("No supported NHANES weight variable found.")
}

apply_lod_rule <- function(df, varname) {
  x <- suppressWarnings(as.numeric(as.character(df[[varname]])))

  base <- sub("^(LBX|URX)", "", varname)
  lc_candidates <- c(
    paste0(varname, "LC"),
    paste0("LBD", base, "LC"),
    paste0("URD", base, "LC"),
    # NHANES often uses *L (not *LC) as the detection-limit flag.
    paste0(varname, "L"),
    paste0("LBD", base, "L"),
    paste0("URD", base, "L")
  )
  lc_candidates <- lc_candidates[lc_candidates %in% names(df)]

  if (length(lc_candidates) > 0) {
    lc <- df[[lc_candidates[1]]]
    lc_num <- suppressWarnings(as.numeric(as.character(lc)))
    lc_chr <- toupper(trimws(as.character(lc)))
    below <- (!is.na(lc_num) & lc_num == 1) | str_detect(lc_chr, "BELOW|<")
    below[is.na(below)] <- FALSE
    # NHANES convention: when LC==1, the concentration column typically holds the LOD value.
    # Standard replacement: value / sqrt(2).
    x[below] <- x[below] / sqrt(2)
    # If value is still NA/non-positive after that, use smallest observed LOD / sqrt(2).
    lod_vals <- x[below & x > 0]
    fallback <- if (length(lod_vals) > 0) min(lod_vals, na.rm = TRUE) else NA_real_
    x[below & (is.na(x) | x <= 0)] <- fallback
    return(x)
  }
  # If no LOD indicator exists, do not impute missing values.
  x
}
```

Data Acquisition and Cycle Availability

```r
# Cycle-aware: 2003-2004 (C) uses Lxx_ names (L40, L25, L06CRP, L06ALB, L24PFC, L06BMT, L06UHM); 2005+ 
table_candidates <- list(
  DEMO = c("DEMO"),
  BMX = c("BMX"),
  SMQ = c("SMQ"),
  ALQ = c("ALQ"),
  PAQ = c("PAQ"),
  DIQ = c("DIQ"),
  BPQ = c("BPQ"),
  MCQ = c("MCQ"),
  BIOPRO = c("L40", "BIOPRO"),
  CBC = c("L25", "CBC"),
  CRP = c("L06CRP", "CRP", "HSCRP"),
  ALB_CR = c("L06ALB", "ALB_CR"),
  GLU = c("L10", "GLU"),
  PFAS = c("L24PFC", "PFAS", "PFC", "P_PFAS"),
  BLOOD_METALS = c("L06BMT", "PBCD"),
  URINE_METALS = c("L06UHM", "UHM", "UM")
)

download_cycle <- function(cycle, exposure_family = "pfas", verbose = TRUE) {
  core_domains <- c("DEMO", "BMX", "SMQ", "ALQ", "PAQ", "DIQ", "BPQ", "MCQ", "BIOPRO", "CBC", "CRP", "A
  exp_domains <- switch(
    exposure_family,
    pfas = c("PFAS"),
    metals = c("BLOOD_METALS", "URINE_METALS"),
    both = c("PFAS", "BLOOD_METALS", "URINE_METALS"),
    c("PFAS")
  )
  all_domains <- c(core_domains, exp_domains)

  resolved <- lapply(all_domains, function(dom) resolve_table(cycle, table_candidates[[dom]], domain = 
  names(resolved) <- all_domains
  resolved <- resolved[!vapply(resolved, is.null, logical(1))]
  if (!"DEMO" %in% names(resolved)) return(NULL)

  dat <- resolved$DEMO$data
  if (!"SEQN" %in% names(dat)) return(NULL)

  for (nm in setdiff(names(resolved), "DEMO")) {
    d <- resolved[[nm]]$data
    # Rename glucose vars from GLU table before joining so they do not collide with BIOPRO/L40.
    if (nm == "GLU") {
      if ("LBXGLU" %in% names(d)) names(d)[names(d) == "LBXGLU"] <- "LBXGLU_GLU"
      if ("LBXSGL" %in% names(d)) names(d)[names(d) == "LBXSGL"] <- "LBXSGL_GLU"
    }
    if ("SEQN" %in% names(d)) dat <- left_join(dat, d, by = "SEQN")
  }
  # After merge: define explicit glucose sources. Primary analysis uses MEC chemistry glucose for measu
  dat$glu_mec  <- coalesce_num(dat, c("LBXGLU", "LBXSGL"))
  dat$glu_glu  <- coalesce_num(dat, c("LBXGLU_GLU", "LBXSGL_GLU"))
  dat$glu     <- dat$glu_mec
  dat$cycle <- cycle
```

```r
  # Store provenance for key tables so we can audit weight/source compatibility.
  if ("CRP" %in% names(resolved)) dat$crp_table <- resolved$CRP$table
  if ("GLU" %in% names(resolved)) dat$glu_table <- resolved$GLU$table
  # Warn if joins created .x/.y duplicate columns (would break coalesce_num).
  dup_suffix_cols <- names(dat)[grepl("\\.x$|\\.y$", names(dat))]
  if (length(dup_suffix_cols) > 0) {
    warning("Cycle ", cycle, ": duplicate columns from join: ", paste(dup_suffix_cols, collapse = ", ")
  }
  dat
}

cycle_data <- lapply(config$cycles, download_cycle, exposure_family = config$exposure_family, verbose =
cycle_data <- cycle_data[!vapply(cycle_data, is.null, logical(1))]
if (length(cycle_data) == 0) stop("No cycles were successfully downloaded.")

all_cols <- sort(unique(unlist(lapply(cycle_data, names))))
cycle_data <- lapply(cycle_data, function(df) {
  miss <- setdiff(all_cols, names(df))
  if (length(miss) > 0) for (m in miss) df[[m]] <- NA
  df <- df[, all_cols, drop = FALSE]
  df %>% mutate(across(everything(), as.character))
})
nhanes_raw <- bind_rows(cycle_data)

kable(
  data.frame(
    cycle = names(table(config$cycles))[names(table(config$cycles)) %in% unique(nhanes_raw$cycle)],
    n = as.integer(table(nhanes_raw$cycle))
  ),
  caption = "Cycle availability and downloaded participant counts"
)
```

Table 1: Cycle availability and downloaded participant counts

cycle	n
2005-2006	10348
2007-2008	10149
2009-2010	10537
2011-2012	9756
2013-2014	10175
2015-2016	9971
2017-2018	9254

Variable Construction: PhenoAge, CKD, Covariates

```r
# Defensive guard: ensure cycle-dependent NHANES columns exist before referencing them
# in the mutate below. cycle_data harmonization (Data Acquisition section) already pads
# cycles to the union of column names, so these will normally be present -- but if the
# user reconfigures `config$cycles` to drop certain ranges, some columns can vanish from
# the union (e.g., ALQ142 only exists in 2017+; PAD200/PAD320 only exist in 1999-2006).
# This block guarantees they exist as NA so the harmonized derivations below still run.
```

```r
required_optional_cols <- c(
  # Smoking
  "SMQ020", "SMQ040",
  # Alcohol (cycle-dependent question wording -- see parser comments above)
  "ALQ101", "ALQ110", "ALQ111", "ALQ120Q", "ALQ121", "ALQ142", "ALQ151",
  # Physical activity (GPAQ in 2007+, legacy module in 1999-2006)
  "PAD200", "PAD320", "PAQ650", "PAQ665"
)
for (.col in required_optional_cols) {
  if (!.col %in% names(nhanes_raw)) nhanes_raw[[.col]] <- NA_character_
}
rm(.col)

dpq_vars <- names(nhanes_raw)[str_detect(names(nhanes_raw), "^DPQ0")]

dat <- nhanes_raw %>%
  mutate(across(any_of(dpq_vars), parse_phq_item)) %>%
  mutate(
    age = suppressWarnings(as.numeric(as.character(RIDAGEYR))),
    sex = case_when(
      suppressWarnings(as.numeric(as.character(RIAGENDR))) == 1 ~ "Male",
      suppressWarnings(as.numeric(as.character(RIAGENDR))) == 2 ~ "Female",
      toupper(trimws(as.character(RIAGENDR))) == "MALE" ~ "Male",
      toupper(trimws(as.character(RIAGENDR))) == "FEMALE" ~ "Female",
      TRUE ~ NA_character_
    ),
    race_eth = {
      r3 <- suppressWarnings(as.numeric(as.character(RIDRETH3)))
      r1 <- suppressWarnings(as.numeric(as.character(RIDRETH1)))
      r3_chr <- toupper(trimws(as.character(RIDRETH3)))
      r1_chr <- toupper(trimws(as.character(RIDRETH1)))
      case_when(
        !is.na(r3) & r3 %in% c(1, 2) ~ "Hispanic",
        !is.na(r3) & r3 == 3 ~ "Non-Hispanic White",
        !is.na(r3) & r3 == 4 ~ "Non-Hispanic Black",
        !is.na(r3) & r3 == 6 ~ "Non-Hispanic Asian",
        !is.na(r3) & r3 %in% c(5, 7) ~ "Other/Multi",
        is.na(r3) & r1 %in% c(1, 2) ~ "Hispanic",
        is.na(r3) & r1 == 3 ~ "Non-Hispanic White",
        is.na(r3) & r1 == 4 ~ "Non-Hispanic Black",
        is.na(r3) & r1 == 5 ~ "Other/Multi",
        str_detect(r3_chr, "NON-HISPANIC WHITE") ~ "Non-Hispanic White",
        str_detect(r3_chr, "NON-HISPANIC BLACK") ~ "Non-Hispanic Black",
        str_detect(r3_chr, "NON-HISPANIC ASIAN") ~ "Non-Hispanic Asian",
        str_detect(r3_chr, "OTHER RACE|MULTI") ~ "Other/Multi",
        str_detect(r3_chr, "^MEXICAN AMERICAN$|^OTHER HISPANIC$|^HISPANIC$") ~ "Hispanic",
        str_detect(r1_chr, "NON-HISPANIC WHITE") ~ "Non-Hispanic White",
        str_detect(r1_chr, "NON-HISPANIC BLACK") ~ "Non-Hispanic Black",
        str_detect(r1_chr, "OTHER RACE|MULTI") ~ "Other/Multi",
        str_detect(r1_chr, "^MEXICAN AMERICAN$|^OTHER HISPANIC$|^HISPANIC$") ~ "Hispanic",
        TRUE ~ NA_character_
      )
    },
```

```r
education = {
  e_num <- suppressWarnings(as.numeric(as.character(DMDEDUC2)))
  e_chr <- toupper(trimws(as.character(DMDEDUC2)))
  case_when(
    e_num %in% c(1, 2) ~ "<HS",
    e_num == 3 ~ "HS/GED",
    e_num %in% c(4, 5) ~ ">HS",
    str_detect(e_chr, "LESS THAN 9TH|9TH.*12TH") ~ "<HS",
    str_detect(e_chr, "HIGH SCHOOL|GED") ~ "HS/GED",
    str_detect(e_chr, "SOME COLLEGE|COLLEGE GRADUATE") ~ ">HS",
    TRUE ~ NA_character_
  )
},
pir = suppressWarnings(as.numeric(as.character(INDFMPIR))),
bmi = suppressWarnings(as.numeric(as.character(BMXBMI))),
# ---- Categorical-variable derivations ---------------------------------
# Smoking: NHANES uses a 2-stage instrument. SMQ020 ("Ever smoked >=100 cigarettes
# in life?") is the GATE question; SMQ040 ("Do you now smoke?") is asked only of
# SMQ020==Yes respondents. Therefore an SMQ040-only derivation silently drops every
# never-smoker (SMQ020==No) to NA and excludes them from any complete-case modeling
# sample. Below we derive the standard 3-level smoker3 (Never/Former/Current) using
# SMQ020 + SMQ040, and a true binary smoker (Current vs. Not-current) that retains
# never-smokers as 0. This binary is what downstream models continue to use.
smoker3 = {
  ever <- parse_smq020(SMQ020)
  smq040_chr <- toupper(trimws(as.character(SMQ040)))
  smq040_num <- suppressWarnings(as.numeric(as.character(SMQ040)))
  curr <- case_when(
    smq040_num %in% c(1, 2) ~ 1,
    smq040_num == 3 ~ 0,
    str_detect(smq040_chr, "EVERY DAY|SOME DAYS") ~ 1,
    str_detect(smq040_chr, "^NOT AT ALL$") ~ 0,
    TRUE ~ NA_real_
  )
  case_when(
    ever == 0 ~ "Never",
    ever == 1 & curr == 1 ~ "Current",
    ever == 1 & curr == 0 ~ "Former",
    TRUE ~ NA_character_
  )
},
smoker = case_when(
  smoker3 == "Current" ~ 1,
  smoker3 %in% c("Never", "Former") ~ 0,
  TRUE ~ NA_real_
),
# Alcohol: harmonized 3-level Never/Former/Current drinker, parallel to smoker3.
# The previous derivation used ALQ151 ("ever heavy drinker") only, which (a) silently
# dropped never-drinkers (skip pattern), (b) returned NA for the entire 2017-2018 (J)
# cycle (variable discontinued), and (c) measured a different construct than the
# NHANES-literature standard. The new derivation uses the cycle-harmonized helpers:
#   parse_alq_ever()        : lifetime ever-drinker from ALQ101 / ALQ110 / ALQ111
#   parse_alq_current_yr()  : drank in past 12 mo from ALQ121 (2013+) or ALQ120Q (1999-2016)
```

```r
    # Never-drinkers are now coded as "Never" (instead of NA), and J-cycle participants
    # get a defined value via ALQ111 + ALQ121.
    alcohol3 = {
      ever <- parse_alq_ever(ALQ101, ALQ110, ALQ111)
      curr <- parse_alq_current_yr(ALQ121, ALQ120Q)
      case_when(
        ever == 0 ~ "Never",
        ever == 1 & curr == 1 ~ "Current",
        ever == 1 & curr == 0 ~ "Former",
        TRUE ~ NA_character_
      )
    },
    # Binary alcohol covariate (used by downstream models): "Current drinker" vs.
    # "Not current drinker" (combines never + former). This matches the NHANES literature
    # convention and is far less sparse than the previous ALQ151-based binary.
    alcohol = case_when(
      alcohol3 == "Current" ~ 1,
      alcohol3 %in% c("Never", "Former") ~ 0,
      TRUE ~ NA_real_
    ),
    # Physical activity: harmonized across the 2006/2007 GPAQ transition.
    #   PAQ650 (vigorous recreational, 2007+) OR PAQ665 (moderate recreational, 2007+)
    #   PAD200 (vigorous past 30 days, 1999-2006) OR PAD320 (moderate past 30 days, 1999-2006)
    # Either questionnaire format counts as "physically active" for the harmonized binary;
    # otherwise the participant is asked one set or the other depending on cycle.
    phys_active = case_when(
      parse_yes_no(PAQ650) == 1 | parse_yes_no(PAQ665) == 1 ~ 1,
      parse_yes_no(PAQ650) == 0 & parse_yes_no(PAQ665) == 0 ~ 0,
      parse_yes_no(PAD200) == 1 | parse_yes_no(PAD320) == 1 ~ 1,
      parse_yes_no(PAD200) == 0 & parse_yes_no(PAD320) == 0 ~ 0,
      TRUE ~ NA_real_
    ),
    # Diabetes: DIQ010 ("Doctor told you have diabetes?") codes 1=Yes, 2=No, 3=Borderline,
    # 7/9=Refused/DK. yesno01() maps 1->1, 2->0, and ALL others (including Borderline) to NA.
    # This drops borderline-diabetics from the analytic sample. If PI prefers, borderline
    # can be coded as 1 (any diabetes) or 0 (definite diabetes only).
    diabetes = yesno01(DIQ010),
    # Hypertension: BPQ020 ("Doctor told you have HBP?") asked of all adults; no gate.
    hypertension = yesno01(BPQ020),
    # CVD composite: ANY of CHF (MCQ160B), CHD (MCQ160C), heart attack (MCQ160E), stroke
    # (MCQ160F). Not asked behind a gate; respondent NA in all four -> overall NA.
    cvd = case_when(
      parse_yes_no(MCQ160B) == 1 | parse_yes_no(MCQ160C) == 1 | parse_yes_no(MCQ160E) == 1 | parse_yes_
      parse_yes_no(MCQ160B) == 0 & parse_yes_no(MCQ160C) == 0 & parse_yes_no(MCQ160E) == 0 & parse_yes_
      TRUE ~ NA_real_
    ),
    phenoage_accel = NA_real_
  )

# Compute biomarkers, CKD markers, and design variables outside mutate (avoids cur_data_all()).
dat$alb <- coalesce_num(dat, c("LBXSAL"))
dat$scr <- coalesce_num(dat, c("LBXSCR", "LBDSCRSI"))
# Glucose: glu_mec / glu_glu / glu set in download_cycle (primary uses glu_mec by design).
```

```r
if (!"glu" %in% names(dat)) dat$glu <- dat$glu_mec
dat$crp <- coalesce_num(dat, c("LBXCRP", "LBXHSCRP"))
dat$lym <- coalesce_num(dat, c("LBXLYPCT"))
dat$mcv <- coalesce_num(dat, c("LBXMCV", "LBXMCVSI"))
dat$rdw <- coalesce_num(dat, c("LBXRW", "LBXRDW"))
dat$alp <- coalesce_num(dat, c("LBXAPSI", "LBXSAPSI"))
dat$wbc <- coalesce_num(dat, c("LBXWBCSI", "LBXWBC"))

dat$ualb <- coalesce_num(dat, c("URXUMA"))
dat$ucr <- coalesce_num(dat, c("URXUCR", "URDUCR"))
dat$acr_direct <- coalesce_num(dat, c("URDACT"))
dat$acr <- compute_acr(dat$ualb, dat$ucr, dat$acr_direct)
dat$egfr <- compute_egfr_2021(dat$scr, dat$age, dat$RIAGENDR)
dat$ckd_egfr <- ifelse(!is.na(dat$egfr), as.integer(dat$egfr < 60), NA_integer_)
dat$ckd_acr <- ifelse(!is.na(dat$acr), as.integer(dat$acr >= 30), NA_integer_)
dat$ckd <- case_when(
  config$ckd_def == "egfr_or_acr" & (dat$ckd_egfr == 1 | dat$ckd_acr == 1) ~ 1L,
  config$ckd_def == "egfr_or_acr" & (!is.na(dat$ckd_egfr) | !is.na(dat$ckd_acr)) ~ 0L,
  config$ckd_def == "egfr_only" ~ dat$ckd_egfr,
  config$ckd_def == "acr_only" ~ dat$ckd_acr,
  TRUE ~ NA_integer_
)
dat$ckd_stage <- cut(
  dat$egfr, breaks = c(-Inf, 15, 30, 45, 60, 90, Inf),
  labels = c("G5", "G4", "G3b", "G3a", "G2", "G1"),
  right = FALSE  # KDIGO: left-closed intervals [lo, hi). E.g., 60 -> G2, 59.9 -> G3a.
)
dat$strata <- coalesce_num(dat, c("SDMVSTRA"))
dat$psu <- coalesce_num(dat, c("SDMVPSU"))

# Coding audits (raw -> derived) to prevent silent recode failures.
kable(
  dat %>% count(cycle, ridreth3_missing = is.na(RIDRETH3)) %>% arrange(cycle, ridreth3_missing),
  caption = "Race audit: availability of RIDRETH3 by cycle"
)
```

Table 2: Race audit: availability of RIDRETH3 by cycle

cycle	ridreth3_missing	n
2005-2006	TRUE	10348
2007-2008	TRUE	10149
2009-2010	TRUE	10537
2011-2012	FALSE	9756
2013-2014	FALSE	10175
2015-2016	FALSE	9971
2017-2018	FALSE	9254

```r
kable(as.data.frame(table(dat$RIDRETH1, useNA = "ifany")), caption = "Raw RIDRETH1 distribution")
```

Table 3: Raw RIDRETH1 distribution

Var1	Freq
Mexican American	13761
Non-Hispanic Black	16072
Non-Hispanic White	25326
Other Hispanic	6847
Other Race - Including Multi-Racial	8184

```
kable(as.data.frame(table(dat$RIDRETH3, useNA = "ifany")), caption = "Raw RIDRETH3 distribution")
```

Table 4: Raw RIDRETH3 distribution

Var1	Freq
Mexican American	6373
Non-Hispanic Asian	4566
Non-Hispanic Black	9194
Non-Hispanic White	12863
Other Hispanic	4164
Other Race - Including Multi-Racial	1996
NA	31034

```
kable(as.data.frame(table(dat$race_eth, useNA = "ifany")), caption = "Derived race_eth distribution")
```

Table 5: Derived race_eth distribution

Var1	Freq
Hispanic	20608
Non-Hispanic Asian	4566
Non-Hispanic Black	16072
Non-Hispanic White	25326
Other/Multi	3618

```
kable(as.data.frame(table(dat$SMQ040, useNA = "ifany")), caption = "Raw SMQ040 distribution")
```

Table 6: Raw SMQ040 distribution

Var1	Freq
Every day	6659
Not at all	9527
Refused	3
Some days	1556
NA	52445

```
kable(as.data.frame(table(dat$DIQ010, useNA = "ifany")), caption = "Raw DIQ010 distribution")
```

Table 7: Raw DIQ010 distribution

Var1	Freq
Borderline	984
Don't know	41
No	60942
Refused	3
Yes	5231
NA	2989

```
kable(as.data.frame(table(dat$BPQ020, useNA = "ifany")), caption = "Raw BPQ020 distribution")
```

Table 8: Raw BPQ020 distribution

Var1	Freq
Don't know	65
No	30215
Yes	14420
NA	25490

```
kable(as.data.frame(table(dat$smoker, useNA = "ifany")), caption = "Derived smoker distribution")
```

Table 9: Derived smoker distribution

Var1	Freq
0	32403
1	8215
NA	29572

```
kable(as.data.frame(table(dat$diabetes, useNA = "ifany")), caption = "Derived diabetes distribution")
```

Table 10: Derived diabetes distribution

Var1	Freq
0	60942
1	5231
NA	4017

```
kable(as.data.frame(table(dat$hypertension, useNA = "ifany")), caption = "Derived hypertension distribu
```

Table 11: Derived hypertension distribution

Var1	Freq
0	30215
1	14420
NA	25555

```r
# Codebook-aligned harmonization checks (raw coding support by cycle).
check_raw_levels <- function(df, var, allowed_patterns) {
  if (!(var %in% names(df))) return(tibble(variable = var, cycle = NA_character_, n_unexpected = NA_int
  df %>%
    mutate(.raw = toupper(trimws(as.character(.data[[var]])))) %>%
    group_by(cycle) %>%
    summarise(
      n_nonmissing = sum(!is.na(.raw) & .raw != ""),
      n_unexpected = sum(!is.na(.raw) & .raw != "" & !str_detect(.raw, allowed_patterns)),
      unexpected_examples = paste(head(unique(.raw[!is.na(.raw) & .raw != "" & !str_detect(.raw, allowe
      .groups = "drop"
    ) %>%
    mutate(variable = var) %>%
    select(variable, cycle, n_nonmissing, n_unexpected, unexpected_examples)
}

harmonization_checks <- bind_rows(
  check_raw_levels(dat, "RIAGENDR", "^(1|2|MALE|FEMALE)$"),
  check_raw_levels(dat, "DMDEDUC2", "^(1|2|3|4|5|7|9)$|LESS THAN 9TH|9-11TH|HIGH SCHOOL|GED|SOME COLLEG
  check_raw_levels(dat, "SMQ040", "^(1|2|3|7|9)$|EVERY DAY|SOME DAYS|NOT AT ALL|REFUSED|DON'T KNOW|DONT
  check_raw_levels(dat, "DIQ010", "^(1|2|3|7|9)$|YES|NO|BORDERLINE|REFUSED|DON'T KNOW|DONT KNOW"),
  check_raw_levels(dat, "BPQ020", "^(1|2|7|9)$|YES|NO|REFUSED|DON'T KNOW|DONT KNOW")
)
kable(harmonization_checks, caption = "Codebook-aligned raw coding checks by cycle. Non-zero unexpected
```

Table 12: Codebook-aligned raw coding checks by cycle. Non-zero unexpected counts indicate coding values not covered by harmonization rules.

variable	cycle	n_nonmissing	n_unexpected	unexpected_examples
RIAGENDR	2005-2006	10348	0	
RIAGENDR	2007-2008	10149	0	
RIAGENDR	2009-2010	10537	0	
RIAGENDR	2011-2012	9756	0	
RIAGENDR	2013-2014	10175	0	
RIAGENDR	2015-2016	9971	0	
RIAGENDR	2017-2018	9254	0	
DMDEDUC2	2005-2006	4979	0	
DMDEDUC2	2007-2008	5935	0	
DMDEDUC2	2009-2010	6218	0	
DMDEDUC2	2011-2012	5560	0	
DMDEDUC2	2013-2014	5769	0	
DMDEDUC2	2015-2016	5719	0	
DMDEDUC2	2017-2018	5569	0	
SMQ040	2005-2006	2348	0	
SMQ040	2007-2008	2802	0	
SMQ040	2009-2010	2866	0	
SMQ040	2011-2012	2369	0	
SMQ040	2013-2014	2579	0	
SMQ040	2015-2016	2422	0	
SMQ040	2017-2018	2359	0	
DIQ010	2005-2006	9822	0	
DIQ010	2007-2008	9666	0	

variable	cycle	n_nonmissing	n_unexpected	unexpected_examples
DIQ010	2009-2010	10109	0	
DIQ010	2011-2012	9363	0	
DIQ010	2013-2014	9769	0	
DIQ010	2015-2016	9575	0	
DIQ010	2017-2018	8897	0	
BPQ020	2005-2006	6139	0	
BPQ020	2007-2008	6546	0	
BPQ020	2009-2010	6889	0	
BPQ020	2011-2012	6174	0	
BPQ020	2013-2014	6464	0	
BPQ020	2015-2016	6327	0	
BPQ020	2017-2018	6161	0	

```r
# --- CRP unit detection and conversion ---
# PhenoAge (Levine 2018) uses CRP in mg/dL. NHANES LBXCRP is typically mg/dL;
# LBXHSCRP or CRP tables in some cycles may report mg/L.
# Step 1: check variable/table provenance.
crp_by_cycle <- dat %>%
  group_by(cycle) %>%
  summarise(
    source_table = if ("crp_table" %in% names(pick(everything()))) paste(unique(na.omit(crp_table)), co
    n_obs = sum(!is.na(crp)),
    median_crp = median(crp, na.rm = TRUE),
    p75 = quantile(crp, 0.75, na.rm = TRUE),
    p90 = quantile(crp, 0.90, na.rm = TRUE),
    .groups = "drop"
  )
kable(crp_by_cycle, caption = "CRP diagnostic by cycle -- verify units (PhenoAge expects mg/dL). If sou
```

Table 13: CRP diagnostic by cycle – verify units (PhenoAge expects mg/dL). If source table is HSCRP, values may be mg/L.

cycle	source_table	n_obs	median_crp	p75	p90
2005-2006	CRP_D	8172	0.120	0.38	0.890
2007-2008	CRP_E	6809	0.160	0.40	0.920
2009-2010	CRP_F	8299	0.130	0.35	0.830
2011-2012		0	NA	NA	NA
2013-2014		0	NA	NA	NA
2015-2016	HSCRP_I	7867	1.200	3.60	8.200
2017-2018	HSCRP_J	7250	1.355	3.59	7.462

```r
# Step 2: provenance-based detection (strongest signal).
crp_tables_used <- if ("crp_table" %in% names(dat)) unique(na.omit(dat$crp_table)) else character(0)
provenance_suggests_mgl <- length(crp_tables_used) > 0 && any(str_detect(crp_tables_used, regex("HSCRP"
# Step 3: distribution-based backup -- require median > 1.5 AND p75 > 3 to trigger conversion.
crp_overall_med <- median(dat$crp, na.rm = TRUE)
crp_overall_p75 <- quantile(dat$crp, 0.75, na.rm = TRUE)
distribution_suggests_mgl <- !is.na(crp_overall_med) && crp_overall_med > 1.5 &&
                             !is.na(crp_overall_p75) && crp_overall_p75 > 3
```

```r
if (provenance_suggests_mgl || distribution_suggests_mgl) {
  reason <- if (provenance_suggests_mgl) {
    paste0("CRP source table(s) [", paste(crp_tables_used, collapse = ", "), "] suggest mg/L.")
  } else {
    paste0("Distribution heuristic: median=", round(crp_overall_med, 3), ", p75=", round(crp_overall_p75
  }
  message(">>> Converting CRP from mg/L to mg/dL (/ 10). Reason: ", reason)
  dat$crp <- dat$crp / 10
} else {
  message(">>> CRP assumed to be in mg/dL (median=", round(crp_overall_med, 4),
          ", p75=", round(crp_overall_p75, 4), "). Verify against NHANES documentation.")
}

# --- Glucose provenance diagnostic ---
# Coerce glu to numeric (nhanes_raw has character columns from bind_rows); needed for summarise and lat
dat$glu <- suppressWarnings(as.numeric(as.character(dat$glu)))
glu_by_cycle <- dat %>%
  group_by(cycle) %>%
  summarise(
    source_table = if ("glu_table" %in% names(pick(everything()))) paste(unique(na.omit(glu_table)), co
    n_glu = sum(!is.na(glu)),
    median_glu = median(glu, na.rm = TRUE),
    min_glu = min(glu, na.rm = TRUE),
    max_glu = max(glu, na.rm = TRUE),
    .groups = "drop"
  )
kable(glu_by_cycle, caption = "Glucose provenance by cycle. If source_table is a fasting component (GLU_
```

Table 14: Glucose provenance by cycle. If source_table is a fasting component (GLU_*), use a compatible fasting subsample weight.

cycle	source_table	n_glu	median_glu	min_glu	max_glu
2005-2006	GLU_D	6434	90	42	513
2007-2008	GLU_E	6377	91	33	559
2009-2010	GLU_F	6860	91	34	777
2011-2012	GLU_G	5976	91	31	617
2013-2014	GLU_H	6553	93	49	577
2015-2016	GLU_I	6257	92	19	610
2017-2018	GLU_J	5901	92	47	626

```r
glu_med <- median(dat$glu, na.rm = TRUE)
if (!is.na(glu_med) && glu_med < 6) {
  warning("Glucose median is < 6 -- this looks like mmol/L. Converting to mg/dL ( x 18.018).")
  dat$glu <- dat$glu * 18.018
}

# Warn based on stored table provenance, not just weight variable existence.
glu_tables_used <- if ("glu_table" %in% names(dat)) unique(na.omit(dat$glu_table)) else character(0)
glu_is_fasting <- length(glu_tables_used) > 0 && any(str_detect(glu_tables_used, regex("^GLU_", ignore_
if (glu_is_fasting) {
  message(">>> IMPORTANT: Glucose sourced from fasting component table(s) [",
          paste(glu_tables_used, collapse = ", "),
```

```r
          "]. A compatible fasting subsample weight (e.g., WTSAF2YR) is required. ",
          "Using a non-fasting or otherwise incompatible weight can bias estimates. ",
          "Review the weight audit; override config$weight_type only if auto-selection does not resolve
} else if (length(glu_tables_used) > 0) {
  message(">>> Glucose source table(s): ", paste(glu_tables_used, collapse = ", "),
          ". Verify whether this is a fasting component.")
}

# ------------------------------------------------------------
# CLEAN SPECIAL NHANES MISSING CODES FOR LAB VARIABLES
# ------------------------------------------------------------
clean_nhanes_numeric <- function(x) {
  x <- suppressWarnings(as.numeric(as.character(x)))
  special_codes <- c(
    7, 77, 777, 7777, 77777,
    9, 99, 999, 9999, 99999,
    888, 8888, 88888
  )
  x[x %in% special_codes] <- NA_real_
  x
}

dat$alb <- clean_nhanes_numeric(dat$alb)
dat$scr <- clean_nhanes_numeric(dat$scr)
dat$glu <- clean_nhanes_numeric(dat$glu)
dat$crp <- clean_nhanes_numeric(dat$crp)
dat$lym <- clean_nhanes_numeric(dat$lym)
dat$mcv <- clean_nhanes_numeric(dat$mcv)
dat$rdw <- clean_nhanes_numeric(dat$rdw)
dat$alp <- clean_nhanes_numeric(dat$alp)
dat$wbc <- clean_nhanes_numeric(dat$wbc)

# ------------------------------------------------------------
# HARD PLAUSIBILITY CHECK (retain CRP >1 mg/dL in primary; flag as acute)
# ------------------------------------------------------------
dat <- dat %>%
  mutate(
    crp_acute = ifelse(!is.na(crp) & crp > 1, 1L, 0L),
    wbc = ifelse(wbc > 50, NA_real_, wbc),
    glu = ifelse(glu > 600, NA_real_, glu),
    alp = ifelse(alp > 500, NA_real_, alp)
  )

# --- General biomarker plausibility ---
biomarker_ranges <- list(
  alb  = c(1, 7,    "g/dL"),
  scr  = c(0.1, 25, "mg/dL"),
  glu  = c(20, 600,  "mg/dL"),
  crp  = c(0.001, 3, "mg/dL (post-conversion if applied)"),
  lym  = c(1, 80,    "%"),
  mcv  = c(50, 150,  "fL"),
  rdw  = c(8, 30,    "%"),
  alp  = c(5, 500,   "U/L"),
```

```r
  wbc  = c(1, 50,    "1000 cells/uL")
)
biomarker_check <- lapply(names(biomarker_ranges), function(nm) {
  vals <- dat[[nm]]
  rng  <- biomarker_ranges[[nm]]
  lo   <- as.numeric(rng[1]); hi <- as.numeric(rng[2]); unit <- rng[3]
  v    <- vals[!is.na(vals)]
  n_out <- sum(v < lo | v > hi)
  tibble(biomarker = nm, unit = unit, n_obs = length(v),
         min = if (length(v) > 0) min(v) else NA_real_,
         p25 = if (length(v) > 0) quantile(v, 0.25) else NA_real_,
         median = if (length(v) > 0) median(v) else NA_real_,
         p75 = if (length(v) > 0) quantile(v, 0.75) else NA_real_,
         max = if (length(v) > 0) max(v) else NA_real_,
         expected_lo = lo, expected_hi = hi,
         n_out_of_range = n_out)
}) %>% bind_rows()
kable(biomarker_check, caption = "PhenoAge biomarker ranges after unit conversion (verify units)")
```

Table 15: PhenoAge biomarker ranges after unit conversion (verify units)

biomarker	unit	n_obs	min	p25	median	p75	max	expected_lo	expected_hi	n_out_of_range
alb	g/dL	44361	1.200	4.000	4.300	4.500	5.60	1e+00	7	0
scr	mg/dL	44357	0.140	0.690	0.820	0.990	17.80	1e-01	25	0
glu	mg/dL	42903	19.000	85.000	92.000	102.000	577.00	2e+01	600	1
crp	mg/dL (post-conversion if applied)	38397	0.001	0.008	0.032	0.116	18.85	1e-03	3	197
lym	%	57511	2.600	26.500	32.700	40.000	94.50	1e+00	80	16
mcv	fL	57529	35.400	83.400	87.600	91.300	125.30	5e+01	150	1
rdw	%	57608	6.300	12.400	13.000	13.700	37.80	8e+00	30	7
alp	U/L	43395	10.000	56.000	70.000	89.000	500.00	5e+00	500	0
wbc	1000 cells/uL	56369	1.400	5.700	6.900	8.400	44.80	1e+00	50	0

```r
for (nm in names(biomarker_ranges)) {
  v <- dat[[nm]][!is.na(dat[[nm]])]
  lo <- as.numeric(biomarker_ranges[[nm]][1])
  hi <- as.numeric(biomarker_ranges[[nm]][2])
  pct_out <- if (length(v) > 0) mean(v < lo | v > hi) else 0
  if (pct_out > 0.20) warning(sprintf(
    "Biomarker '%s': %.1f%% of values outside plausible range [%s, %s]. Check units.",
    nm, pct_out * 100, lo, hi
  ))
}

# Recompute PhenoAge after any unit conversions (CRP mg/L->mg/dL, glucose mmol/L->mg/dL).
pheno_core <- compute_phenoage_components(dat, include_scr = TRUE, include_alb = TRUE)
dat$phenoage <- pheno_core$phenoage

message(
  "PhenoAge diagnostics -- xb median=", round(stats::median(pheno_core$xb, na.rm = TRUE), 3),
```

```r
  ", mort median=", format(stats::median(pheno_core$mort, na.rm = TRUE), scientific = TRUE, digits = 3)
  ", pct(mort>0.9999)=", round(mean(pheno_core$mort > 0.9999, na.rm = TRUE) * 100, 1), "%",
  ", PhenoAge SD=", round(stats::sd(dat$phenoage, na.rm = TRUE), 3)
)
ph_q <- stats::quantile(dat$phenoage, probs = c(0, 0.25, 0.5, 0.75, 1), na.rm = TRUE)
message(
  "PhenoAge five-number summary -- min=", round(ph_q[[1]], 3),
  ", p25=", round(ph_q[[2]], 3),
  ", median=", round(ph_q[[3]], 3),
  ", p75=", round(ph_q[[4]], 3),
  ", max=", round(ph_q[[5]], 3)
)
if (mean(pheno_core$mort > 0.9999, na.rm = TRUE) > 0.25) {
  warning("More than 25% of participants have mort > 0.9999 after stabilization. Re-check biomarker uni
}
```

Exposure Processing and LOD Handling

```r
pfas_vars <- c("LBXPFOS", "LBXPFOA", "LBXPFNA", "LBXPFHS", "LBXPFDA")

# --- ISOMER HARMONIZATION (2013-2018) ---
# NHANES split PFOS and PFOA into linear and branched isomers.
# We must apply the LOD/sqrt(2) rule to the individual isomers FIRST, then sum them.

isomer_map <- list(
  pfos_lin = c("SSNPFOS", "LBXNFOS"),
  pfos_br  = c("SSMPFOS", "LBXMFOS"),
  pfoa_lin = c("SSNPFOA", "LBXNFOA"),
  pfoa_br  = c("SSBPFOA", "LBXBFOA")
)

lc_map <- list(
  pfos_lin = c("SDNPFOSL", "LBDNFOSL"),
  pfos_br  = c("SDMPFOSL", "LBDMFOSL"),
  pfoa_lin = c("SDNPFOAL", "LBDNFOAL"),
  pfoa_br  = c("SDBPFOAL", "LBDBFOAL")
)

for (chem in c("pfos", "pfoa")) {
  # Get available variables for the cycle
  lin_vars <- isomer_map[[paste0(chem, "_lin")]]
  br_vars  <- isomer_map[[paste0(chem, "_br")]]
  lin_lc   <- lc_map[[paste0(chem, "_lin")]]
  br_lc    <- lc_map[[paste0(chem, "_br")]]

  lin_val <- coalesce_num(dat, lin_vars)
  br_val  <- coalesce_num(dat, br_vars)
  lin_lc_val <- coalesce_num(dat, lin_lc)
  br_lc_val  <- coalesce_num(dat, br_lc)

  # Apply LOD rule: if indicator == 1, divide by sqrt(2)
  lin_val <- ifelse(!is.na(lin_lc_val) & lin_lc_val == 1, lin_val / sqrt(2), lin_val)
```

```r
  br_val  <- ifelse(!is.na(br_lc_val) & br_lc_val == 1, br_val / sqrt(2), br_val)

  # Sum them to recreate the total concentration
  summed_val <- lin_val + br_val

  # Inject back into the legacy total variables (LBXPFOS or LBXPFOA)
  target_var <- ifelse(chem == "pfos", "LBXPFOS", "LBXPFOA")

  if (target_var %in% names(dat)) {
    dat[[target_var]] <- coalesce_num(dat, c(target_var))
    dat[[target_var]] <- ifelse(is.na(dat[[target_var]]) & !is.na(summed_val), summed_val, dat[[target_
  } else {
    dat[[target_var]] <- summed_val
  }
}
# --- END ISOMER HARMONIZATION ---

blood_metal_vars <- c("LBXBPB", "LBXBCD", "LBXTHG")
urine_metal_vars <- c("URXUAS", "URXUCD", "URXUPB")

chosen_exposures <- switch(
  config$exposure_family,
  pfas = pfas_vars,
  metals = c(blood_metal_vars, urine_metal_vars),
  both = c(pfas_vars, blood_metal_vars, urine_metal_vars),
  pfas_vars
)
chosen_exposures <- chosen_exposures[chosen_exposures %in% names(dat)]
if (length(chosen_exposures) < 3) stop("Too few exposure variables found; choose different exposure fam

for (v in chosen_exposures) {
  vv <- apply_lod_rule(dat, v)
  dat[[paste0(v, "_log2")]] <- log2(vv)
}

if (any(urine_metal_vars %in% chosen_exposures)) {
  for (uv in intersect(urine_metal_vars, chosen_exposures)) {
    vv <- apply_lod_rule(dat, uv)
    dat[[paste0(uv, "_ucr_adj_log2")]] <- log2((vv / pmax(dat$ucr, 1e-6)) * 100)
  }
}

exposure_vars <- names(dat)[str_detect(names(dat), "_log2$|_ucr_adj_log2$")]
exposure_vars <- unique(exposure_vars)
# Prevent collinear double-counting: use creatinine-adjusted urine metals only, drop raw _log2 for them
raw_urine_vars <- paste0(urine_metal_vars, "_log2")
exposure_vars <- setdiff(exposure_vars, raw_urine_vars)
# Primary analysis: fixed 4-PFAS subset common across all retained cycles. Sensitivity: available-expos
primary_exposure_vars <- c("LBXPFOS_log2", "LBXPFOA_log2", "LBXPFNA_log2", "LBXPFHS_log2")
primary_exposure_vars <- primary_exposure_vars[primary_exposure_vars %in% exposure_vars]
adaptive_exposure_vars <- exposure_vars

# LOD diagnostic table -- computed on dat (post-merge, pre-log transform) stratified by cycle.
```

```r
lod_diag <- lapply(chosen_exposures, function(v) {
  x_raw <- suppressWarnings(as.numeric(as.character(dat[[v]])))
  base <- sub("^(LBX|URX)", "", v)
  lc_candidates <- c(
    paste0(v, "LC"), paste0("LBD", base, "LC"), paste0("URD", base, "LC"),
    paste0(v, "L"),  paste0("LBD", base, "L"),  paste0("URD", base, "L")
  )
  lc_var <- lc_candidates[lc_candidates %in% names(dat)]
  has_lc <- length(lc_var) > 0
  if (has_lc) {
    lc_vals <- suppressWarnings(as.numeric(as.character(dat[[lc_var[1]]])))
    below <- !is.na(lc_vals) & lc_vals == 1
  } else {
    below <- rep(FALSE, length(x_raw))
  }

  by_cycle <- dat %>%
    mutate(.x = x_raw, .below = below) %>%
    group_by(cycle) %>%
    summarise(
      n = sum(!is.na(.x)),
      n_below = sum(.below, na.rm = TRUE),
      pct_below = if (sum(!is.na(.x)) > 0) round(sum(.below, na.rm = TRUE) / sum(!is.na(.x)) * 100, 1) e
      median_below = if (sum(.below & !is.na(.x)) > 0) median(.x[.below], na.rm = TRUE) else NA_real_,
      median_above = if (sum(!.below & !is.na(.x)) > 0) median(.x[!.below & !is.na(.x)], na.rm = TRUE) e
      min_positive = if (any(.x > 0, na.rm = TRUE)) min(.x[.x > 0], na.rm = TRUE) else NA_real_,
      .groups = "drop"
    ) %>%
    mutate(exposure = v, lc_indicator = if (has_lc) lc_var[1] else "none") %>%
    select(exposure, cycle, lc_indicator, everything())
  by_cycle
}) %>% bind_rows()
kable(lod_diag, caption = "LOD diagnostic per exposure by cycle. Verify: median_below ~ LOD value; if i
```

Table 16: LOD diagnostic per exposure by cycle. Verify: median_below ~ LOD value; if it diverges from min_positive, review the LC->x/sqrt(2) assumption.

exposure	cycle	lc_indicator	n	n_below	pct_below	median_below	median_above	min_positive
LBXPFOS	2005-2006	LBDPFOSL	2120	0	0	NA	15.900	0.140
LBXPFOS	2007-2008	LBDPFOSL	2100	0	0	NA	13.400	0.140
LBXPFOS	2009-2010	LBDPFOSL	2233	0	0	NA	9.000	0.140
LBXPFOS	2011-2012	LBDPFOSL	1904	0	0	NA	6.425	0.140
LBXPFOS	2013-2014	LBDPFOSL	0	0	NA	NA	NA	NA
LBXPFOS	2015-2016	LBDPFOSL	1993	0	0	NA	4.700	0.140
LBXPFOS	2017-2018	LBDPFOSL	1929	0	0	NA	4.200	0.140

exposure	cycle	lc_indicator	n	n_below	pct_below	median_below	median_above	min_positive
LBXPFOA	2005-2006	LBDPFOAL	2120	0	0	NA	3.700	0.070
LBXPFOA	2007-2008	LBDPFOAL	2100	0	0	NA	4.200	0.070
LBXPFOA	2009-2010	LBDPFOAL	2233	0	0	NA	2.900	0.070
LBXPFOA	2011-2012	LBDPFOAL	1904	0	0	NA	2.050	0.070
LBXPFOA	2013-2014	LBDPFOAL	0	0	NA	NA	NA	NA
LBXPFOA	2015-2016	LBDPFOAL	1993	0	0	NA	1.470	0.140
LBXPFOA	2017-2018	LBDPFOAL	1929	0	0	NA	1.370	0.140
LBXPFNA	2005-2006	LBDPFNAL	2120	0	0	NA	1.000	0.070
LBXPFNA	2007-2008	LBDPFNAL	2100	0	0	NA	1.148	0.058
LBXPFNA	2009-2010	LBDPFNAL	2233	0	0	NA	1.148	0.058
LBXPFNA	2011-2012	LBDPFNAL	1904	0	0	NA	0.890	0.060
LBXPFNA	2013-2014	LBDPFNAL	2168	0	0	NA	0.700	0.070
LBXPFNA	2015-2016	LBDPFNAL	1993	0	0	NA	0.600	0.070
LBXPFNA	2017-2018	LBDPFNAL	1929	0	0	NA	0.400	0.070
LBXPFHS	2005-2006	LBDPFHSL	2120	0	0	NA	1.700	0.070
LBXPFHS	2007-2008	LBDPFHSL	2100	0	0	NA	2.000	0.070
LBXPFHS	2009-2010	LBDPFHSL	2233	0	0	NA	1.600	0.070
LBXPFHS	2011-2012	LBDPFHSL	1904	0	0	NA	1.190	0.070
LBXPFHS	2013-2014	LBDPFHSL	2168	0	0	NA	1.300	0.070
LBXPFHS	2015-2016	LBDPFHSL	1993	0	0	NA	1.200	0.070
LBXPFHS	2017-2018	LBDPFHSL	1929	0	0	NA	1.100	0.070

Analytic Sample and Survey Design

```
analytic <- dat %>%
  filter(age >= config$min_age)

# PFAS workflow: harmonize rotating 2-year subsample weights into one unified PFAS weight (WTPFAS2YR).
analytic <- analytic %>%
  mutate(
```

```r
    WTPFAS2YR = coalesce_num(cur_data(), c("WTSA2YR", "WTSB2YR", "WTSC2YR", "WTSAF2YR"))
  )

weight_candidates <- names(analytic)[stringr::str_detect(names(analytic), "^WT")]
kable(data.frame(weight_variable = weight_candidates), caption = "Weight audit: all WT* variables in ana
```

Table 17: Weight audit: all WT* variables in analytic dataset.

weight_variable
WTINT2YR
WTMEC2YR
WTSA2YR
WTSAF2YR
WTSB2YR
WTSC2YR
WTPFAS2YR

```r
# Exposure-specific weight audit in the exposure-complete analytic sample (fixed primary PFAS set).
complete_expo <- analytic %>%
  mutate(.has_all_expo = if_all(all_of(primary_exposure_vars), ~ !is.na(.x)))
wt_cols <- names(analytic)[str_detect(names(analytic), "^WT")]
# PFAS auto-selection eligibility: exclude full-sample interview/MEC weights; keep PFAS-appropriate sub
if (tolower(config$exposure_family) == "pfas") {
  wt_cols <- setdiff(wt_cols, c("WTINT2YR", "WTMEC2YR"))
}
if (length(wt_cols) > 0) {
  wt_audit <- complete_expo %>%
    filter(.has_all_expo) %>%
    summarise(across(all_of(wt_cols), ~ round(mean(!is.na(.x)) * 100, 1))) %>%
    pivot_longer(everything(), names_to = "weight", values_to = "pct_nonmissing_complete_expo") %>%
    arrange(desc(pct_nonmissing_complete_expo))
  kable(wt_audit, caption = "Weight availability (%) among participants with complete exposure set. Use
}
```

Table 18: Weight availability (%) among participants with complete exposure set. Use the weight with highest coverage that matches your subsample.

weight	pct_nonmissing_complete_expo
WTPFAS2YR	100.0
WTSAF2YR	48.9
WTSC2YR	36.5
WTSB2YR	32.9
WTSA2YR	30.7

```r
# Auto-weight: rank eligible candidate weights by coverage and apply the top PFAS-appropriate subsample
if (config$weight_type == "auto" && exists("wt_audit") && nrow(wt_audit) > 0) {
  wt_audit_auto <- wt_audit
  if (tolower(config$exposure_family) == "pfas") {
    wt_audit_auto <- wt_audit_auto %>% filter(!(weight %in% c("WTINT2YR", "WTMEC2YR")))
  }
```

```
  if (nrow(wt_audit_auto) == 0) {
    stop("Auto-weight selection failed for PFAS: no eligible WTS* subsample weights were available in t
  }
  top_row <- wt_audit_auto %>% slice(1)
  config$weight_type <- top_row$weight
  message(">>> Auto-weight resolved: Applying '", config$weight_type, "' (Coverage: ", top_row$pct_nonm
}

weight_info <- resolve_weight_var(analytic, config$weight_type, config$exposure_family)

analytic$weight_2yr <- suppressWarnings(as.numeric(as.character(analytic[[weight_info$var]])))

analytic <- analytic %>%
  filter(!is.na(phenoage), !is.na(age), !is.na(ckd), !is.na(weight_2yr), !is.na(strata), !is.na(psu))

n_cycles <- length(unique(analytic$cycle))
# After resolving the PFAS-appropriate 2-year weight, pooled analyses divide by the number of contribut
n_cycles_exposure_available <- analytic %>%
  filter(if_any(all_of(primary_exposure_vars), ~ !is.na(.x))) %>%
  summarise(n = n_distinct(cycle)) %>%
  pull(n)
if (!is.finite(n_cycles_exposure_available) || n_cycles_exposure_available < 1) {
  n_cycles_exposure_available <- n_cycles
}
repool_weights <- function(df) {
  ncy <- dplyr::n_distinct(df$cycle[!is.na(df$weight_2yr)])
  if (!is.finite(ncy) || ncy < 1) stop("Invalid cycle count for weight pooling.")
  df$weight_combined <- df$weight_2yr / ncy
  df
}
analytic <- repool_weights(analytic)
message("Weight pooling: repool_weights() applied to analytic (n_cycles in sample = ", n_cycles_exposur

# Audit B: weight availability in the final analytic sample (after all exclusions).
if (length(wt_cols) > 0) {
  wt_audit_final <- analytic %>%
    summarise(across(any_of(wt_cols), ~ round(mean(!is.na(.x)) * 100, 1))) %>%
    pivot_longer(everything(), names_to = "weight", values_to = "pct_nonmissing_final_sample") %>%
    arrange(desc(pct_nonmissing_final_sample))
  kable(wt_audit_final, caption = "Weight availability (%) in final analytic sample (post-exclusion). T
}
```

Table 19: Weight availability (%) in final analytic sample (post-exclusion). This is the definitive audit.

weight	pct_nonmissing_final_sample
WTPFAS2YR	100.0
WTSAF2YR	74.1
WTSC2YR	22.1
WTSB2YR	19.5
WTSA2YR	9.0

```r
# PRIMARY: survey-weighted residualization of PhenoAge on age.
# This ensures PhenoAgeAccel is a population-representative construct.
analytic$phenoage_accel_unweighted <- NA_real_
fit_age_uw <- lm(phenoage ~ age, data = analytic)
analytic$phenoage_accel_unweighted <- residuals(fit_age_uw)
keep_w <- which(!is.na(analytic$phenoage) & !is.na(analytic$age) & !is.na(analytic$weight_combined))
if (length(keep_w) > 0) {
  svy_resid <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE,
                         data = analytic[keep_w, , drop = FALSE])
  fit_age_wt <- svyglm(phenoage ~ age, design = svy_resid, family = gaussian())
  pred_wt <- as.numeric(predict(fit_age_wt))
  analytic$phenoage_accel <- NA_real_
  analytic$phenoage_accel[keep_w] <- analytic$phenoage[keep_w] - pred_wt
}
analytic <- analytic %>% filter(!is.na(phenoage_accel))
message(">>> PhenoAgeAccel: primary = survey-weighted residualization; unweighted version retained as s

ckd_marker_availability <- tibble::tibble(
  group = c("Both eGFR and ACR", "eGFR only", "ACR only", "Neither"),
  N = c(
    sum(!is.na(analytic$egfr) & !is.na(analytic$acr)),
    sum(!is.na(analytic$egfr) & is.na(analytic$acr)),
    sum(is.na(analytic$egfr) & !is.na(analytic$acr)),
    sum(is.na(analytic$egfr) & is.na(analytic$acr))
  )
)

if (!all(c(0, 1) %in% unique(analytic$ckd))) {
  stop("Both CKD groups are not represented after exclusions.")
}

svy <- svydesign(
  id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = analytic
)

kable(data.frame(weight_selected = weight_info$var, note = weight_info$note), caption = "Selected NHANES
```

Table 20: Selected NHANES weight variable

weight_selected	note
WTPFAS2YR	User-specified weight: WTPFAS2YR

```r
kable(ckd_marker_availability, caption = "CKD marker availability in analytic sample")
```

Table 21: CKD marker availability in analytic sample

group	N
Both eGFR and ACR	15019
eGFR only	220
ACR only	0
Neither	0

Mixture Modeling

```
valid_num <- function(v, d) v %in% names(d) && length(unique(stats::na.omit(d[[v]]))) >= 2
valid_cat <- function(v, d) v %in% names(d) && length(unique(stats::na.omit(as.character(d[[v]])))) >= 2

# Primary adjustment: lean set to avoid overadjustment/mediator bias (diabetes, HTN, CVD may be on path
# Aligned with EHP/Env Int mixture papers: sex, race, PIR, BMI, smoking, cycle; plus alcohol for confou
# Smoking and alcohol are 3-level factors (Never/Former/Current) for consistency with Table 1.
clinical_covars <- c("diabetes", "hypertension", "cvd")  # used only in sensitivity "cardiometabolic"
# No age: outcome is phenoage_accel (age-residualized). Same covariates for qgcomp and Model B.
covars <- c(
  if (valid_cat("sex", analytic)) "factor(sex)",
  if (valid_cat("race_eth", analytic)) "factor(race_eth)",
  if (valid_num("pir", analytic)) "pir",
  if (valid_cat("smoker3", analytic)) "factor(smoker3)",
  if (valid_num("bmi", analytic)) "bmi",
  if (valid_cat("alcohol3", analytic)) "factor(alcohol3)",
  if (valid_cat("cycle", analytic)) "factor(cycle)"
)

# Sanity checks before modeling.
message("--- Pre-modeling sanity checks ---")
message("Analytic N: ", nrow(analytic))
message("PhenoAgeAccel: mean=", round(mean(analytic$phenoage_accel, na.rm = TRUE), 3),
        ", SD=", round(sd(analytic$phenoage_accel, na.rm = TRUE), 3))
message("CKD prevalence: ", round(mean(analytic$ckd == 1, na.rm = TRUE) * 100, 1), "%")
message("Weight selected: ", weight_info$var)

# qgcomp complete-case: use primary (fixed) exposure vars with sufficient variability and non-missing i
qg_min_frac <- 0.5
qg_exposure_vars_for_cc <- primary_exposure_vars[vapply(primary_exposure_vars, function(v) {
  x <- suppressWarnings(as.numeric(analytic[[v]]))
  finite <- is.finite(x)
  n_ok <- sum(finite)
  pct_ok <- n_ok / max(1, nrow(analytic))
  n_ok > 0 && dplyr::n_distinct(x[finite]) >= 2 && pct_ok >= qg_min_frac
}, logical(1))]
if (length(qg_exposure_vars_for_cc) < 2) {
  qg_exposure_vars_for_cc <- primary_exposure_vars[vapply(primary_exposure_vars, function(v) {
    x <- suppressWarnings(as.numeric(analytic[[v]]))
    x <- x[is.finite(x)]
    length(x) > 0 && dplyr::n_distinct(x) >= 2
  }, logical(1))]
  if (length(qg_exposure_vars_for_cc) < 2) qg_exposure_vars_for_cc <- primary_exposure_vars
  message("qgcomp: few primary exposures met variance + ", round(qg_min_frac * 100), "% non-missing; usi
}
message("qgcomp exposures for complete-case (primary fixed set, variance + >= ", round(qg_min_frac * 10
# Qgcomp uses same adjustment as Model B (sex, race, PIR, BMI, smoking [3-level], alcohol [3-level], cy
rhs_qg_cc <- paste(c(qg_exposure_vars_for_cc, covars), collapse = " + ")
f_qg_cc <- as.formula(paste("phenoage_accel ~", rhs_qg_cc))
qg_cc <- complete.cases(model.frame(f_qg_cc, data = analytic, na.action = na.pass))
analytic_qg <- analytic[qg_cc, , drop = FALSE]
analytic_qg <- repool_weights(analytic_qg)
```

```r
# Normalize precision weights for qgcomp so glm does not treat sum(weights) as N (avoids N = 300M, infi
analytic_qg$weight_normalized <- analytic_qg$weight_combined / mean(analytic_qg$weight_combined, na.rm =
# Ensure covariates are proper type for qgcomp (character -> wrong model matrix, psi ~ 0).
for (v in c("phenoage_accel", "age", "pir", "bmi")) {
  if (v %in% names(analytic_qg)) analytic_qg[[v]] <- suppressWarnings(as.numeric(as.character(analytic_
}
analytic_qg$sex      <- factor(as.character(analytic_qg$sex))
analytic_qg$race_eth <- factor(as.character(analytic_qg$race_eth))
analytic_qg$cycle    <- factor(as.character(analytic_qg$cycle))
if ("smoker3"  %in% names(analytic_qg)) analytic_qg$smoker3  <- factor(as.character(analytic_qg$smoker3
if ("alcohol3" %in% names(analytic_qg)) analytic_qg$alcohol3 <- factor(as.character(analytic_qg$alcohol3
for (v in qg_exposure_vars_for_cc) {
  if (v %in% names(analytic_qg)) analytic_qg[[v]] <- suppressWarnings(as.numeric(as.character(analytic_
}
message("qgcomp complete-case: same covariates as Model B; N = ", nrow(analytic_qg))

# Among those, keep only exposures that still have variance in the complete-case sample (in case one co
qg_exposure_vars <- qg_exposure_vars_for_cc[vapply(qg_exposure_vars_for_cc, function(v) {
  x <- suppressWarnings(as.numeric(analytic_qg[[v]]))
  x <- x[is.finite(x)]
  length(x) > 0 && dplyr::n_distinct(x) >= 2
}, logical(1))]
if (length(qg_exposure_vars) < 2) {
  warning("Too few finite/variable exposures for qgcomp after complete-case filtering; Table 2 will be
}
if (length(qg_exposure_vars) > 0) {
  analytic_qg <- analytic_qg %>%
    filter(if_all(all_of(c("phenoage_accel", qg_exposure_vars)), ~ is.finite(.x)))
}
q_qg <- as.integer(config$q)
if (!is.finite(q_qg) || q_qg < 2) stop("config$q must be an integer >= 2.")

# Qgcomp model: same adjustment as Model B (no age).
covars_qg <- c(
  if (valid_cat("sex", analytic_qg)) "factor(sex)",
  if (valid_cat("race_eth", analytic_qg)) "factor(race_eth)",
  if (valid_num("pir", analytic_qg)) "pir",
  if (valid_cat("smoker3", analytic_qg)) "factor(smoker3)",
  if (valid_num("bmi", analytic_qg)) "bmi",
  if (valid_cat("alcohol3", analytic_qg)) "factor(alcohol3)",
  if (valid_cat("cycle", analytic_qg)) "factor(cycle)"
)
qg_covars_intended <- covars_qg
build_qg_formula <- function(exps, covs) {
  rhs <- paste(c(exps, covs), collapse = " + ")
  as.formula(paste("phenoage_accel ~", rhs))
}
f_qg <- build_qg_formula(qg_exposure_vars, covars_qg)

repeat {
  mf_qg <- tryCatch(model.frame(f_qg, data = analytic_qg, na.action = na.omit), error = function(e) NULL
  if (is.null(mf_qg) || nrow(mf_qg) == 0) break
  factor_terms_qg <- names(mf_qg)[vapply(mf_qg, is.factor, logical(1))]
```

```r
  bad_factor_terms_qg <- factor_terms_qg[vapply(mf_qg[factor_terms_qg], function(x) nlevels(droplevels(
  removable_qg <- intersect(covars_qg, bad_factor_terms_qg)
  if (length(removable_qg) == 0) break
  message("qgcomp overall: dropping collapsed factor term(s): ", paste(removable_qg, collapse = ", "))
  covars_qg <- setdiff(covars_qg, removable_qg)
  f_qg <- build_qg_formula(qg_exposure_vars, covars_qg)
}
qg_covars_realized <- covars_qg
message("qgcomp exposures used: ", paste(qg_exposure_vars, collapse = ", "))
message("qgcomp finite analytic N: ", nrow(analytic_qg))

qgcomp_overall_error <- NULL
fit_qg_overall <- tryCatch(
  qgcomp::qgcomp.noboot(
    f = f_qg, expnms = qg_exposure_vars, data = analytic_qg,
    family = gaussian(), q = q_qg, weights = analytic_qg$weight_normalized
  ),
  error = function(e) {
    qgcomp_overall_error <<- conditionMessage(e)
    NULL
  }
)
if (!is.null(fit_qg_overall)) {
  message("qgcomp N used: ", fit_qg_overall$fit$n[1])
} else {
  warning("qgcomp overall model failed: ", qgcomp_overall_error)
}

# Survey-weighted interaction model using quantized summed score.
# Quantile cutpoints are computed within each NHANES cycle using cycle-specific sampling weights.
qdat <- analytic
qdat$ckd <- as.integer(qdat$ckd)
# Strict sample = complete-case on qgcomp-compatible PFAS subset (same as analytic_qg). Label reflects
qdat_strict <- analytic_qg
qdat_strict$ckd <- as.integer(qdat_strict$ckd)

# Primary model: fixed PFAS set (primary_exposure_vars). Restrict to complete case on primary set.
qdat <- qdat %>% filter(if_all(all_of(primary_exposure_vars), ~ !is.na(.x)))
if (config$require_complete_exposure_set) {
  # Optional extra: could also require full adaptive set here; we use primary only for main.
}
message("Primary interaction: fixed PFAS set = ", paste(primary_exposure_vars, collapse = ", "))
message("N primary pre-quantile sample: ", nrow(qdat))
message("N strict sample (Strict complete-case, qgcomp-compatible PFAS subset): ", nrow(qdat_strict))

weighted_quantile <- function(x, w, probs) {
  keep <- is.finite(x) & is.finite(w) & w > 0
  x <- x[keep]; w <- w[keep]
  if (length(x) == 0) return(rep(NA_real_, length(probs)))
  ord <- order(x)
  x <- x[ord]; w <- w[ord]
  cw <- cumsum(w) / sum(w)
  sapply(probs, function(p) {
```

```r
    idx <- which(cw >= p)[1]
    if (is.na(idx)) x[length(x)] else x[idx]
  })
}

# Primary mixture score: fixed set, cycle-specific quantiles to avoid temporal confounding (PFAS trends
for (v in primary_exposure_vars) {
  qdat[[paste0(v, "_q")]] <- NA_real_
  for (cy in unique(qdat$cycle)) {
    idx <- qdat$cycle == cy
    x_cy <- suppressWarnings(as.numeric(qdat[[v]][idx]))
    w_cy <- suppressWarnings(as.numeric(qdat$weight_combined[idx]))
    probs <- weighted_quantile(x_cy, w_cy, probs = seq(0, 1, length.out = config$q + 1))
    br <- sort(unique(as.numeric(probs)))
    if (length(br) > 1) {
      br[1] <- -Inf
      br[length(br)] <- Inf
    }
    if (length(br) < 3 || any(diff(br) <= 0)) {
      r <- rank(qdat[[v]][idx], na.last = "keep", ties.method = "average")
      qdat[[paste0(v, "_q")]][idx] <- as.numeric(cut(r, breaks = config$q, labels = FALSE, include.lowe
    } else {
      qdat[[paste0(v, "_q")]][idx] <- as.numeric(cut(qdat[[v]][idx], breaks = br, include.lowest = TRUE
    }
  }
}
qvars <- paste0(primary_exposure_vars, "_q")
qdat$nonmissing_q_exposures <- rowSums(!is.na(qdat[, qvars, drop = FALSE]))
qdat$mixture_qscore <- rowMeans(qdat[, qvars, drop = FALSE], na.rm = TRUE)
# Primary: require all primary PFAS non-missing (fixed set).
required_primary <- length(primary_exposure_vars)
qdat$mixture_qscore <- ifelse(qdat$nonmissing_q_exposures < required_primary, NA_real_, qdat$mixture_qs
# By-cycle table for reporting (primary set is fixed; this documents it).
cycle_capacity <- tibble(
  cycle = unique(qdat$cycle),
  measured_exposures = length(primary_exposure_vars),
  PFAS_measured = paste(primary_exposure_vars, collapse = ", "),
  required_nonmissing_exposures = required_primary
)

# Build strict mixture score: same exposure set as qgcomp, cycle-specific quantiles.
for (v in qg_exposure_vars) {
  qdat_strict[[paste0(v, "_q")]] <- NA_real_
  for (cy in unique(qdat_strict$cycle)) {
    idx <- qdat_strict$cycle == cy
    x_cy <- suppressWarnings(as.numeric(qdat_strict[[v]][idx]))
    w_cy <- suppressWarnings(as.numeric(qdat_strict$weight_combined[idx]))
    probs_s <- weighted_quantile(x_cy, w_cy, probs = seq(0, 1, length.out = config$q + 1))
    br_s <- sort(unique(as.numeric(probs_s)))
    if (length(br_s) > 1) {
      br_s[1] <- -Inf
      br_s[length(br_s)] <- Inf
    }
```

```r
    if (length(br_s) < 3 || any(diff(br_s) <= 0)) {
      r_s <- rank(qdat_strict[[v]][idx], na.last = "keep", ties.method = "average")
      qdat_strict[[paste0(v, "_q")]][idx] <- as.numeric(cut(r_s, breaks = config$q, labels = FALSE, incl
    } else {
      qdat_strict[[paste0(v, "_q")]][idx] <- as.numeric(cut(qdat_strict[[v]][idx], breaks = br_s, inclu
    }
  }
}
qvars_strict <- paste0(qg_exposure_vars, "_q")
qdat_strict$nonmissing_q_exposures <- rowSums(!is.na(qdat_strict[, qvars_strict, drop = FALSE]))
qdat_strict$mixture_qscore <- rowMeans(qdat_strict[, qvars_strict, drop = FALSE], na.rm = TRUE)

# Hard type enforcement to prevent silent coercion issues in svyglm.
qdat <- qdat %>%
  mutate(
    phenoage_accel = suppressWarnings(as.numeric(phenoage_accel)),
    mixture_qscore = suppressWarnings(as.numeric(mixture_qscore)),
    weight_combined = suppressWarnings(as.numeric(weight_combined)),
    psu = suppressWarnings(as.numeric(psu)),
    strata = suppressWarnings(as.numeric(strata)),
    ckd = as.integer(ckd)
  )
qdat_strict <- qdat_strict %>%
  mutate(
    phenoage_accel = suppressWarnings(as.numeric(phenoage_accel)),
    mixture_qscore = suppressWarnings(as.numeric(mixture_qscore)),
    weight_combined = suppressWarnings(as.numeric(weight_combined)),
    psu = suppressWarnings(as.numeric(psu)),
    strata = suppressWarnings(as.numeric(strata)),
    ckd = as.integer(ckd)
  )

# Primary model: fixed PFAS set (same in every cycle).
cycle_capacity_with_min <- cycle_capacity
kable(
  cycle_capacity_with_min,
  caption = "Primary analysis: fixed PFAS set by cycle. Same chemicals and required non-missing in all c
)
```

Table 22: Primary analysis: fixed PFAS set by cycle. Same chemicals and required non-missing in all cycles.

cycle	measured_exposures	PFAS_measured	required_nonmissing_exposures
2005-2006	4	LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2	4
2007-2008	4	LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2	4
2009-2010	4	LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2	4
2015-2016	4	LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2	4
2017-2018	4	LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2	4

```r
kable(
  tibble(
    statistic = c("N", "Mean", "SD", "Min", "P25", "Median", "P75", "Max"),
    value = c(
      sum(!is.na(qdat$mixture_qscore)),
      round(mean(qdat$mixture_qscore, na.rm = TRUE), 3),
      round(sd(qdat$mixture_qscore, na.rm = TRUE), 3),
      round(min(qdat$mixture_qscore, na.rm = TRUE), 3),
      round(quantile(qdat$mixture_qscore, 0.25, na.rm = TRUE), 3),
      round(median(qdat$mixture_qscore, na.rm = TRUE), 3),
      round(quantile(qdat$mixture_qscore, 0.75, na.rm = TRUE), 3),
      round(max(qdat$mixture_qscore, na.rm = TRUE), 3)
    )
  ),
  caption = paste0("Primary mixture quantile score (fixed set, N chemicals = ", length(primary_exposure
                   "). Score = mean quantile bin across ", paste(primary_exposure_vars, collapse = ", "
                   ". Complete case on primary set required.")
)
```

Table 23: Primary mixture quantile score (fixed set, N chemicals = 4). Score = mean quantile bin across LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2. Complete case on primary set required.

statistic	value
N	7633.000
Mean	2.419
SD	0.933
Min	1.000
P25	1.500
Median	2.500
P75	3.250
Max	4.000

```r
stopifnot(all(c("psu", "strata", "weight_combined") %in% names(qdat)))
qdat <- qdat %>% filter(!is.na(psu), !is.na(strata), !is.na(weight_combined), !is.na(phenoage_accel), !
qdat_strict <- qdat_strict %>% filter(!is.na(psu), !is.na(strata), !is.na(weight_combined), !is.na(phen
qdat         <- repool_weights(qdat)
qdat_strict  <- repool_weights(qdat_strict)

# Ensure all model variables are proper type so svyglm does not zero out coefficients (character -> wro
for (v in exposure_vars) {
  if (v %in% names(qdat)) qdat[[v]] <- suppressWarnings(as.numeric(as.character(qdat[[v]])))
}
qdat$phenoage_accel <- suppressWarnings(as.numeric(as.character(qdat$phenoage_accel)))
qdat$age            <- suppressWarnings(as.numeric(as.character(qdat$age)))
qdat$pir            <- suppressWarnings(as.numeric(as.character(qdat$pir)))
qdat$bmi            <- suppressWarnings(as.numeric(as.character(qdat$bmi)))
qdat$sex            <- factor(as.character(qdat$sex))
qdat$race_eth       <- factor(as.character(qdat$race_eth))
qdat$cycle          <- factor(as.character(qdat$cycle))
if ("smoker3"  %in% names(qdat)) qdat$smoker3  <- factor(as.character(qdat$smoker3),  levels = c("Never
```

```r
if ("alcohol3" %in% names(qdat)) qdat$alcohol3 <- factor(as.character(qdat$alcohol3), levels = c("Never
qdat$ckd            <- as.integer(qdat$ckd)

for (v in exposure_vars) {
  if (v %in% names(qdat_strict)) qdat_strict[[v]] <- suppressWarnings(as.numeric(as.character(qdat_stri
}
qdat_strict$phenoage_accel <- suppressWarnings(as.numeric(as.character(qdat_strict$phenoage_accel)))
qdat_strict$age            <- suppressWarnings(as.numeric(as.character(qdat_strict$age)))
qdat_strict$pir            <- suppressWarnings(as.numeric(as.character(qdat_strict$pir)))
qdat_strict$bmi            <- suppressWarnings(as.numeric(as.character(qdat_strict$bmi)))
qdat_strict$sex            <- factor(as.character(qdat_strict$sex))
qdat_strict$race_eth       <- factor(as.character(qdat_strict$race_eth))
qdat_strict$cycle          <- factor(as.character(qdat_strict$cycle))
if ("smoker3"  %in% names(qdat_strict)) qdat_strict$smoker3  <- factor(as.character(qdat_strict$smoker3
if ("alcohol3" %in% names(qdat_strict)) qdat_strict$alcohol3 <- factor(as.character(qdat_strict$alcohol3
qdat_strict$ckd            <- as.integer(qdat_strict$ckd)

message("---- Model B diagnostics ----")
message("N qdat: ", nrow(qdat))
print(table(qdat$ckd, useNA = "ifany"))
```

```
##
##    0    1
## 6315 1318
```

```r
message("N qdat_strict: ", nrow(qdat_strict))
print(table(qdat_strict$ckd, useNA = "ifany"))
```

```
##
##    0    1
## 4394  873
```

```r
sd_all <- sd(qdat$mixture_qscore, na.rm = TRUE)
sd_by <- tapply(qdat$mixture_qscore, qdat$ckd, sd, na.rm = TRUE)
message("SD(mixture_qscore) overall: ", sd_all)
print(sd_by)
```

```
##         0         1
## 0.9268457 0.9599365
```

```r
if (length(unique(stats::na.omit(qdat$ckd))) < 2) {
  stop("After exposure/design filtering, only one CKD group remains in qdat. ",
       "Interaction mixture_qscore*ckd cannot be estimated. ",
       "Set config$require_complete_exposure_set = FALSE or reduce exposure set/cycles.")
}
if (any(is.finite(sd_by) & sd_by < 1e-8)) {
  stop("mixture_qscore has near-zero variance within at least one CKD group. ",
       "Interaction cannot be estimated; check quantile binning/ties and exposure variability.")
}

miss_compare_ckd <- qdat %>%
  mutate(
    retained_primary = complete.cases(phenoage_accel, mixture_qscore, ckd, age, sex, race_eth, pir, smo
  ) %>%
  group_by(ckd, retained_primary) %>%
```

```r
  summarise(
    N = n(),
    mean_age = round(mean(age, na.rm = TRUE), 1),
    female_pct = round(mean(sex == "Female", na.rm = TRUE) * 100, 1),
    .groups = "drop"
  ) %>%
  group_by(ckd) %>%
  mutate(pct_within_ckd = round(100 * N / sum(N), 1)) %>%
  ungroup() %>%
  mutate(
    ckd = ifelse(ckd == 1, "CKD", "No CKD"),
    retained_primary = ifelse(retained_primary, "Retained in Model B", "Dropped from Model B")
  )
kable(miss_compare_ckd, caption = "Primary interaction model retention by CKD status (retained vs droppe
```

Table 24: Primary interaction model retention by CKD status (retained vs dropped participants).

ckd	retained_primary	N	mean_age	female_pct	pct_within_ckd
No CKD	Dropped from Model B	1921	47.7	52.7	30.4
No CKD	Retained in Model B	4394	46.8	51.6	69.6
CKD	Dropped from Model B	445	63.9	53.7	33.8
CKD	Retained in Model B	873	63.3	51.5	66.2

```r
# Use explicit data-frame copy so survey design uses correct types (avoids ~0 coefficients from referen
qdat_svy <- as.data.frame(qdat)
qdat_svy$phenoage_accel <- as.numeric(qdat_svy$phenoage_accel)
qdat_svy$mixture_qscore <- as.numeric(qdat_svy$mixture_qscore)
qdat_svy$ckd            <- as.integer(qdat_svy$ckd)
qdat_svy$age            <- as.numeric(qdat_svy$age)
qdat_svy$pir            <- as.numeric(qdat_svy$pir)
qdat_svy$bmi            <- as.numeric(qdat_svy$bmi)
qdat_svy$sex            <- factor(qdat_svy$sex)
qdat_svy$race_eth       <- factor(qdat_svy$race_eth)
qdat_svy$cycle          <- factor(qdat_svy$cycle)
if ("smoker3"  %in% names(qdat_svy)) qdat_svy$smoker3  <- factor(as.character(qdat_svy$smoker3),  levels
if ("alcohol3" %in% names(qdat_svy)) qdat_svy$alcohol3 <- factor(as.character(qdat_svy$alcohol3), levels
qdat_svy$weight_combined <- as.numeric(qdat_svy$weight_combined)
qdat_svy$psu            <- as.numeric(qdat_svy$psu)
qdat_svy$strata         <- as.numeric(qdat_svy$strata)
# Drop unused factor levels to avoid singularity with full survey design.
qdat_svy$sex      <- droplevels(factor(qdat_svy$sex))
qdat_svy$race_eth <- droplevels(factor(qdat_svy$race_eth))
qdat_svy$cycle    <- droplevels(factor(qdat_svy$cycle))
if ("smoker3"  %in% names(qdat_svy)) qdat_svy$smoker3  <- droplevels(qdat_svy$smoker3)
if ("alcohol3" %in% names(qdat_svy)) qdat_svy$alcohol3 <- droplevels(qdat_svy$alcohol3)

# ---- Pre-compute sensitivity-derived columns on qdat_svy so they ride into MI imputations ----
# These outcomes/redefinitions depend only on biomarkers + age (not imputed), so they can be computed o
ph2_comp_qsvy <- compute_phenoage_components(qdat_svy, include_scr = FALSE, include_alb = TRUE)
ph2_qsvy <- ph2_comp_qsvy$phenoage
qdat_svy$phenoage_nokid <- NA_real_
```

```r
keep2_qsvy <- which(!is.na(ph2_qsvy) & !is.na(qdat_svy$age))
if (length(keep2_qsvy) > 0) {
  fit_nokid_qsvy <- lm(ph2_qsvy[keep2_qsvy] ~ qdat_svy$age[keep2_qsvy])
  qdat_svy$phenoage_nokid[keep2_qsvy] <- residuals(fit_nokid_qsvy)
}
ph3_comp_qsvy <- compute_phenoage_components(qdat_svy, include_scr = FALSE, include_alb = FALSE)
ph3_qsvy <- ph3_comp_qsvy$phenoage
qdat_svy$phenoage_nokid_both <- NA_real_
keep3_qsvy <- which(!is.na(ph3_qsvy) & !is.na(qdat_svy$age))
if (length(keep3_qsvy) > 0) {
  fit_nokid_both_qsvy <- lm(ph3_qsvy[keep3_qsvy] ~ qdat_svy$age[keep3_qsvy])
  qdat_svy$phenoage_nokid_both[keep3_qsvy] <- residuals(fit_nokid_both_qsvy)
}
qdat_svy$ckd_sens <- ifelse(!is.na(qdat_svy$egfr), as.integer(qdat_svy$egfr < 60), NA_integer_)

# Mirror the same derivations on qdat for complete-case sensitivities.
qdat$phenoage_nokid       <- qdat_svy$phenoage_nokid[match(qdat$SEQN, qdat_svy$SEQN)]
qdat$phenoage_nokid_both  <- qdat_svy$phenoage_nokid_both[match(qdat$SEQN, qdat_svy$SEQN)]
qdat$ckd_sens             <- qdat_svy$ckd_sens[match(qdat$SEQN, qdat_svy$SEQN)]

# Full complex survey design (PSU/strata) for valid inference; droplevels above avoids empty cells.
svy_q <- tryCatch(
  svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = qdat_svy),
  error = function(e) {
    message("Full design failed (", conditionMessage(e), "); falling back to weights-only design.")
    svydesign(id = ~1, strata = NULL, weights = ~weight_combined, data = qdat_svy)
  }
)

# ---- Multiple imputation (MICE) for missing model covariates ----
# We NEVER impute the outcome (phenoage_accel), exposures (PFAS / mixture_qscore), CKD, or design vars.
# Imputation base = qdat_svy (already requires non-missing PhenoAge, CKD, weights/PSU/strata, and full
mice_target_vars <- intersect(
  c("smoker3", "alcohol3", "bmi", "pir", "education",
    "phys_active", "diabetes", "hypertension", "cvd"),
  names(qdat_svy)
)
mice_predictor_vars <- intersect(
  c("phenoage_accel", "ckd", "mixture_qscore", "age", "sex", "race_eth", "cycle"),
  names(qdat_svy)
)

mids_obj <- NULL
imp_list <- NULL
svy_q_mi <- NULL
mice_pre_n_missing <- if (length(mice_target_vars) > 0) {
  vapply(mice_target_vars, function(v) sum(is.na(qdat_svy[[v]])), integer(1))
} else integer(0)

run_mice <- isTRUE(config$use_mice) && length(mice_target_vars) > 0 && any(mice_pre_n_missing > 0)
if (run_mice) {
  mi_data <- qdat_svy[, unique(c(mice_target_vars, mice_predictor_vars)), drop = FALSE]
  # Force factor types correctly for mice methods.
```

```r
for (v in c("smoker3", "alcohol3")) {
  if (v %in% names(mi_data)) mi_data[[v]] <- factor(as.character(mi_data[[v]]),
                                                    levels = c("Never", "Former", "Current"))
}
for (v in intersect(c("sex", "race_eth", "cycle", "education"), names(mi_data))) {
  mi_data[[v]] <- factor(as.character(mi_data[[v]]))
}
for (v in intersect(c("phys_active", "diabetes", "hypertension", "cvd"), names(mi_data))) {
  mi_data[[v]] <- factor(as.character(mi_data[[v]]))
}
for (v in intersect(c("phenoage_accel", "mixture_qscore", "age", "bmi", "pir"), names(mi_data))) {
  mi_data[[v]] <- as.numeric(mi_data[[v]])
}
if ("ckd" %in% names(mi_data)) mi_data$ckd <- factor(as.character(mi_data$ckd))

pred_mat <- matrix(0, nrow = ncol(mi_data), ncol = ncol(mi_data),
                   dimnames = list(names(mi_data), names(mi_data)))
for (v in mice_target_vars) {
  others <- setdiff(names(mi_data), v)
  pred_mat[v, others] <- 1L
}

meth <- character(ncol(mi_data)); names(meth) <- names(mi_data)
for (v in names(mi_data)) {
  if (!(v %in% mice_target_vars)) { meth[v] <- ""; next }
  if (is.factor(mi_data[[v]])) {
    meth[v] <- if (nlevels(droplevels(mi_data[[v]])) == 2) "logreg" else "polyreg"
  } else {
    meth[v] <- "pmm"
  }
}
message(">>> MICE: imputing ", paste(mice_target_vars, collapse = ", "),
        " with m=", config$mice_m, ", maxit=", config$mice_maxit)
mids_obj <- tryCatch(
  mice::mice(mi_data, m = config$mice_m, maxit = config$mice_maxit,
             method = meth, predictorMatrix = pred_mat,
             printFlag = FALSE, seed = config$mice_seed),
  error = function(e) { message("mice() failed: ", conditionMessage(e)); NULL }
)
if (!is.null(mids_obj)) {
  imp_list <- mitools::imputationList(lapply(seq_len(config$mice_m), function(i) {
    df <- qdat_svy
    completed <- mice::complete(mids_obj, i)
    for (v in mice_target_vars) df[[v]] <- completed[[v]]
    # Re-pin factor levels in the merged frame so model.frame is consistent across imputations.
    if ("smoker3"  %in% names(df)) df$smoker3  <- factor(as.character(df$smoker3),  levels = c("Never
    if ("alcohol3" %in% names(df)) df$alcohol3 <- factor(as.character(df$alcohol3), levels = c("Never
    df
  }))
  svy_q_mi <- tryCatch(
    survey::svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined,
                      nest = TRUE, data = imp_list),
    error = function(e) survey::svydesign(id = ~1, strata = NULL,
```

```r
                                                weights = ~weight_combined, data = imp_list)
    )
  }
}
n_mi_base <- nrow(qdat_svy)
mice_diag <- tibble::tibble(
  variable = mice_target_vars,
  n_missing_pre = as.integer(mice_pre_n_missing),
  pct_missing_pre = round(100 * mice_pre_n_missing / max(1, n_mi_base), 2)
)
if (length(mice_target_vars) > 0) {
  kable(mice_diag, caption = paste0(
    "MICE pre-imputation missingness (within Model B base sample, N = ",
    format(n_mi_base, big.mark = ","), ")."
  ))
}
```

Table 25: MICE pre-imputation missingness (within Model B base sample, N = 7,633).

variable	n_missing_pre	pct_missing_pre
smoker3	5	0.07
alcohol3	1829	23.96
bmi	88	1.15
pir	731	9.58
education	1123	14.71
phys_active	51	0.67
diabetes	172	2.25
hypertension	12	0.16
cvd	36	0.47

```r
mice_active <- isTRUE(config$use_mice) && !is.null(svy_q_mi)
message(">>> MICE active for inference: ", mice_active,
        if (mice_active) paste0(" (m=", config$mice_m, ", N=", n_mi_base, ")") else "")

# ---- Helpers: tidy + svycontrast pooling that handle single-fit OR MI list ----
fit_svyglm_pool <- function(formula, design, family = stats::gaussian()) {
  if (inherits(design, "svyimputationList") || inherits(design, "svy.designs.imputationList")) {
    fits <- with(design, svyglm(formula, family = family))
    pooled <- mitools::MIcombine(fits)
    list(fits = fits, pooled = pooled, mi = TRUE)
  } else {
    fit <- svyglm(formula, design = design, family = family)
    list(fit = fit, mi = FALSE)
  }
}

tidy_pool <- function(x, conf.level = 0.95) {
  alpha <- 1 - conf.level
  if (isTRUE(x$mi)) {
    p <- x$pooled
    coefs <- coef(p); ses <- sqrt(diag(vcov(p)))
```

```r
    df_v <- if (!is.null(p$df) && length(p$df) == length(coefs)) as.numeric(p$df) else rep(Inf, length(
    df_v[!is.finite(df_v) | df_v <= 0] <- Inf
    t_val <- coefs / ses
    p_val <- 2 * pt(-abs(t_val), df = df_v)
    crit <- qt(1 - alpha / 2, df = df_v)
    tibble::tibble(
      term = names(coefs),
      estimate = unname(coefs),
      std.error = unname(ses),
      statistic = unname(t_val),
      p.value = unname(p_val),
      conf.low = unname(coefs - crit * ses),
      conf.high = unname(coefs + crit * ses)
    )
  } else {
    broom::tidy(x$fit, conf.int = TRUE, conf.level = conf.level)
  }
}

coef_pool <- function(x) {
  if (isTRUE(x$mi)) coef(x$pooled) else coef(x$fit)
}

contrast_pool <- function(x, c_vec, conf.level = 0.95) {
  alpha <- 1 - conf.level
  if (isTRUE(x$mi)) {
    contrasts <- lapply(x$fits, function(f) survey::svycontrast(f, c_vec))
    pooled <- mitools::MIcombine(contrasts)
    est <- as.numeric(coef(pooled)); se <- as.numeric(sqrt(diag(vcov(pooled))))
    df_v <- if (!is.null(pooled$df) && length(pooled$df) == length(est)) as.numeric(pooled$df) else Inf
    df_v[!is.finite(df_v) | df_v <= 0] <- Inf
  } else {
    ctr <- survey::svycontrast(x$fit, c_vec)
    est <- as.numeric(coef(ctr)); se <- as.numeric(SE(ctr))
    df_v <- x$fit$df.residual
    if (is.null(df_v) || !is.finite(df_v)) df_v <- Inf
  }
  t_val <- est / se
  list(
    estimate = est, se = se, df = df_v,
    p.value = 2 * pt(-abs(t_val), df = df_v),
    conf.low = est - qt(1 - alpha / 2, df = df_v) * se,
    conf.high = est + qt(1 - alpha / 2, df = df_v) * se
  )
}

# ---- qgcomp Rubin pooling: run qgcomp.noboot on each imputation, pool psi (overall mixture coef) ----
# Always writes a small diagnostic log next to the Rmd so failures are visible even when
# knitr suppresses message()/warning() output.
qgcomp_mi_pool <- function(formula, expnms, imp_list_obj, q, weight_col = "weight_combined",
                           log_path = "qgcomp_mi_pool_log.txt") {
  log_lines <- character(0)
  log <- function(...) log_lines <<- c(log_lines, paste0(...))
```

```r
on.exit({
  try(writeLines(log_lines, log_path), silent = TRUE)
}, add = TRUE)
log("[", format(Sys.time()), "] qgcomp_mi_pool called.")
log("  formula: ", deparse(formula))
log("  expnms : ", paste(expnms, collapse = ", "))
log("  q      : ", q)
log("  weight : ", weight_col)
if (is.null(imp_list_obj)) { log("ABORT: imp_list_obj is NULL."); return(NULL) }
imps <- imp_list_obj$imputations
m <- length(imps)
log("  m (#imputations): ", m)
if (m == 0) { log("ABORT: empty imputation list."); return(NULL) }
log("  imp[1] cols (first 12): ", paste(head(names(imps[[1]]), 12), collapse = ", "))
results <- lapply(seq_len(m), function(i) {
  df <- as.data.frame(imps[[i]])
  if (!(weight_col %in% names(df))) {
    log("  imp ", i, ": missing weight column '", weight_col, "'.")
    return(NULL)
  }
  df$weight_normalized <- df[[weight_col]] / mean(df[[weight_col]], na.rm = TRUE)
  needed <- intersect(c("phenoage_accel", expnms, weight_col), names(df))
  df <- df[complete.cases(df[, needed, drop = FALSE]), , drop = FALSE]
  n_unwt <- nrow(df)
  if (n_unwt == 0) {
    log("  imp ", i, ": 0 complete-case rows after filtering on outcome/exposure/weight.")
    return(NULL)
  }
  # Bind data into a temporary global so qgcomp's match.call()-based weights handling
  # resolves the weights vector cleanly outside the lapply closure.
  assign("qgcomp_data_local", df, envir = .GlobalEnv)
  on.exit(if (exists("qgcomp_data_local", envir = .GlobalEnv))
    rm("qgcomp_data_local", envir = .GlobalEnv), add = TRUE)
  fit <- tryCatch(
    do.call(
      qgcomp::qgcomp.noboot,
      list(
        f = formula, expnms = expnms, data = qgcomp_data_local,
        family = stats::gaussian(), q = q,
        weights = qgcomp_data_local$weight_normalized
      )
    ),
    error = function(e) {
      log("  imp ", i, ": qgcomp error: ", conditionMessage(e))
      NULL
    }
  )
  if (is.null(fit)) return(NULL)
  log("  imp ", i, ": OK (n_unwt=", n_unwt,
      ", psi=", tryCatch(round(fit$psi[1], 4), error = function(e) NA), ")")
  list(fit = fit, n_unwt = n_unwt)
})
results <- Filter(Negate(is.null), results)
```

```r
  if (length(results) == 0) {
    log("ABORT: all per-imputation fits failed.")
    return(NULL)
  }
  fits <- lapply(results, `[[`, "fit")
  m_eff <- length(fits)
  psis <- vapply(fits, function(f) unname(f$psi[1]), numeric(1))
  vars <- vapply(fits, function(f) unname(f$var.psi[1]), numeric(1))
  ns   <- vapply(results, function(r) as.integer(r$n_unwt), integer(1))
  qbar <- mean(psis)
  ubar <- mean(vars)
  Bvar <- if (m_eff > 1) stats::var(psis) else 0
  Tvar <- ubar + (1 + 1 / m_eff) * Bvar
  se   <- sqrt(Tvar)
  rubin_df <- if (Bvar > 0) (m_eff - 1) * (1 + ubar / ((1 + 1 / m_eff) * Bvar))^2 else Inf
  if (!is.finite(rubin_df) || rubin_df <= 0) rubin_df <- Inf
  t_val <- qbar / se
  log("OK: pooled m_eff=", m_eff, "/", m,
      "; psi=", round(qbar, 4), "; se=", round(se, 4),
      "; df=", round(rubin_df, 1))
  list(
    psi = qbar, se = se, df = rubin_df, m = m_eff, m_attempted = m,
    n_per_fit = ns,
    p.value = 2 * pt(-abs(t_val), df = rubin_df),
    conf.low = qbar - qt(0.975, df = rubin_df) * se,
    conf.high = qbar + qt(0.975, df = rubin_df) * se,
    fits = fits
  )
}
svy_q_strict <- if (nrow(qdat_strict) > 0) {
  qdat_strict_svy <- as.data.frame(qdat_strict)
  for (v in c("phenoage_accel", "mixture_qscore", "ckd", "age", "pir", "bmi", "weight_combined", "psu",
    if (v %in% names(qdat_strict_svy)) qdat_strict_svy[[v]] <- suppressWarnings(as.numeric(as.character
  }
  qdat_strict_svy$ckd <- as.integer(qdat_strict_svy$ckd)
  qdat_strict_svy$sex <- droplevels(factor(qdat_strict_svy$sex))
  qdat_strict_svy$race_eth <- droplevels(factor(qdat_strict_svy$race_eth))
  qdat_strict_svy$cycle <- droplevels(factor(qdat_strict_svy$cycle))
  if ("smoker3"  %in% names(qdat_strict_svy)) qdat_strict_svy$smoker3  <- droplevels(factor(as.characte
  if ("alcohol3" %in% names(qdat_strict_svy)) qdat_strict_svy$alcohol3 <- droplevels(factor(as.characte
  tryCatch(
    svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = qdat_strict_
    error = function(e) svydesign(id = ~1, strata = NULL, weights = ~weight_combined, data = qdat_stric
  )
} else {
  NULL
}

# Model B: outcome is phenoage_accel (already age-residualized); do not adjust for age again (no double
covars_int <- c(
  if (valid_cat("sex", qdat)) "factor(sex)",
  if (valid_cat("race_eth", qdat)) "factor(race_eth)",
  if (valid_num("pir", qdat)) "pir",
```

```r
  if (valid_cat("smoker3", qdat)) "factor(smoker3)",
  if (valid_num("bmi", qdat)) "bmi",
  if (valid_cat("alcohol3", qdat)) "factor(alcohol3)",
  if (valid_cat("cycle", qdat)) "factor(cycle)"
)
intended_covars_int <- covars_int
build_int_formula <- function(covars_vec) {
  rhs <- if (length(covars_vec) > 0) paste("+", paste(covars_vec, collapse = " + ")) else ""
  as.formula(paste("phenoage_accel ~ mixture_qscore * ckd", rhs))
}
f_int <- build_int_formula(covars_int)

repeat {
  mf_int <- tryCatch(model.frame(f_int, data = qdat, na.action = na.omit), error = function(e) NULL)
  if (is.null(mf_int) || nrow(mf_int) == 0) {
    stop("Model B preprocessing failed: no complete-case rows available for interaction formula.")
  }
  factor_terms <- names(mf_int)[vapply(mf_int, is.factor, logical(1))]
  bad_factor_terms <- factor_terms[vapply(mf_int[factor_terms], function(x) nlevels(droplevels(x)) < 2,
  removable_terms <- intersect(covars_int, bad_factor_terms)
  if (length(removable_terms) == 0) break
  message("Model B: dropping collapsed factor term(s) after complete-case filtering: ",
          paste(removable_terms, collapse = ", "))
  covars_int <- setdiff(covars_int, removable_terms)
  f_int <- build_int_formula(covars_int)
}
n_modelb <- nrow(mf_int)
realized_covars_int <- covars_int
covar_report <- tibble(
  intended_term = intended_covars_int,
  realized_in_model_b = ifelse(intended_covars_int %in% realized_covars_int, "Yes", "No")
)
kable(covar_report, caption = "Model B covariate transparency: intended vs realized adjustment terms.")
```

Table 26: Model B covariate transparency: intended vs realized adjustment terms.

intended_term	realized_in_model_b
factor(sex)	Yes
factor(race_eth)	Yes
pir	Yes
factor(smoker3)	Yes
bmi	Yes
factor(alcohol3)	Yes
factor(cycle)	Yes

```r
# ---- Model B fit: complete-case (always) AND MI-pooled (when use_mice = TRUE) ----
fit_int_cc_error <- NULL
fit_int_cc_raw <- tryCatch(
  svyglm(f_int, design = svy_q, family = gaussian()),
  error = function(e) { fit_int_cc_error <<- conditionMessage(e); NULL }
)
```

```r
if (is.null(fit_int_cc_raw)) stop("Model B complete-case (svyglm interaction) failed: ", fit_int_cc_erro
fit_int_cc <- list(fit = fit_int_cc_raw, mi = FALSE)
message("Model B complete-case fit OK. N = ", n_modelb)
print(coef(fit_int_cc_raw))
```

```
##                        (Intercept)                     mixture_qscore
##                        -9.02953978                       -0.97403243
##                                ckd                    factor(sex)Male
##                         9.02902903                        2.23455946
## factor(race_eth)Non-Hispanic Asian factor(race_eth)Non-Hispanic Black
##                        -0.14218927                        1.43046961
## factor(race_eth)Non-Hispanic White        factor(race_eth)Other/Multi
##                        -0.42608132                       -0.74131548
##                                pir              factor(smoker3)Former
##                        -0.43774919                       -0.08647637
##             factor(smoker3)Current                               bmi
##                         2.49075800                        0.33608299
##             factor(alcohol3)Former            factor(alcohol3)Current
##                        -0.20360945                       -0.88973545
##             factor(cycle)2007-2008             factor(cycle)2009-2010
##                        -0.86223821                       -0.89630932
##             factor(cycle)2015-2016             factor(cycle)2017-2018
##                         4.11959052                        4.18113757
##                 mixture_qscore:ckd
##                        -1.53791293
```

```r
fit_int_mi_error <- NULL
fit_int_mi <- if (mice_active) {
  tryCatch(
    fit_svyglm_pool(f_int, svy_q_mi),
    error = function(e) { fit_int_mi_error <<- conditionMessage(e); NULL }
  )
} else NULL
if (mice_active && is.null(fit_int_mi)) {
  message("Model B MI pooling failed: ", fit_int_mi_error, ". Falling back to complete-case.")
}

# Primary `fit_int` exposed downstream: MI-pooled if available, else complete-case.
fit_int <- if (!is.null(fit_int_mi)) fit_int_mi else fit_int_cc
n_modelb_primary <- if (!is.null(fit_int_mi)) n_mi_base else n_modelb
modelb_primary_label <- if (!is.null(fit_int_mi)) "MI-pooled (Rubin)" else "Complete-case"
message("Model B primary inference: ", modelb_primary_label, "; N = ", n_modelb_primary,
        " (complete-case N = ", n_modelb, ")")
# Sanity check: weighted lm and svyglm (weights-only design) should match (uses complete-case data).
fit_int_lm <- lm(f_int, data = qdat_svy, weights = weight_combined)
message("Weighted lm mixture_qscore coef: ", round(coef(fit_int_lm)["mixture_qscore"], 6),
        "; svyglm CC: ", round(coef(fit_int_cc_raw)["mixture_qscore"], 6))

aqg0 <- analytic_qg %>% filter(ckd == 0)
aqg1 <- analytic_qg %>% filter(ckd == 1)
if (nrow(aqg0) > 0) aqg0$weight_normalized <- aqg0$weight_combined / mean(aqg0$weight_combined, na.rm =
if (nrow(aqg1) > 0) aqg1$weight_normalized <- aqg1$weight_combined / mean(aqg1$weight_combined, na.rm =
fit_qg_ckd0 <- tryCatch(
  qgcomp::qgcomp.noboot(
```

```
    f_qg, expnms = qg_exposure_vars,
    data = aqg0,
    family = gaussian(), q = q_qg,
    weights = aqg0$weight_normalized
  ),
  error = function(e) NULL
)
fit_qg_ckd1 <- tryCatch(
  qgcomp::qgcomp.noboot(
    f_qg, expnms = qg_exposure_vars,
    data = aqg1,
    family = gaussian(), q = q_qg,
    weights = aqg1$weight_normalized
  ),
  error = function(e) NULL
)
```

Sample Flow (PRISMA-style)

```
n0 <- nrow(nhanes_raw)
n1 <- sum(dat$age >= config$min_age, na.rm = TRUE)
# Split the old "PhenoAge AND CKD non-missing" step into two consecutive exclusions:
# (a) missing PhenoAge components -> phenoage is NA when any required biomarker is missing;
# (b) missing CKD modelling components -> ckd is NA when both eGFR and ACR cannot be computed.
n2a <- sum(dat$age >= config$min_age & !is.na(dat$phenoage), na.rm = TRUE)
n2b <- sum(dat$age >= config$min_age & !is.na(dat$phenoage) & !is.na(dat$ckd), na.rm = TRUE)
n3 <- nrow(analytic)
n4 <- if (exists("qdat")) nrow(qdat) else NA_integer_
n5 <- if (exists("analytic_qg")) nrow(analytic_qg) else NA_integer_
n6 <- if (exists("qdat_strict")) nrow(qdat_strict) else NA_integer_
n7 <- if (exists("n_modelb")) n_modelb else NA_integer_
n8 <- if (exists("n_modelb_primary")) n_modelb_primary else NA_integer_
mi_label_step <- if (exists("mice_active") && isTRUE(mice_active)) {
  paste0("Model B PRIMARY estimation sample (MICE m=", config$mice_m, "; covariates imputed)")
} else {
  "Model B PRIMARY estimation sample (complete-case; MICE off)"
}

flow <- tibble(
  step = c(
    "Downloaded NHANES participants (selected cycles)",
    paste0("Age >= ", config$min_age),
    "Non-missing PhenoAge components (all 9 biomarkers required)",
    "Non-missing CKD modelling components (eGFR or ACR available)",
    "Non-missing selected weight, PSU, and strata (analytic sample)",
    "Primary interaction model sample (fixed 4-PFAS complete-case score + design vars)",
    "Complete-case qgcomp sample",
    "Strict complete-case (qgcomp-compatible PFAS subset)",
    "Model B complete-case estimation sample (covariate complete-case)",
    mi_label_step
  ),
  included = c(n0, n1, n2a, n2b, n3, n4, n5, n6, n7, n8)
```

```r
) %>%
  mutate(
    excluded_at_step = pmax(lag(included, default = first(included)) - included, 0),
    excluded_at_step = if_else(row_number() == 1, NA_integer_, as.integer(excluded_at_step)),
    excluded_label = ifelse(is.na(excluded_at_step), "", paste0("Excluded: ", excluded_at_step))
  )

kable(flow, caption = "Sample flow counts (PRISMA-style): inclusion and exclusion at each analytic step
```

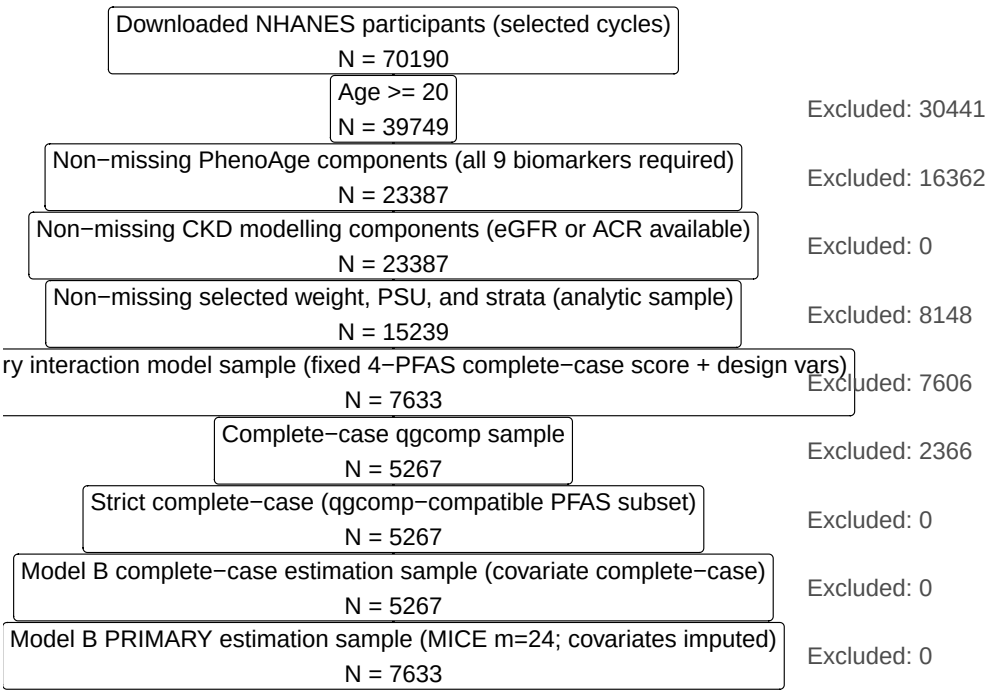
Table 27: Sample flow counts (PRISMA-style): inclusion and exclusion at each analytic step

step	included	excluded_at_step	excluded_label
Downloaded NHANES participants (selected cycles)	70190	NA	
Age >= 20	39749	30441	Excluded: 30441
Non-missing PhenoAge components (all 9 biomarkers required)	23387	16362	Excluded: 16362
Non-missing CKD modelling components (eGFR or ACR available)	23387	0	Excluded: 0
Non-missing selected weight, PSU, and strata (analytic sample)	15239	8148	Excluded: 8148
Primary interaction model sample (fixed 4-PFAS complete-case score + design vars)	7633	7606	Excluded: 7606
Complete-case qgcomp sample	5267	2366	Excluded: 2366
Strict complete-case (qgcomp-compatible PFAS subset)	5267	0	Excluded: 0
Model B complete-case estimation sample (covariate complete-case)	5267	0	Excluded: 0
Model B PRIMARY estimation sample (MICE m=24; covariates imputed)	7633	0	Excluded: 0

```r
flow_plot <- flow %>%
  mutate(
    y = rev(seq_len(n())),
    box_label = paste0(step, "\nN = ", included)
  )

ggplot(flow_plot, aes(x = 1, y = y)) +
  geom_segment(
    data = flow_plot %>% filter(y > min(y)),
    aes(x = 1, xend = 1, y = y - 0.42, yend = y - 0.95),
    arrow = arrow(length = unit(0.12, "inches")),
    linewidth = 0.5
  ) +
  geom_label(aes(label = box_label), size = 3.2, label.size = 0.2, fill = "white") +
  geom_text(aes(x = 1.62, label = excluded_label), hjust = 0, size = 3.1, color = "gray30") +
  xlim(0.5, 2.2) +
  ylim(0.5, max(flow_plot$y) + 0.8) +
  labs(
    title = "Participant flow from downloaded sample to analytic samples",
    subtitle = "PRISMA-style flow for NHANES analytic cohort development",
```

```
    x = NULL, y = NULL
  ) +
  theme_void(base_size = 11)
```



PFAS Exposure Distributions

```
# Raw serum PFAS (ng/mL) for the four primary chemicals used in the mixture.
pfas_raw_map <- c(
  "PFOS"  = "LBXPFOS",
  "PFOA"  = "LBXPFOA",
  "PFNA"  = "LBXPFNA",
  "PFHxS" = "LBXPFHS"
)
pfas_present <- pfas_raw_map[pfas_raw_map %in% names(analytic)]

# Wide -> long for plotting (one panel per chemical) using the analytic sample.
# Coerce all PFAS columns to numeric first (some NHANES vars arrive as character).
pfas_wide <- analytic %>%
  dplyr::select(dplyr::any_of(unname(pfas_present))) %>%
  dplyr::mutate(dplyr::across(dplyr::everything(), function(x) {
    suppressWarnings(as.numeric(trimws(as.character(x))))
  }))

pfas_long <- pfas_wide %>%
```

```r
  tidyr::pivot_longer(dplyr::everything(), names_to = "var", values_to = "value") %>%
  dplyr::filter(!is.na(value), value > 0) %>%
  dplyr::mutate(chemical = factor(names(pfas_present)[match(var, pfas_present)],
                                  levels = names(pfas_present)))

# Geometric mean / SD helpers (applied on positive values only)
gm_mean <- function(x) {
  x <- x[is.finite(x) & x > 0]
  if (!length(x)) NA_real_ else exp(mean(log(x)))
}
gm_sd <- function(x) {
  x <- x[is.finite(x) & x > 0]
  if (length(x) < 2) NA_real_ else exp(stats::sd(log(x)))
}

pfas_desc <- pfas_long %>%
  dplyr::group_by(Chemical = chemical) %>%
  dplyr::summarise(
    N        = sum(!is.na(value)),
    Mean     = mean(value, na.rm = TRUE),
    SD       = stats::sd(value, na.rm = TRUE),
    Median   = stats::median(value, na.rm = TRUE),
    Q1       = stats::quantile(value, 0.25, na.rm = TRUE),
    Q3       = stats::quantile(value, 0.75, na.rm = TRUE),
    IQR      = Q3 - Q1,
    Min      = min(value, na.rm = TRUE),
    Max      = max(value, na.rm = TRUE),
    GeoMean  = gm_mean(value),
    GeoSD    = gm_sd(value),
    .groups  = "drop"
  ) %>%
  dplyr::mutate(dplyr::across(c(Mean, SD, Median, Q1, Q3, IQR, Min, Max, GeoMean, GeoSD),
                              ~ round(.x, 3)))

knitr::kable(
  pfas_desc,
  caption = "PFAS descriptive statistics in the analytic sample (serum, ng/mL). Geometric mean / SD com
)
```

Table 28: PFAS descriptive statistics in the analytic sample (serum, ng/mL). Geometric mean / SD computed on positive values only.

Chemical	N	Mean	SD	Median	Q1	Q3	IQR	Min	Max	GeoMean	GeoSD
PFOS	7633	13.752	15.567	9.400	4.80	17.300	12.500	0.140	281.000	8.801	2.706
PFOA	7633	3.361	3.120	2.600	1.47	4.400	2.930	0.070	104.000	2.473	2.269
PFNA	7633	1.205	1.247	0.902	0.50	1.476	0.976	0.058	25.748	0.862	2.329
PFHxS	7633	2.213	2.884	1.500	0.80	2.600	1.800	0.070	81.600	1.403	2.670

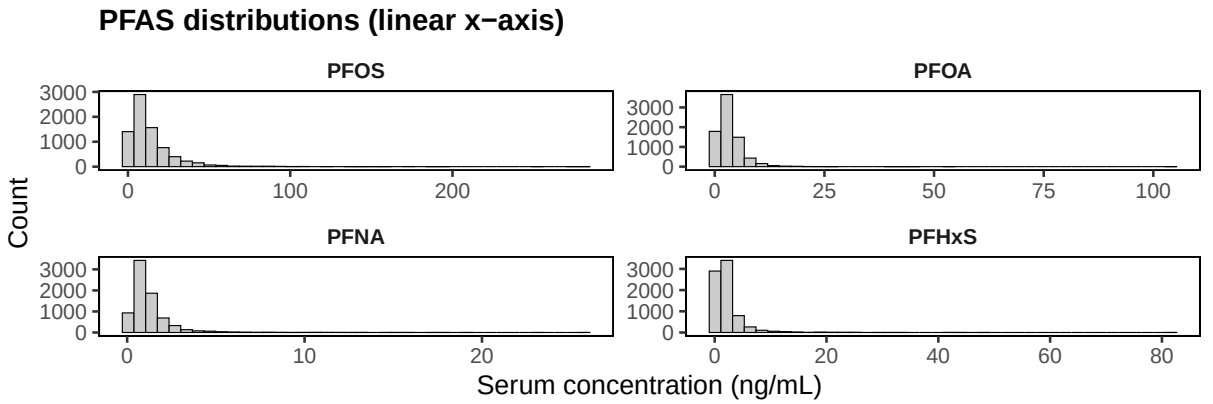
```r
# Histogram panels (2x2): export both linear axis (true raw scale) and log10-axis (skewness readability
hist_base <- function() {
  ggplot(pfas_long, aes(x = value)) +
    geom_histogram(bins = 40, fill = "grey80", color = "black", linewidth = 0.2) +
```

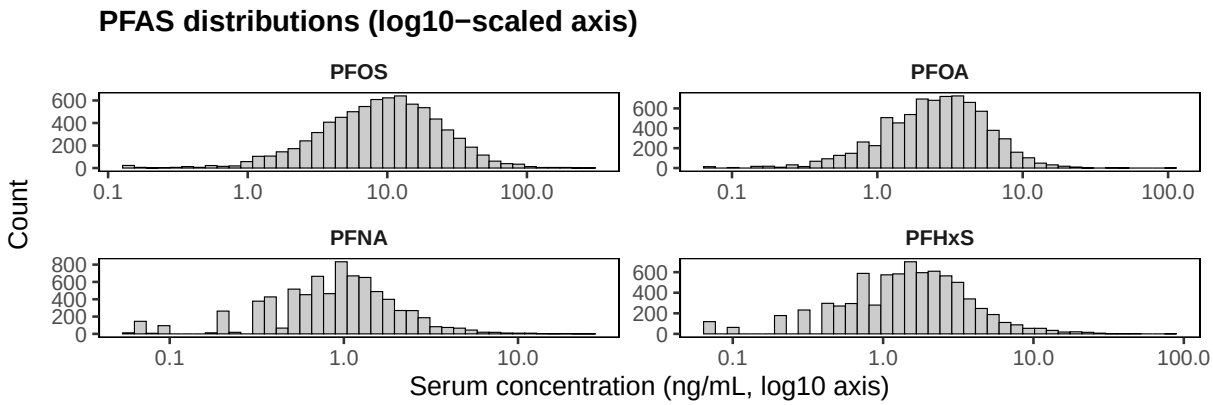
```r
    facet_wrap(~ chemical, ncol = 2, scales = "free") +
    theme_bw(base_size = 10) +
    theme(panel.grid.major = element_blank(),
          panel.grid.minor = element_blank(),
          panel.border = element_rect(color = "black", fill = NA, linewidth = 0.4),
          strip.background = element_blank(),
          strip.text = element_text(face = "bold"))
}
pfas_hist_linear <- hist_base() +
  labs(x = "Serum concentration (ng/mL)", y = "Count",
       title = "PFAS distributions (linear x-axis)")
pfas_hist_log10 <- hist_base() +
  scale_x_log10() +
  labs(x = "Serum concentration (ng/mL, log10 axis)", y = "Count",
       title = "PFAS distributions (log10-scaled axis)")

dir.create("figures", showWarnings = FALSE)
ggsave("figures/fig0_pfas_distributions_linear.jpg", plot = pfas_hist_linear,
       width = 8.5, height = 5.5, dpi = 300, device = "jpeg",
       quality = 95, bg = "white")
ggsave("figures/fig0_pfas_distributions_log10.jpg", plot = pfas_hist_log10,
       width = 8.5, height = 5.5, dpi = 300, device = "jpeg",
       quality = 95, bg = "white")
# Backwards-compatible alias (linear = raw concentrations)
file.copy("figures/fig0_pfas_distributions_linear.jpg",
          "figures/fig0_pfas_distributions.jpg", overwrite = TRUE)
```

```
## [1] TRUE
```

```r
pfas_hist_linear / pfas_hist_log10 +
  plot_layout(ncol = 1) &
  theme(plot.title = element_text(face = "bold", size = 11))
```





Tables

```
fmt <- function(x, digits = 3) {
  ifelse(
    is.na(x),
    "NA",
    ifelse(
      x != 0 & abs(x) < 10^-digits,
      format(x, scientific = TRUE, digits = 2),
      format(round(x, digits), nsmall = digits)
    )
  )
}
fmt_p <- function(x) ifelse(is.na(x), "NA", ifelse(x < 0.001, "<0.001", format(round(x, 3), nsmall = 3)

# Table 1: characteristics by CKD -- continuous: mean (SD); categorical: n (weighted %)
t1_mean_sd <- function(formula, design) {
  est <- svymean(formula, design, na.rm = TRUE)
  v_obj <- svyvar(formula, design, na.rm = TRUE)
  sd_val <- sqrt(as.numeric(coef(v_obj)[1]))
  paste0(format(round(coef(est)[1], 2), nsmall = 2), " (", format(round(sd_val, 2), nsmall = 2), ")")
}
t1_mean_obj <- function(formula, design) {
  m <- svymean(formula, design, na.rm = TRUE)
```

```
  v <- svyvar(formula, design, na.rm = TRUE)
  list(est = unname(coef(m)[1]), se = unname(SE(m)[1]), var = unname(coef(v)[1]))
}
t1_pct_se <- function(formula, design) {
  est <- svymean(formula, design, na.rm = TRUE)
  cf <- coef(est)
  se <- SE(est)
  idx <- 1
  if (length(cf) > 1) {
    nms <- names(cf)
    idx_true <- which(str_detect(nms, "TRUE$"))
    if (length(idx_true) == 0) idx_true <- which(str_detect(nms, "1$"))
    if (length(idx_true) > 0) idx <- idx_true[1]
  }
  paste0(
    format(round(unname(cf[idx]) * 100, 1), nsmall = 1),
    " (",
    format(round(unname(se[idx]) * 100, 1), nsmall = 1),
    ")"
  )
}
t1_n_wpct <- function(cond_vec, formula, design) {
  n_unwt <- sum(cond_vec, na.rm = TRUE)
  est <- svymean(formula, design, na.rm = TRUE)
  cf <- coef(est)
  idx <- 1
  if (length(cf) > 1) {
    nms <- names(cf)
    idx_true <- which(str_detect(nms, "TRUE$"))
    if (length(idx_true) == 0) idx_true <- which(str_detect(nms, "1$"))
    if (length(idx_true) > 0) idx <- idx_true[1]
  }
  paste0(
    format(n_unwt, big.mark = ","),
    " (",
    format(round(unname(cf[idx]) * 100, 1), nsmall = 1),
    ")"
  )
}
t1_pct_obj <- function(formula, design) {
  est <- svymean(formula, design, na.rm = TRUE)
  cf <- coef(est)
  se <- SE(est)
  idx <- 1
  if (length(cf) > 1) {
    nms <- names(cf)
    idx_true <- which(str_detect(nms, "TRUE$"))
    if (length(idx_true) == 0) idx_true <- which(str_detect(nms, "1$"))
    if (length(idx_true) > 0) idx <- idx_true[1]
  }
  list(est = unname(cf[idx]), se = unname(se[idx]))
}
t1_factor_level_pct_se <- function(var, level, design) {
```

```r
  est <- svymean(as.formula(paste0("~factor(", var, ")")), design, na.rm = TRUE)
  key <- paste0("factor(", var, ")", level)
  if (!(key %in% names(coef(est)))) return("NA")
  paste0(
    format(round(unname(coef(est)[key]) * 100, 1), nsmall = 1),
    " (",
    format(round(unname(SE(est)[key]) * 100, 1), nsmall = 1),
    ")"
  )
}
t1_factor_level_n_wpct <- function(var, level, design, data) {
  n_unwt <- sum(data[[var]] == level, na.rm = TRUE)
  est <- svymean(as.formula(paste0("~factor(", var, ")")), design, na.rm = TRUE)
  key <- paste0("factor(", var, ")", level)
  if (!(key %in% names(coef(est)))) {
    return(paste0(format(n_unwt, big.mark = ","), " (NA)"))
  }
  paste0(
    format(n_unwt, big.mark = ","),
    " (",
    format(round(unname(coef(est)[key]) * 100, 1), nsmall = 1),
    ")"
  )
}
t1_factor_level_obj <- function(var, level, design) {
  est <- svymean(as.formula(paste0("~factor(", var, ")")), design, na.rm = TRUE)
  key <- paste0("factor(", var, ")", level)
  if (!(key %in% names(coef(est)))) return(list(est = NA_real_, se = NA_real_))
  list(est = unname(coef(est)[key]), se = unname(SE(est)[key]))
}
# Missing category: weighted % of rows with NA on `var` (for MICE / missingness diagnostics).
t1_miss_pct_obj <- function(var, design) {
  t1_pct_obj(as.formula(paste0("~I(is.na(", var, "))")), design)
}
t1_miss_n_wpct <- function(var, design, data_sub) {
  n_unwt <- sum(is.na(data_sub[[var]]))
  o <- t1_miss_pct_obj(var, design)
  paste0(
    format(n_unwt, big.mark = ","),
    " (",
    format(round(o$est * 100, 1), nsmall = 1),
    ")"
  )
}
smd_cont <- function(o0, o1) {
  if (any(!is.finite(c(o0$est, o1$est, o0$var, o1$var)))) return(NA_real_)
  denom <- sqrt((o0$var + o1$var) / 2)
  if (!is.finite(denom) || denom <= 0) return(NA_real_)
  abs(o1$est - o0$est) / denom
}
smd_prop <- function(o0, o1) {
  if (any(!is.finite(c(o0$est, o1$est)))) return(NA_real_)
  denom <- sqrt((o0$est * (1 - o0$est) + o1$est * (1 - o1$est)) / 2)
```

```r
  if (!is.finite(denom) || denom <= 0) return(NA_real_)
  abs(o1$est - o0$est) / denom
}
fmt_smd <- function(x) ifelse(is.na(x), "", format(round(x, 3), nsmall = 3))

# Build Part A (Model B PRIMARY estimation sample = MICE base; PFAS+outcome+design complete) and
# Part B (analytic-sample participants excluded from primary inference: missing PFAS / outcome / design
# Within Part A we display Missing rows so the reader sees exactly which covariate cells are imputed by
if ("SEQN" %in% names(analytic) && exists("qdat_svy") && "SEQN" %in% names(qdat_svy)) {
  inc_idx <- analytic$SEQN %in% qdat_svy$SEQN
} else {
  inc_idx <- !is.na(analytic$ckd) & !is.na(analytic$phenoage_accel)
}
analytic_inc <- analytic[inc_idx, , drop = FALSE]
analytic_exc <- analytic[!inc_idx, , drop = FALSE]

svy_inc <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined,
                     nest = TRUE, data = analytic_inc)
svy_exc <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined,
                     nest = TRUE, data = analytic_exc)

# Block builder: returns a tibble with columns characteristic, No CKD, CKD, SMD.
# Continuous: mean (SD).  Categorical: n (weighted %).  No SE displayed anywhere.
build_t1_block <- function(design, data) {
  d0 <- subset(design, ckd == 0)
  d1 <- subset(design, ckd == 1)
  data0 <- data[!is.na(data$ckd) & data$ckd == 0, , drop = FALSE]
  data1 <- data[!is.na(data$ckd) & data$ckd == 1, , drop = FALSE]

  t1_show_miss <- function(var) var %in% names(data) && any(is.na(data[[var]]))

  age0 <- t1_mean_obj(~age, d0); age1 <- t1_mean_obj(~age, d1)
  age_miss0 <- t1_miss_pct_obj("age", d0); age_miss1 <- t1_miss_pct_obj("age", d1)
  pir0 <- t1_mean_obj(~pir, d0); pir1 <- t1_mean_obj(~pir, d1)
  pir_miss0 <- t1_miss_pct_obj("pir", d0); pir_miss1 <- t1_miss_pct_obj("pir", d1)
  bmi0 <- t1_mean_obj(~bmi, d0); bmi1 <- t1_mean_obj(~bmi, d1)
  bmi_miss0 <- t1_miss_pct_obj("bmi", d0); bmi_miss1 <- t1_miss_pct_obj("bmi", d1)

  fem0 <- t1_pct_obj(~I(sex == "Female"), d0); fem1 <- t1_pct_obj(~I(sex == "Female"), d1)
  sex_miss0 <- t1_miss_pct_obj("sex", d0); sex_miss1 <- t1_miss_pct_obj("sex", d1)
  his0 <- t1_factor_level_obj("race_eth", "Hispanic", d0); his1 <- t1_factor_level_obj("race_eth", "His
  whi0 <- t1_factor_level_obj("race_eth", "Non-Hispanic White", d0); whi1 <- t1_factor_level_obj("race_
  blk0 <- t1_factor_level_obj("race_eth", "Non-Hispanic Black", d0); blk1 <- t1_factor_level_obj("race_
  asn0 <- t1_factor_level_obj("race_eth", "Non-Hispanic Asian", d0); asn1 <- t1_factor_level_obj("race_
  oth0 <- t1_factor_level_obj("race_eth", "Other/Multi", d0); oth1 <- t1_factor_level_obj("race_eth", "
  race_miss0 <- t1_miss_pct_obj("race_eth", d0); race_miss1 <- t1_miss_pct_obj("race_eth", d1)
  edu_lths0 <- t1_factor_level_obj("education", "<HS", d0); edu_lths1 <- t1_factor_level_obj("education
  edu_hs0   <- t1_factor_level_obj("education", "HS/GED", d0); edu_hs1   <- t1_factor_level_obj("educati
  edu_gt0   <- t1_factor_level_obj("education", ">HS", d0); edu_gt1   <- t1_factor_level_obj("education
  edu_miss0 <- t1_miss_pct_obj("education", d0); edu_miss1 <- t1_miss_pct_obj("education", d1)
  smk_never0 <- t1_factor_level_obj("smoker3", "Never",   d0); smk_never1 <- t1_factor_level_obj("smoke
  smk_form0  <- t1_factor_level_obj("smoker3", "Former",  d0); smk_form1  <- t1_factor_level_obj("smoke
  smk_curr0  <- t1_factor_level_obj("smoker3", "Current", d0); smk_curr1  <- t1_factor_level_obj("smoke
```

```r
smk_miss0 <- t1_miss_pct_obj("smoker3", d0); smk_miss1 <- t1_miss_pct_obj("smoker3", d1)
alc_never0 <- t1_factor_level_obj("alcohol3", "Never",   d0); alc_never1 <- t1_factor_level_obj("alcol
alc_form0  <- t1_factor_level_obj("alcohol3", "Former",  d0); alc_form1  <- t1_factor_level_obj("alcol
alc_curr0  <- t1_factor_level_obj("alcohol3", "Current", d0); alc_curr1  <- t1_factor_level_obj("alcol
alc_miss0 <- t1_miss_pct_obj("alcohol3", d0); alc_miss1 <- t1_miss_pct_obj("alcohol3", d1)
pa0  <- t1_pct_obj(~I(phys_active == 1), d0); pa1  <- t1_pct_obj(~I(phys_active == 1), d1)
pa_miss0 <- t1_miss_pct_obj("phys_active", d0); pa_miss1 <- t1_miss_pct_obj("phys_active", d1)
dm0  <- t1_pct_obj(~I(diabetes == 1), d0); dm1  <- t1_pct_obj(~I(diabetes == 1), d1)
dm_miss0 <- t1_miss_pct_obj("diabetes", d0); dm_miss1 <- t1_miss_pct_obj("diabetes", d1)
htn0 <- t1_pct_obj(~I(hypertension == 1), d0); htn1 <- t1_pct_obj(~I(hypertension == 1), d1)
htn_miss0 <- t1_miss_pct_obj("hypertension", d0); htn_miss1 <- t1_miss_pct_obj("hypertension", d1)
cvd0 <- t1_pct_obj(~I(cvd == 1), d0); cvd1 <- t1_pct_obj(~I(cvd == 1), d1)
cvd_miss0 <- t1_miss_pct_obj("cvd", d0); cvd_miss1 <- t1_miss_pct_obj("cvd", d1)

show_age_miss <- t1_show_miss("age")
show_pir_miss <- t1_show_miss("pir")
show_bmi_miss <- t1_show_miss("bmi")
show_sex_miss <- t1_show_miss("sex")
show_race_miss <- t1_show_miss("race_eth")
show_edu_miss <- t1_show_miss("education")
show_smk_miss <- t1_show_miss("smoker3")
show_alc_miss <- t1_show_miss("alcohol3")
show_pa_miss <- t1_show_miss("phys_active")
show_dm_miss <- t1_show_miss("diabetes")
show_htn_miss <- t1_show_miss("hypertension")
show_cvd_miss <- t1_show_miss("cvd")

tibble(
  characteristic = c(
    "N (unweighted)", "Age, mean (SD)",
    if (show_age_miss) "Age missing, n (weighted %)" else character(0),
    "Female, n (weighted %)",
    if (show_sex_miss) "Sex missing, n (weighted %)" else character(0),
    "Hispanic, n (weighted %)", "Non-Hispanic White, n (weighted %)", "Non-Hispanic Black, n (weighte
    "Non-Hispanic Asian, n (weighted %)", "Other/Multi, n (weighted %)",
    if (show_race_miss) "Race/ethnicity missing, n (weighted %)" else character(0),
    "Education <HS, n (weighted %)", "Education HS/GED, n (weighted %)", "Education >HS, n (weighted 
    if (show_edu_miss) "Education missing, n (weighted %)" else character(0),
    "PIR, mean (SD)",
    if (show_pir_miss) "PIR missing, n (weighted %)" else character(0),
    "BMI, mean (SD)",
    if (show_bmi_miss) "BMI missing, n (weighted %)" else character(0),
    "Never smoker, n (weighted %)", "Former smoker, n (weighted %)", "Current smoker, n (weighted %)"
    if (show_smk_miss) "Smoking status missing, n (weighted %)" else character(0),
    "Never drinker, n (weighted %)", "Former drinker, n (weighted %)", "Current drinker, n (weighted 
    if (show_alc_miss) "Alcohol status missing, n (weighted %)" else character(0),
    "Physically active, n (weighted %)",
    if (show_pa_miss) "Physical activity missing, n (weighted %)" else character(0),
    "Diabetes, n (weighted %)",
    if (show_dm_miss) "Diabetes missing, n (weighted %)" else character(0),
    "Hypertension, n (weighted %)",
    if (show_htn_miss) "Hypertension missing, n (weighted %)" else character(0),
    "CVD history, n (weighted %)",
```

```
    if (show_cvd_miss) "CVD history missing, n (weighted %)" else character(0)
  ),
  `No CKD` = c(
    format(nrow(data0), big.mark = ","),
    t1_mean_sd(~age, d0),
    if (show_age_miss) t1_miss_n_wpct("age", d0, data0) else character(0),
    t1_n_wpct(data0$sex == "Female", ~I(sex == "Female"), d0),
    if (show_sex_miss) t1_miss_n_wpct("sex", d0, data0) else character(0),
    t1_factor_level_n_wpct("race_eth", "Hispanic", d0, data0),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic White", d0, data0),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic Black", d0, data0),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic Asian", d0, data0),
    t1_factor_level_n_wpct("race_eth", "Other/Multi", d0, data0),
    if (show_race_miss) t1_miss_n_wpct("race_eth", d0, data0) else character(0),
    t1_factor_level_n_wpct("education", "<HS", d0, data0),
    t1_factor_level_n_wpct("education", "HS/GED", d0, data0),
    t1_factor_level_n_wpct("education", ">HS", d0, data0),
    if (show_edu_miss) t1_miss_n_wpct("education", d0, data0) else character(0),
    t1_mean_sd(~pir, d0),
    if (show_pir_miss) t1_miss_n_wpct("pir", d0, data0) else character(0),
    t1_mean_sd(~bmi, d0),
    if (show_bmi_miss) t1_miss_n_wpct("bmi", d0, data0) else character(0),
    t1_factor_level_n_wpct("smoker3", "Never",   d0, data0),
    t1_factor_level_n_wpct("smoker3", "Former",  d0, data0),
    t1_factor_level_n_wpct("smoker3", "Current", d0, data0),
    if (show_smk_miss) t1_miss_n_wpct("smoker3", d0, data0) else character(0),
    t1_factor_level_n_wpct("alcohol3", "Never",   d0, data0),
    t1_factor_level_n_wpct("alcohol3", "Former",  d0, data0),
    t1_factor_level_n_wpct("alcohol3", "Current", d0, data0),
    if (show_alc_miss) t1_miss_n_wpct("alcohol3", d0, data0) else character(0),
    t1_n_wpct(data0$phys_active == 1, ~I(phys_active == 1), d0),
    if (show_pa_miss) t1_miss_n_wpct("phys_active", d0, data0) else character(0),
    t1_n_wpct(data0$diabetes == 1, ~I(diabetes == 1), d0),
    if (show_dm_miss) t1_miss_n_wpct("diabetes", d0, data0) else character(0),
    t1_n_wpct(data0$hypertension == 1, ~I(hypertension == 1), d0),
    if (show_htn_miss) t1_miss_n_wpct("hypertension", d0, data0) else character(0),
    t1_n_wpct(data0$cvd == 1, ~I(cvd == 1), d0),
    if (show_cvd_miss) t1_miss_n_wpct("cvd", d0, data0) else character(0)
  ),
  CKD = c(
    format(nrow(data1), big.mark = ","),
    t1_mean_sd(~age, d1),
    if (show_age_miss) t1_miss_n_wpct("age", d1, data1) else character(0),
    t1_n_wpct(data1$sex == "Female", ~I(sex == "Female"), d1),
    if (show_sex_miss) t1_miss_n_wpct("sex", d1, data1) else character(0),
    t1_factor_level_n_wpct("race_eth", "Hispanic", d1, data1),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic White", d1, data1),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic Black", d1, data1),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic Asian", d1, data1),
    t1_factor_level_n_wpct("race_eth", "Other/Multi", d1, data1),
    if (show_race_miss) t1_miss_n_wpct("race_eth", d1, data1) else character(0),
    t1_factor_level_n_wpct("education", "<HS", d1, data1),
    t1_factor_level_n_wpct("education", "HS/GED", d1, data1),
```

```
    t1_factor_level_n_wpct("education", ">HS", d1, data1),
    if (show_edu_miss) t1_miss_n_wpct("education", d1, data1) else character(0),
    t1_mean_sd(~pir, d1),
    if (show_pir_miss) t1_miss_n_wpct("pir", d1, data1) else character(0),
    t1_mean_sd(~bmi, d1),
    if (show_bmi_miss) t1_miss_n_wpct("bmi", d1, data1) else character(0),
    t1_factor_level_n_wpct("smoker3", "Never",   d1, data1),
    t1_factor_level_n_wpct("smoker3", "Former",  d1, data1),
    t1_factor_level_n_wpct("smoker3", "Current", d1, data1),
    if (show_smk_miss) t1_miss_n_wpct("smoker3", d1, data1) else character(0),
    t1_factor_level_n_wpct("alcohol3", "Never",   d1, data1),
    t1_factor_level_n_wpct("alcohol3", "Former",  d1, data1),
    t1_factor_level_n_wpct("alcohol3", "Current", d1, data1),
    if (show_alc_miss) t1_miss_n_wpct("alcohol3", d1, data1) else character(0),
    t1_n_wpct(data1$phys_active == 1, ~I(phys_active == 1), d1),
    if (show_pa_miss) t1_miss_n_wpct("phys_active", d1, data1) else character(0),
    t1_n_wpct(data1$diabetes == 1, ~I(diabetes == 1), d1),
    if (show_dm_miss) t1_miss_n_wpct("diabetes", d1, data1) else character(0),
    t1_n_wpct(data1$hypertension == 1, ~I(hypertension == 1), d1),
    if (show_htn_miss) t1_miss_n_wpct("hypertension", d1, data1) else character(0),
    t1_n_wpct(data1$cvd == 1, ~I(cvd == 1), d1),
    if (show_cvd_miss) t1_miss_n_wpct("cvd", d1, data1) else character(0)
  ),
  SMD = c(
    "",
    fmt_smd(smd_cont(age0, age1)),
    if (show_age_miss) fmt_smd(smd_prop(age_miss0, age_miss1)) else character(0),
    fmt_smd(smd_prop(fem0, fem1)),
    if (show_sex_miss) fmt_smd(smd_prop(sex_miss0, sex_miss1)) else character(0),
    fmt_smd(smd_prop(his0, his1)),
    fmt_smd(smd_prop(whi0, whi1)),
    fmt_smd(smd_prop(blk0, blk1)),
    fmt_smd(smd_prop(asn0, asn1)),
    fmt_smd(smd_prop(oth0, oth1)),
    if (show_race_miss) fmt_smd(smd_prop(race_miss0, race_miss1)) else character(0),
    fmt_smd(smd_prop(edu_lths0, edu_lths1)),
    fmt_smd(smd_prop(edu_hs0, edu_hs1)),
    fmt_smd(smd_prop(edu_gt0, edu_gt1)),
    if (show_edu_miss) fmt_smd(smd_prop(edu_miss0, edu_miss1)) else character(0),
    fmt_smd(smd_cont(pir0, pir1)),
    if (show_pir_miss) fmt_smd(smd_prop(pir_miss0, pir_miss1)) else character(0),
    fmt_smd(smd_cont(bmi0, bmi1)),
    if (show_bmi_miss) fmt_smd(smd_prop(bmi_miss0, bmi_miss1)) else character(0),
    fmt_smd(smd_prop(smk_never0, smk_never1)),
    fmt_smd(smd_prop(smk_form0,  smk_form1)),
    fmt_smd(smd_prop(smk_curr0,  smk_curr1)),
    if (show_smk_miss) fmt_smd(smd_prop(smk_miss0, smk_miss1)) else character(0),
    fmt_smd(smd_prop(alc_never0, alc_never1)),
    fmt_smd(smd_prop(alc_form0,  alc_form1)),
    fmt_smd(smd_prop(alc_curr0,  alc_curr1)),
    if (show_alc_miss) fmt_smd(smd_prop(alc_miss0, alc_miss1)) else character(0),
    fmt_smd(smd_prop(pa0, pa1)),
    if (show_pa_miss) fmt_smd(smd_prop(pa_miss0, pa_miss1)) else character(0),
```

```r
      fmt_smd(smd_prop(dm0, dm1)),
      if (show_dm_miss) fmt_smd(smd_prop(dm_miss0, dm_miss1)) else character(0),
      fmt_smd(smd_prop(htn0, htn1)),
      if (show_htn_miss) fmt_smd(smd_prop(htn_miss0, htn_miss1)) else character(0),
      fmt_smd(smd_prop(cvd0, cvd1)),
      if (show_cvd_miss) fmt_smd(smd_prop(cvd_miss0, cvd_miss1)) else character(0)
    )
  )
}

block_a <- build_t1_block(svy_inc, analytic_inc)
block_b <- build_t1_block(svy_exc, analytic_exc)

n_inc <- nrow(analytic_inc)
n_exc <- nrow(analytic_exc)
n_inc_0 <- sum(analytic_inc$ckd == 0, na.rm = TRUE)
n_inc_1 <- sum(analytic_inc$ckd == 1, na.rm = TRUE)
n_exc_0 <- sum(analytic_exc$ckd == 0, na.rm = TRUE)
n_exc_1 <- sum(analytic_exc$ckd == 1, na.rm = TRUE)

header_a <- tibble(
  characteristic = "PART A -- INCLUDED IN STUDY",
  `No CKD` = "", CKD = "", SMD = ""
)
header_b <- tibble(
  characteristic = "PART B -- EXCLUDED FROM STUDY",
  `No CKD` = "", CKD = "", SMD = ""
)

table1 <- bind_rows(header_a, block_a, header_b, block_b)

kable(
  table1,
  caption = "Table 1. Sample characteristics by CKD status. Part A = MICE base sample (Model B PRIMARY e
)
```

Table 29: Table 1. Sample characteristics by CKD status. Part A = MICE base sample (Model B PRIMARY estimation sample; PFAS + outcome + design complete). Missing rows show pre-imputation missingness on each variable shown (continuous and categorical alike); those cells are then imputed under MICE for primary inference. Continuous: mean (SD); categorical: n (weighted %); Missing rows: n (weighted %) of NA. SMD is the absolute standardized mean difference between CKD vs No CKD within each part.

characteristic	No CKD	CKD	SMD
PART A – INCLUDED IN STUDY			
N (unweighted)	6,315	1,318	
Age, mean (SD)	45.27 (15.87)	61.34 (17.10)	0.974
Female, n (weighted %)	3,279 (51.0)	689 (56.9)	0.117
Hispanic, n (weighted %)	1,747 (14.1)	298 (12.5)	0.045
Non-Hispanic White, n (weighted %)	2,699 (67.8)	594 (65.1)	0.058

characteristic	No CKD	CKD	SMD
Non-Hispanic Black, n (weighted %)	1,259 (10.4)	320 (14.3)	0.119
Non-Hispanic Asian, n (weighted %)	319 (2.4)	44 (2.1)	0.021
Other/Multi, n (weighted %)	291 (5.3)	62 (6.0)	0.028
Education <HS, n (weighted %)	672 (5.6)	217 (11.9)	0.222
Education HS/GED, n (weighted %)	1,420 (25.8)	304 (30.0)	0.094
Education >HS, n (weighted %)	3,316 (68.6)	581 (58.1)	0.218
Education missing, n (weighted %)	907 (10.3)	216 (12.6)	0.072
PIR, mean (SD)	3.12 (1.63)	2.66 (1.59)	0.284
PIR missing, n (weighted %)	594 (7.3)	137 (9.0)	0.063
BMI, mean (SD)	28.69 (6.83)	30.55 (7.43)	0.260
BMI missing, n (weighted %)	50 (0.5)	38 (2.8)	0.178
Never smoker, n (weighted %)	3,571 (56.6)	651 (52.7)	0.079
Former smoker, n (weighted %)	1,403 (22.3)	462 (33.4)	0.249
Current smoker, n (weighted %)	1,338 (21.0)	203 (13.9)	0.189
Smoking status missing, n (weighted %)	3 (0.0)	2 (0.1)	0.035
Never drinker, n (weighted %)	779 (12.6)	196 (17.1)	0.126
Former drinker, n (weighted %)	817 (14.3)	318 (28.7)	0.355
Current drinker, n (weighted %)	3,217 (73.1)	477 (54.2)	0.399
Alcohol status missing, n (weighted %)	1,502 (24.9)	327 (27.3)	0.055
Physically active, n (weighted %)	3,219 (58.7)	468 (42.7)	0.324
Physical activity missing, n (weighted %)	26 (0.4)	25 (1.6)	0.121
Diabetes, n (weighted %)	536 (6.4)	430 (29.1)	0.623
Diabetes missing, n (weighted %)	124 (1.7)	48 (3.0)	0.083
Hypertension, n (weighted %)	1,812 (25.5)	846 (61.2)	0.773
Hypertension missing, n (weighted %)	9 (0.1)	3 (0.2)	0.030
CVD history, n (weighted %)	428 (5.4)	348 (23.3)	0.528
CVD history missing, n (weighted %)	19 (0.2)	17 (1.3)	0.129
PART B – EXCLUDED FROM STUDY			
N (unweighted)	6,248	1,358	
Age, mean (SD)	45.13 (15.81)	59.83 (17.27)	0.888
Female, n (weighted %)	3,234 (51.4)	708 (58.3)	0.138
Hispanic, n (weighted %)	1,807 (14.8)	322 (13.4)	0.041
Non-Hispanic White, n (weighted %)	2,638 (66.6)	600 (66.7)	0.002
Non-Hispanic Black, n (weighted %)	1,150 (10.3)	342 (14.3)	0.122
Non-Hispanic Asian, n (weighted %)	353 (2.6)	49 (2.0)	0.040
Other/Multi, n (weighted %)	300 (5.6)	45 (3.6)	0.098
Education <HS, n (weighted %)	666 (6.0)	237 (12.6)	0.230
Education HS/GED, n (weighted %)	1,437 (26.7)	372 (36.6)	0.215
Education >HS, n (weighted %)	3,262 (67.4)	508 (50.8)	0.341
Education missing, n (weighted %)	883 (10.1)	241 (13.5)	0.105
PIR, mean (SD)	3.07 (1.62)	2.56 (1.52)	0.323
PIR missing, n (weighted %)	551 (6.3)	127 (6.8)	0.022
BMI, mean (SD)	28.87 (6.64)	30.25 (8.39)	0.183
BMI missing, n (weighted %)	77 (1.0)	40 (2.0)	0.082
Never smoker, n (weighted %)	3,416 (54.1)	693 (52.5)	0.032
Former smoker, n (weighted %)	1,499 (25.1)	425 (30.8)	0.126
Current smoker, n (weighted %)	1,325 (20.8)	240 (16.8)	0.104
Smoking status missing, n (weighted %)	8 (0.1)	0 (0.0)	0.041
Never drinker, n (weighted %)	726 (12.3)	219 (19.2)	0.189
Former drinker, n (weighted %)	776 (14.3)	290 (23.9)	0.245
Current drinker, n (weighted %)	3,232 (73.4)	507 (56.9)	0.350
Alcohol status missing, n (weighted %)	1,514 (24.4)	342 (24.5)	0.002

characteristic	No CKD	CKD	SMD
Physically active, n (weighted %)	3,169 (57.9)	471 (43.5)	0.289
Physical activity missing, n (weighted %)	25 (0.4)	26 (2.1)	0.155
Diabetes, n (weighted %)	595 (7.2)	458 (27.3)	0.550
Diabetes missing, n (weighted %)	139 (2.1)	38 (3.4)	0.084
Hypertension, n (weighted %)	1,923 (28.4)	871 (58.5)	0.636
Hypertension missing, n (weighted %)	9 (0.1)	4 (0.3)	0.038
CVD history, n (weighted %)	463 (6.1)	376 (23.8)	0.513
CVD history missing, n (weighted %)	24 (0.4)	15 (0.9)	0.066

```
cat("\n*Table 1 note:* Weighted percentages use the NHANES complex survey design (strata, PSU, combined
    "Part A = MICE base sample = participants entering Model B primary inference (non-missing PhenoAge,
    "Within Part A, the Missing rows quantify pre-imputation covariate missingness; those cells are impu
    ") and pooled with Rubin's rules. ",
    "Part B = analytic-sample participants excluded from primary inference because they are missing one
    "SMD in Part B is computed between excluded No CKD vs excluded CKD; values >0.10 are typically cons
```

```
##
## *Table 1 note:* Weighted percentages use the NHANES complex survey design (strata, PSU, combined wei
```

```
# Table 2: overall mixture effect (MI-pooled qgcomp via Rubin when MICE is active; CC fallback otherwis
qg_mi_overall <- if (mice_active) {
  qgcomp_mi_pool(
    formula = f_qg, expnms = qg_exposure_vars,
    imp_list_obj = imp_list, q = q_qg,
    weight_col = "weight_combined"
  )
} else NULL

tab2_mi <- if (!is.null(qg_mi_overall)) {
  tibble(
    model = paste0("Model A: Overall mixture effect [MI-pooled, m = ", qg_mi_overall$m, "]"),
    beta = fmt(qg_mi_overall$psi),
    conf.low = fmt(qg_mi_overall$conf.low),
    conf.high = fmt(qg_mi_overall$conf.high),
    p.value = fmt_p(qg_mi_overall$p.value),
    N = round(mean(qg_mi_overall$n_per_fit)),
    source = "MI-pooled"
  )
} else NULL

tab2_cc <- safe_qgcomp_extract(
  fit_qg_overall,
  if (!is.null(qg_mi_overall)) "Model A: Overall mixture effect [Complete-case sensitivity]"
  else "Model A: Overall mixture effect [Complete-case]",
  nrow(analytic_qg)
) %>%
  mutate(
    beta = fmt(beta),
    conf.low = fmt(conf.low),
    conf.high = fmt(conf.high),
    p.value = fmt_p(p.value),
    source = "Complete-case"
```

```r
  )

tab2_full <- if (!is.null(tab2_mi)) bind_rows(tab2_mi, tab2_cc) else tab2_cc

kable(
  tab2_full,
  caption = paste0(
    "Table 2. Overall association between PFAS mixture score and PhenoAge acceleration (years). ",
    "Mixture score ranges from 1 to 4; a 1-unit increase represents a 1-quantile increase averaged ",
    "across the fixed primary PFAS set. Estimates from qgcomp with normalized pooled NHANES sampling wei
    if (!is.null(qg_mi_overall)) "; primary row is MI-pooled across imputations via Rubin's rules; CC ro
  )
)
```

Table 30: Table 2. Overall association between PFAS mixture score and PhenoAge acceleration (years). Mixture score ranges from 1 to 4; a 1-unit increase represents a 1-quantile increase averaged across the fixed primary PFAS set. Estimates from qgcomp with normalized pooled NHANES sampling weights; primary row is MI-pooled across imputations via Rubin's rules; CC row shown for transparency.

model	beta	conf.low	conf.high	p.value	N	source
Model A: Overall mixture effect [MI-pooled, m = 24]	-1.472	-1.689	-1.255	<0.001	7633	MI-pooled
Model A: Overall mixture effect [Complete-case sensitivity]	-1.258	-1.504	-1.012	<0.001	5267	Complete-case

```r
cat("\n*Table 2 note:* qgcomp model adjusted for the same covariates as Model B: sex, race/ethnicity, PI
    "qgcomp intended covariates: ", paste(qg_covars_intended, collapse = ", "), ". ",
    "qgcomp realized covariates (after complete-case support checks): ", paste(qg_covars_realized, colla
    if (!is.null(qg_mi_overall)) {
      paste0("Primary row uses ", qg_mi_overall$m, " MICE imputations of model covariates; psi (overall
             "qgcomp variance does NOT incorporate PSU/strata; statistical inference for the interaction
    } else if (!is.null(fit_qg_overall)) {
      "Inference here uses qgcomp's analytic variance on the complete-case sample; see Table 3 (svyglm)
    } else {
      paste0("qgcomp failed: ",
             ifelse(is.null(qgcomp_overall_error), "unknown error", qgcomp_overall_error), ". ")
    },
    "PhenoAge acceleration defined as survey-weighted residual of PhenoAge on chronological age.\n", sep
```

```
##
## *Table 2 note:* qgcomp model adjusted for the same covariates as Model B: sex, race/ethnicity, PIR, E
```

```r
# Table 3: interaction results + design-based marginal effects by CKD group
if (!is.null(fit_int)) {
  int_tidy <- tidy_pool(fit_int, conf.level = 0.95)
  cf_names <- names(coef_pool(fit_int))

  int_terms_wanted <- c("mixture_qscore", cf_names[str_detect(cf_names, "^ckd$|mixture_qscore:ckd")])
  tab3_raw <- int_tidy %>%
    filter(term %in% int_terms_wanted) %>%
```

```r
    transmute(
      term,
      beta = fmt(estimate),
      conf.low = fmt(conf.low),
      conf.high = fmt(conf.high),
      p.value = fmt_p(p.value),
      N = n_modelb_primary,
      model = paste0("Model B: Mixture*CKD interaction [", modelb_primary_label, "]")
    )

  # Design-based marginal effects via svycontrast with named contrast vectors (pooled across imputation

  # Robustly identify the interaction term regardless of factor coding.
  int_name <- cf_names[str_detect(cf_names, "mixture_qscore:ckd")]
  if (length(int_name) != 1) {
    warning("Could not uniquely identify mixture*ckd interaction term among: ",
            paste(cf_names, collapse = ", "), ". Skipping marginal effects.")
    tab3 <- tab3_raw
  } else {
    c_ckd0 <- setNames(rep(0, length(cf_names)), cf_names); c_ckd0["mixture_qscore"] <- 1
    c_ckd1 <- setNames(rep(0, length(cf_names)), cf_names); c_ckd1["mixture_qscore"] <- 1; c_ckd1[int_na

    ctr0 <- contrast_pool(fit_int, c_ckd0)
    ctr1 <- contrast_pool(fit_int, c_ckd1)

    make_contrast_row <- function(ctr, label) {
      tibble(term = label, beta = fmt(ctr$estimate),
             conf.low = fmt(ctr$conf.low),
             conf.high = fmt(ctr$conf.high),
             p.value = fmt_p(ctr$p.value),
             N = n_modelb_primary, model = "Marginal effects (design-based)")
    }

    p_int_tidy <- int_tidy %>% filter(term == int_name) %>% pull(p.value)
    p_interaction <- if (length(p_int_tidy) > 0) p_int_tidy[1] else NA_real_

    tab3_marginal <- bind_rows(
      make_contrast_row(ctr0, "Mixture effect among CKD=0"),
      make_contrast_row(ctr1, "Mixture effect among CKD=1"),
      tibble(term = "P-interaction", beta = "", conf.low = "", conf.high = "",
             p.value = fmt_p(p_interaction), N = n_modelb_primary, model = "Marginal effects (design-bas
    )
    tab3 <- bind_rows(tab3_raw, tab3_marginal)
  }
} else {
  tab3 <- tibble(
    term = c("mixture_qscore", "ckd", "mixture_qscore:ckd"),
    beta = "NA", conf.low = "NA", conf.high = "NA", p.value = "NA",
    N = n_modelb_primary, model = "Model B: Mixture*CKD interaction (not estimable)"
  )
}
kable(tab3, caption = "Table 3. Interaction model with design-based marginal effects by CKD group")
```

Table 31: Table 3. Interaction model with design-based marginal effects by CKD group

term	beta	conf.low	conf.high	p.value	N	model
mixture_qscore	-1.070	-1.267	-0.872	<0.001	7633	Model B: Mixture*CKD interaction [MI-pooled (Rubin)]
ckd	8.409	5.570	11.247	<0.001	7633	Model B: Mixture*CKD interaction [MI-pooled (Rubin)]
mixture_qscore:ckd	-1.287	-2.156	-0.418	0.004	7633	Model B: Mixture*CKD interaction [MI-pooled (Rubin)]
Mixture effect among CKD=0	-1.070	-1.267	-0.872	<0.001	7633	Marginal effects (design-based)
Mixture effect among CKD=1	-2.357	-3.259	-1.454	<0.001	7633	Marginal effects (design-based)
P-interaction				0.004	7633	Marginal effects (design-based)

```
cat("\n*Table 3 note:* Model B is a survey-weighted linear model (`svyglm`) with interaction term `mixtu
    "Inference: ", modelb_primary_label, " (N = ", format(n_modelb_primary, big.mark = ","),
    "); complete-case sensitivity N = ", format(n_modelb, big.mark = ","),
    if (mice_active) paste0("; MICE m = ", config$mice_m, " imputations of missing covariates (outcome/e
    "Intended covariates: ", paste(intended_covars_int, collapse = ", "), ". ",
    "Realized covariates (after complete-case support checks): ", paste(realized_covars_int, collapse =
    "Marginal effects for CKD=0 and CKD=1 are estimated using design-based contrasts (`svycontrast`); "
    "for MI runs, contrasts are computed per imputation and pooled with Rubin's rules (mitools::MIcombin
    "95% CI and p-values use t-distribution on Rubin (or survey residual) degrees of freedom.\n", sep =
```

```
##
## *Table 3 note:* Model B is a survey-weighted linear model (‘svyglm‘) with interaction term ‘mixture_q
```

```
# Table 4: sensitivity analyses -- using same covariate set as primary models.
sens_covars_rhs <- paste(realized_covars_int, collapse = " + ")
make_rhs_formula <- function(outcome, main_terms, covar_rhs) {
  rhs <- paste(main_terms, collapse = " + ")
  if (!is.null(covar_rhs) && nzchar(covar_rhs)) rhs <- paste(rhs, covar_rhs, sep = " + ")
  as.formula(paste(outcome, "~", rhs))
}
prep_svy_frame <- function(data_df) {
  data_svy <- as.data.frame(data_df)
  for (v in c("phenoage_accel", "phenoage_nokid", "phenoage_nokid_both", "phenoage_accel_unweighted",
              "mixture_qscore", "age", "pir", "bmi", "weight_combined", "psu", "strata")) {
    if (v %in% names(data_svy)) data_svy[[v]] <- suppressWarnings(as.numeric(as.character(data_svy[[v]]
  }
  if ("ckd" %in% names(data_svy)) data_svy$ckd <- as.integer(data_svy$ckd)
  if ("ckd_sens" %in% names(data_svy)) data_svy$ckd_sens <- as.integer(data_svy$ckd_sens)
  for (v in c("sex", "race_eth", "cycle")) {
    if (v %in% names(data_svy)) data_svy[[v]] <- factor(as.character(data_svy[[v]]))
  }
  if ("smoker3"  %in% names(data_svy)) data_svy$smoker3  <- factor(as.character(data_svy$smoker3),  leve
  if ("alcohol3" %in% names(data_svy)) data_svy$alcohol3 <- factor(as.character(data_svy$alcohol3), leve
  data_svy$sex      <- droplevels(factor(data_svy$sex))
  data_svy$race_eth <- droplevels(factor(data_svy$race_eth))
  data_svy$cycle    <- droplevels(factor(data_svy$cycle))
  if ("smoker3"  %in% names(data_svy)) data_svy$smoker3  <- droplevels(data_svy$smoker3)
```

```r
  if ("alcohol3" %in% names(data_svy)) data_svy$alcohol3 <- droplevels(data_svy$alcohol3)
  data_svy
}

# MI-aware sensitivity fit. `data_df` = complete-case subset for CC fit.
# `mi_filter_fn` (optional, used when mice_active) = function(df) -> filtered imputed df.
# Pooled MI inference is preferred over CC when available.
fit_svy_with_covar_guard <- function(data_df, outcome, main_terms, covars_terms, label,
                                     mi_filter_fn = NULL) {
  build_formula <- function(covs) {
    covar_rhs <- if (length(covs) > 0) paste(covs, collapse = " + ") else ""
    make_rhs_formula(outcome, main_terms, covar_rhs)
  }
  covs <- covars_terms
  intended <- covs
  f <- build_formula(covs)
  mf <- NULL
  repeat {
    mf <- tryCatch(model.frame(f, data = data_df, na.action = na.omit), error = function(e) NULL)
    if (is.null(mf) || nrow(mf) == 0) break
    factor_terms <- names(mf)[vapply(mf, is.factor, logical(1))]
    bad_factor_terms <- factor_terms[vapply(mf[factor_terms], function(x) nlevels(droplevels(x)) < 2, lo
    removable <- intersect(covs, bad_factor_terms)
    if (length(removable) == 0) break
    message("Sensitivity [", label, "]: dropping collapsed factor term(s): ", paste(removable, collapse
    covs <- setdiff(covs, removable)
    f <- build_formula(covs)
  }
  if (is.null(mf) || nrow(mf) == 0) {
    return(list(
      fit = NULL, pooled = NULL, fits_mi = NULL,
      n_model = 0L, n_model_mi = NA_integer_, mi = FALSE,
      intended = intended, realized = covs,
      dropped = paste(setdiff(intended, covs), collapse = ", "),
      error = "No complete-case rows for model frame."
    ))
  }
  data_svy <- prep_svy_frame(data_df)
  des <- tryCatch(
    svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = data_svy),
    error = function(e) svydesign(id = ~1, strata = NULL, weights = ~weight_combined, data = data_svy)
  )
  err <- NULL
  fit <- tryCatch(
    svyglm(f, design = des, family = gaussian()),
    error = function(e) { err <<- conditionMessage(e); NULL }
  )

  pooled <- NULL; fits_mi <- NULL; n_model_mi <- NA_integer_; err_mi <- NULL
  if (mice_active && !is.null(mi_filter_fn)) {
    mi_dfs <- tryCatch(
      lapply(seq_len(config$mice_m), function(i) {
        df <- imp_list$imputations[[i]]
```

```r
        df <- mi_filter_fn(df)
        prep_svy_frame(df)
      }),
      error = function(e) { err_mi <<- conditionMessage(e); NULL }
    )
    if (!is.null(mi_dfs) && length(mi_dfs) > 0) {
      n_model_mi <- nrow(mi_dfs[[1]])
      mi_des <- tryCatch(
        survey::svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE,
                          data = mitools::imputationList(mi_dfs)),
        error = function(e) survey::svydesign(id = ~1, strata = NULL, weights = ~weight_combined,
                                              data = mitools::imputationList(mi_dfs))
      )
      pool_obj <- tryCatch(
        fit_svyglm_pool(f, mi_des),
        error = function(e) { err_mi <<- conditionMessage(e); NULL }
      )
      if (!is.null(pool_obj) && isTRUE(pool_obj$mi)) {
        pooled <- pool_obj$pooled
        fits_mi <- pool_obj$fits
      }
    }
  }
  list(
    fit = fit, pooled = pooled, fits_mi = fits_mi,
    n_model = nrow(mf), n_model_mi = n_model_mi,
    mi = !is.null(pooled),
    intended = intended, realized = covs,
    dropped = paste(setdiff(intended, covs), collapse = ", "),
    error = if (!is.null(pooled) && !is.null(err_mi)) paste(err, "; MI:", err_mi) else err
  )
}

# Wrap a sensitivity object as a `fit_svyglm_pool`-style object for downstream tidy/contrast helpers.
sens_as_fit <- function(obj) {
  if (isTRUE(obj$mi) && !is.null(obj$pooled) && !is.null(obj$fits_mi)) {
    list(fits = obj$fits_mi, pooled = obj$pooled, mi = TRUE)
  } else if (!is.null(obj$fit)) {
    list(fit = obj$fit, mi = FALSE)
  } else {
    NULL
  }
}

sens_ckd_egfr <- qdat %>% filter(!is.na(ckd_sens))
fit_sens1_obj <- fit_svy_with_covar_guard(
  sens_ckd_egfr, "phenoage_accel", c("mixture_qscore", "ckd_sens"), realized_covars_int,
  "CKD redefined as eGFR<60",
  mi_filter_fn = function(df) df[!is.na(df$ckd_sens), , drop = FALSE]
)
fit_sens1 <- fit_sens1_obj$fit

# PhenoAge-recomputed sensitivities use the precomputed columns on qdat / qdat_svy.
```

```r
fit_sens2_obj <- fit_svy_with_covar_guard(
  qdat, "phenoage_nokid", c("mixture_qscore", "ckd"), realized_covars_int,
  "PhenoAge excluding creatinine only",
  mi_filter_fn = identity
)
fit_sens2 <- fit_sens2_obj$fit

fit_sens3_obj <- fit_svy_with_covar_guard(
  qdat, "phenoage_nokid_both", c("mixture_qscore", "ckd"), realized_covars_int,
  "PhenoAge excluding creatinine and albumin",
  mi_filter_fn = identity
)
fit_sens3 <- fit_sens3_obj$fit

get_mix_row <- function(obj, label, n_model_cc = NA_integer_, term_name = "mixture_qscore") {
  fit_for_tidy <- sens_as_fit(obj)
  if (is.null(fit_for_tidy)) {
    return(tibble(sensitivity = label, estimate = NA_real_, conf.low = NA_real_, conf.high = NA_real_,
                  p.value = NA_real_, N = n_model_cc, source = "Failed"))
  }
  tb <- tryCatch(tidy_pool(fit_for_tidy, conf.level = 0.95), error = function(e) NULL)
  if (is.null(tb) || !any(tb$term == term_name)) {
    return(tibble(sensitivity = label, estimate = NA_real_, conf.low = NA_real_, conf.high = NA_real_,
                  p.value = NA_real_, N = n_model_cc, source = "Failed"))
  }
  row <- tb %>% filter(term == term_name) %>% slice(1)
  N_used <- if (isTRUE(fit_for_tidy$mi)) (obj$n_model_mi %||% n_model_cc) else n_model_cc
  src    <- if (isTRUE(fit_for_tidy$mi)) "MI-pooled" else "Complete-case"
  tibble(sensitivity = label, estimate = row$estimate, conf.low = row$conf.low,
         conf.high = row$conf.high, p.value = row$p.value, N = N_used, source = src)
}
# Null-coalesce helper (R < 4.4 doesn't have %||%)
`%||%` <- function(a, b) if (is.null(a) || (is.atomic(a) && length(a) == 1 && is.na(a))) b else a

f_sens_uw <- make_rhs_formula("phenoage_accel_unweighted", c("mixture_qscore", "ckd"), sens_covars_rhs)
f_sens_both <- make_rhs_formula("phenoage_accel", c("mixture_qscore", "ckd"), sens_covars_rhs)
fit_sens_uw_obj <- fit_svy_with_covar_guard(
  qdat, "phenoage_accel_unweighted", c("mixture_qscore", "ckd"), realized_covars_int,
  "Unweighted residualized PhenoAgeAccel (sensitivity)",
  mi_filter_fn = identity
)
fit_sens_both_markers_obj <- fit_svy_with_covar_guard(
  qdat %>% filter(!is.na(egfr), !is.na(acr)), "phenoage_accel", c("mixture_qscore", "ckd"),
  realized_covars_int, "Restricted to participants with both eGFR and ACR",
  mi_filter_fn = function(df) df[!is.na(df$egfr) & !is.na(df$acr), , drop = FALSE]
)
fit_sens_no_acute_obj <- fit_svy_with_covar_guard(
  qdat %>% filter(is.na(crp_acute) | crp_acute == 0L), "phenoage_accel", c("mixture_qscore", "ckd"),
  realized_covars_int, "Exclude participants with CRP > 1 mg/dL",
  mi_filter_fn = function(df) df[is.na(df$crp_acute) | df$crp_acute == 0L, , drop = FALSE]
)
# Strict complete-case sensitivity stays CC by definition.
fit_sens_strict_obj <- fit_svy_with_covar_guard(
```

```r
  qdat_strict, "phenoage_accel", c("mixture_qscore", "ckd"), realized_covars_int,
  "Strict complete-case (qgcomp-compatible PFAS subset), same primary adjustment as Model B",
  mi_filter_fn = NULL
)

# Sensitivity: additional adjustment for cardiometabolic conditions potentially on the causal pathway.
covars_cardiometabolic <- c(
  realized_covars_int,
  if (valid_num("diabetes", qdat)) "diabetes",
  if (valid_num("hypertension", qdat)) "hypertension",
  if (valid_num("cvd", qdat)) "cvd"
)
fit_sens_cardiometabolic_obj <- fit_svy_with_covar_guard(
  qdat, "phenoage_accel", c("mixture_qscore", "ckd"), covars_cardiometabolic,
  "Additional adjustment for diabetes, hypertension, CVD (potentially on causal pathway)",
  mi_filter_fn = identity
)
fit_sens_cardiometabolic <- fit_sens_cardiometabolic_obj$fit

tab4 <- bind_rows(
  get_mix_row(fit_sens1_obj, "CKD redefined as eGFR<60", fit_sens1_obj$n_model),
  get_mix_row(fit_sens2_obj, "PhenoAge excluding creatinine only", fit_sens2_obj$n_model),
  get_mix_row(fit_sens3_obj, "PhenoAge excluding creatinine and albumin", fit_sens3_obj$n_model),
  get_mix_row(fit_sens_uw_obj, "Unweighted residualized PhenoAgeAccel (sensitivity)", fit_sens_uw_obj$n_
  get_mix_row(fit_sens_both_markers_obj, "Restricted to participants with both eGFR and ACR", fit_sens_b
  get_mix_row(fit_sens_no_acute_obj, "Exclude participants with CRP > 1 mg/dL", fit_sens_no_acute_obj$n_
  get_mix_row(fit_sens_cardiometabolic_obj, "Additional adjustment for diabetes, hypertension, CVD (pote
  get_mix_row(fit_sens_strict_obj, "Strict complete-case (qgcomp-compatible PFAS subset), same primary a
) %>%
  transmute(
    sensitivity,
    beta = fmt(estimate),
    conf.low = fmt(conf.low),
    conf.high = fmt(conf.high),
    p.value = fmt_p(p.value),
    N,
    source
  )
kable(tab4, caption = "Table 4. Sensitivity analyses (primary adjustment: sex, race, PIR, BMI, smoking
```

Table 32: Table 4. Sensitivity analyses (primary adjustment: sex, race, PIR, BMI, smoking [3-level], alcohol [3-level], cycle).

sensitivity	beta	conf.low	conf.high	p.value	N	source
CKD redefined as eGFR<60	-1.347	-1.594	-1.100	<0.001	7633	MI-pooled
PhenoAge excluding creatinine only	-1.128	-1.342	-0.915	<0.001	7633	MI-pooled
PhenoAge excluding creatinine and albumin	-0.906	-1.101	-0.711	<0.001	7633	MI-pooled
Unweighted residualized PhenoAgeAccel (sensitivity)	-1.310	-1.554	-1.067	<0.001	7633	MI-pooled

sensitivity	beta	conf.low	conf.high	p.value	N	source
Restricted to participants with both eGFR and ACR	-1.237	-1.476	-0.998	<0.001	7515	MI-pooled
Exclude participants with CRP > 1 mg/dL	-1.181	-1.402	-0.960	<0.001	7375	MI-pooled
Additional adjustment for diabetes, hypertension, CVD (potentially on causal pathway)	-1.338	-1.578	-1.098	<0.001	7633	MI-pooled
Strict complete-case (qgcomp-compatible PFAS subset), same primary adjustment as Model B	-1.171	-1.434	-0.909	<0.001	5267	Complete-case

```
cat("\n*Table 4 note:* Primary and most sensitivity models use the lean adjustment set (sex, race/ethnic
    "Each row reports the `mixture_qscore` coefficient from a survey-weighted model (`svyglm`) with NHA
    "The `source` column distinguishes MI-pooled (Rubin's rules across m=", config$mice_m,
    " imputations) versus complete-case rows; the strict-CC sensitivity remains complete-case by defini
```

```
##
## *Table 4 note:* Primary and most sensitivity models use the lean adjustment set (sex, race/ethnicity
```

```
sens_diagnostics <- bind_rows(
  tibble(sensitivity = "CKD redefined as eGFR<60", N_model = fit_sens1_obj$n_model, dropped_covariates =
  tibble(sensitivity = "PhenoAge excluding creatinine only", N_model = fit_sens2_obj$n_model, dropped_c
  tibble(sensitivity = "PhenoAge excluding creatinine and albumin", N_model = fit_sens3_obj$n_model, dr
  tibble(sensitivity = "Unweighted residualized PhenoAgeAccel (sensitivity)", N_model = fit_sens_uw_obj
  tibble(sensitivity = "Restricted to participants with both eGFR and ACR", N_model = fit_sens_both_mar
  tibble(sensitivity = "Exclude participants with CRP > 1 mg/dL", N_model = fit_sens_no_acute_obj$n_mod
  tibble(sensitivity = "Additional adjustment for diabetes, hypertension, CVD (potentially on causal pa
  tibble(sensitivity = "Strict complete-case (qgcomp-compatible PFAS subset), same primary adjustment a
)
kable(sens_diagnostics, caption = "Sensitivity model diagnostics (dropped covariates and fit errors).")
```

Table 33: Sensitivity model diagnostics (dropped covariates and fit errors).

sensitivity	N_model	dropped_covariates	fit_error
CKD redefined as eGFR<60	5267	None	None
PhenoAge excluding creatinine only	5267	None	None
PhenoAge excluding creatinine and albumin	5267	None	None
Unweighted residualized PhenoAgeAccel (sensitivity)	5267	None	None
Restricted to participants with both eGFR and ACR	5233	None	None
Exclude participants with CRP > 1 mg/dL	5148	None	None
Additional adjustment for diabetes, hypertension, CVD (potentially on causal pathway)	5127	None	None
Strict complete-case (qgcomp-compatible PFAS subset), same primary adjustment as Model B	5267	None	None

```
# Supplementary: single-pollutant models with same primary adjustment as Model B.
single_pollutant_out <- lapply(exposure_vars, function(v) {
  sp_obj <- fit_svy_with_covar_guard(
    qdat, "phenoage_accel", c(v, "ckd"), realized_covars_int,
    paste0("Single pollutant: ", v),
    mi_filter_fn = function(df) df[!is.na(df[[v]]), , drop = FALSE]
  )
```

```r
  diag_row <- tibble(
    exposure = v,
    N_model = if (isTRUE(sp_obj$mi)) sp_obj$n_model_mi else sp_obj$n_model,
    source = if (isTRUE(sp_obj$mi)) "MI-pooled" else "Complete-case",
    dropped_covariates = ifelse(sp_obj$dropped == "", "None", sp_obj$dropped),
    fit_error = ifelse(is.null(sp_obj$error) || sp_obj$error == "", "None", sp_obj$error)
  )
  row <- get_mix_row(sp_obj, paste0("Single pollutant: ", v),
                     n_model_cc = sp_obj$n_model, term_name = v)
  list(
    row  = tibble(
      exposure = v,
      beta = row$estimate, conf.low = row$conf.low, conf.high = row$conf.high,
      p.value = row$p.value, N = row$N, source = row$source
    ),
    diag = diag_row
  )
})
single_pollutant <- bind_rows(lapply(single_pollutant_out, `[[`, "row")) %>%
  mutate(
    beta = fmt(beta),
    conf.low = fmt(conf.low),
    conf.high = fmt(conf.high),
    p.value = fmt_p(p.value)
  )
single_pollutant_diagnostics <- bind_rows(lapply(single_pollutant_out, `[[`, "diag"))

kable(
  single_pollutant,
  caption = "Supplementary Table. Single-pollutant associations with PhenoAge acceleration (survey-weigh
)
```

Table 34: Supplementary Table. Single-pollutant associations with PhenoAge acceleration (survey-weighted, same primary adjustment as Model B); MI-pooled when MICE is active.

exposure	beta	conf.low	conf.high	p.value	N	source
LBXPFOS_log2	-1.245	-1.515	-0.975	<0.001	7633	MI-pooled
LBXPFOA_log2	-1.526	-1.816	-1.235	<0.001	7633	MI-pooled
LBXPFNA_log2	-0.935	-1.244	-0.625	<0.001	7633	MI-pooled
LBXPFHS_log2	-0.904	-1.081	-0.726	<0.001	7633	MI-pooled

```r
kable(single_pollutant_diagnostics, caption = "Supplementary model diagnostics (single-pollutant N, sou
```

Table 35: Supplementary model diagnostics (single-pollutant N, source, dropped covariates, fit errors).

exposure	N_model	source	dropped_covariates	fit_error
LBXPFOS_log2	7633	MI-pooled	None	None
LBXPFOA_log2	7633	MI-pooled	None	None
LBXPFNA_log2	7633	MI-pooled	None	None

exposure	N_model	source	dropped_covariates	fit_error
LBXPFHS_log2	7633	MI-pooled	None	None

Figures

```
dir.create("figures", showWarnings = FALSE)
n_chems <- length(primary_exposure_vars)

# Figure 1: survey-weighted predicted lines per CKD stratum.
# Mirror the proven standalone logic: filter NAs, fall back design->weights-only->lm,
# normalize predict() output (svyglm returns svystat; lm returns atomic).
fit_one_stratum <- function(sub_df) {
  needed <- c("phenoage_accel", "mixture_qscore", "weight_combined", "psu", "strata")
  sub_df <- sub_df[stats::complete.cases(sub_df[, needed, drop = FALSE]), , drop = FALSE]
  if (nrow(sub_df) == 0) return(NULL)
  fit <- tryCatch(
    survey::svyglm(phenoage_accel ~ mixture_qscore,
                   design = survey::svydesign(id = ~psu, strata = ~strata,
                                              weights = ~weight_combined, nest = TRUE,
                                              data = sub_df),
                   family = gaussian()),
    error = function(e) NULL)
  if (is.null(fit)) {
    fit <- tryCatch(
      survey::svyglm(phenoage_accel ~ mixture_qscore,
                     design = survey::svydesign(id = ~1, weights = ~weight_combined,
                                                data = sub_df),
                     family = gaussian()),
      error = function(e) NULL)
  }
  if (is.null(fit)) {
    fit <- tryCatch(stats::lm(phenoage_accel ~ mixture_qscore,
                              data = sub_df, weights = sub_df$weight_combined),
                    error = function(e) NULL)
  }
  fit
}

pred_lines <- tryCatch({
  full_score_range <- range(qdat$mixture_qscore, na.rm = TRUE)
  pdat_list <- lapply(c(0L, 1L), function(ck) {
    sub_df <- qdat %>% filter(ckd == ck)
    fit_pred <- fit_one_stratum(sub_df)
    if (is.null(fit_pred)) return(NULL)
    score_seq <- seq(full_score_range[1], full_score_range[2], length.out = 100)
    newdat <- data.frame(mixture_qscore = score_seq)
    pr <- tryCatch(
      predict(fit_pred, newdata = newdat, se.fit = TRUE, type = "response"),
      error = function(e) NULL)
    if (is.null(pr)) return(NULL)
    fit_v <- as.numeric(if (is.list(pr)) pr$fit else pr)
    se_v  <- if (is.list(pr) && !is.null(pr$se.fit)) as.numeric(pr$se.fit)
```

```
            else as.numeric(survey::SE(pr))
    newdat$fit <- fit_v
    newdat$se  <- se_v
    newdat$ckd <- ifelse(ck == 1L, "CKD", "No CKD")
    cf <- coef(fit_pred)
    vc <- tryCatch(stats::vcov(fit_pred), error = function(e) NULL)
    slope <- as.numeric(cf["mixture_qscore"])
    se_b  <- if (!is.null(vc)) sqrt(vc["mixture_qscore", "mixture_qscore"]) else NA_real_
    newdat$slope    <- slope
    newdat$slope_lo <- if (is.finite(se_b)) slope - 1.96 * se_b else NA_real_
    newdat$slope_hi <- if (is.finite(se_b)) slope + 1.96 * se_b else NA_real_
    newdat
  })
  pdat_list <- Filter(Negate(is.null), pdat_list)
  if (length(pdat_list) == 0) NULL else bind_rows(pdat_list)
}, error = function(e) NULL)

# Overall marginal direction: as mixture increases, what happens to phenoage_accel (pooled model).
overall_direction <- tryCatch({
  svy_full <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = qda
  fit_overall <- svyglm(phenoage_accel ~ mixture_qscore, design = svy_full, family = gaussian())
  score_seq <- seq(min(qdat$mixture_qscore, na.rm = TRUE), max(qdat$mixture_qscore, na.rm = TRUE), leng
  newdat <- data.frame(mixture_qscore = score_seq)
  preds <- predict(fit_overall, newdata = newdat, se.fit = TRUE, type = "response")
  newdat$fit <- as.numeric(preds$fit)
  newdat
}, error = function(e) NULL)

# Faceted hex density of (mixture quantile score, PhenoAge acceleration) by CKD,
# with a thick survey-weighted trend line + 95% CI ribbon and per-facet slope label.
qdat_plot <- qdat %>%
  mutate(ckd_label = factor(ifelse(ckd == 0L, "No CKD", "CKD"),
                            levels = c("No CKD", "CKD")))
pred_plot <- if (!is.null(pred_lines)) {
  pred_lines %>%
    mutate(ckd_label = factor(ckd, levels = c("No CKD", "CKD")))
} else NULL

# Per-facet slope annotation: prefer MI-pooled marginal effects from fit_int
# (design-based contrasts already used in Table 3); fall back to per-stratum fits.
mi_ann <- tryCatch({
  if (exists("fit_int") && !is.null(fit_int)) {
    cf_names_a <- names(coef_pool(fit_int))
    int_name_a <- cf_names_a[stringr::str_detect(cf_names_a, "mixture_qscore:ckd")]
    if (length(int_name_a) == 1 && "mixture_qscore" %in% cf_names_a) {
      c0 <- setNames(rep(0, length(cf_names_a)), cf_names_a); c0["mixture_qscore"] <- 1
      c1 <- setNames(rep(0, length(cf_names_a)), cf_names_a); c1["mixture_qscore"] <- 1; c1[int_name_a]
      ctr0 <- contrast_pool(fit_int, c0)
      ctr1 <- contrast_pool(fit_int, c1)
      tibble::tibble(
        ckd_label = factor(c("No CKD", "CKD"), levels = c("No CKD", "CKD")),
        slope    = c(ctr0$estimate, ctr1$estimate),
        slope_lo = c(ctr0$conf.low, ctr1$conf.low),
```

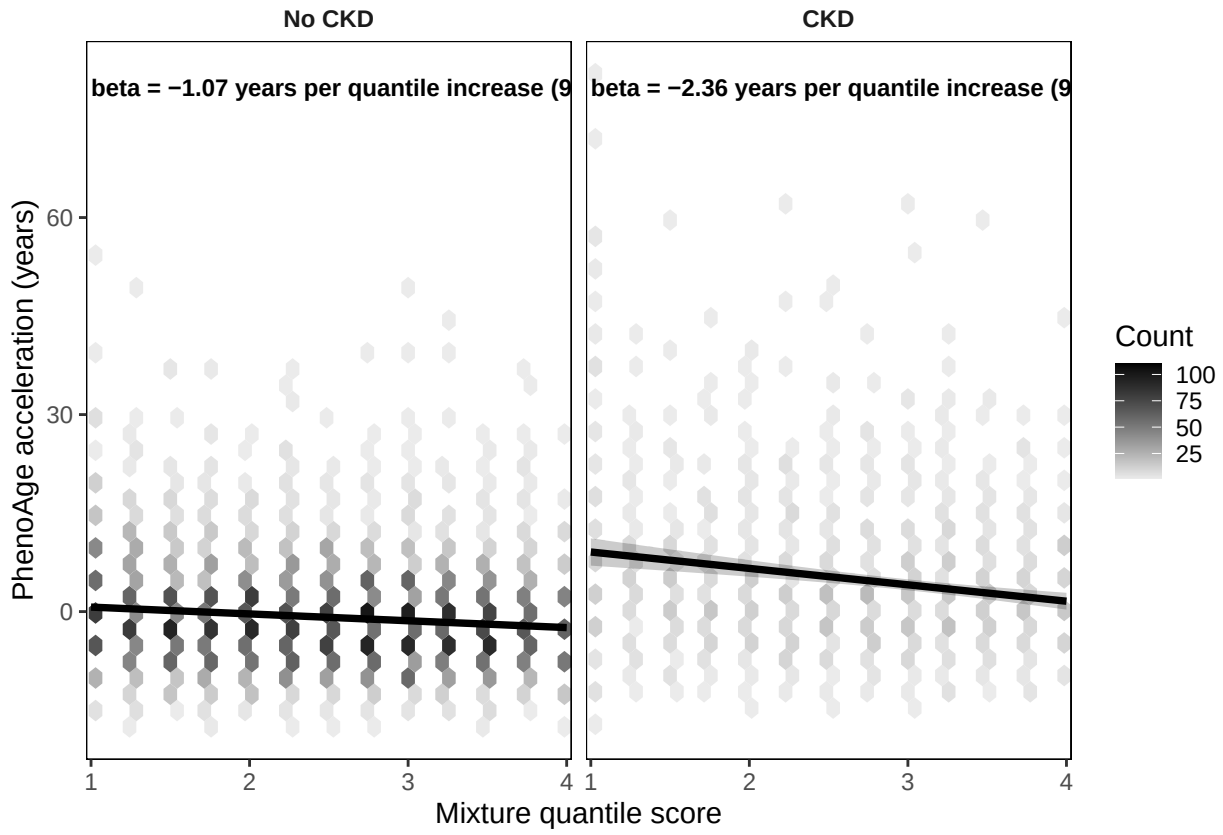
```
        slope_hi = c(ctr0$conf.high, ctr1$conf.high)
      )
    } else NULL
  } else NULL
}, error = function(e) NULL)
ann_df <- if (!is.null(mi_ann)) mi_ann else if (!is.null(pred_plot)) {
  pred_plot %>%
    group_by(ckd_label) %>%
    summarise(slope = unique(slope), slope_lo = unique(slope_lo),
              slope_hi = unique(slope_hi), .groups = "drop")
} else NULL
if (!is.null(ann_df)) {
  ann_df <- ann_df %>%
    mutate(label = ifelse(
      is.finite(slope_lo) & is.finite(slope_hi),
      sprintf("beta = %+0.2f years per quantile increase (95%% CI %+0.2f, %+0.2f)",
              slope, slope_lo, slope_hi),
      sprintf("beta = %+0.2f years per quantile increase", slope)
    ))
}

theme_facet_box <- theme_bw(base_size = 11) +
  theme(panel.grid.major = element_blank(),
        panel.grid.minor = element_blank(),
        panel.border = element_rect(color = "black", fill = NA, linewidth = 0.4),
        strip.background = element_blank(),
        strip.text = element_text(face = "bold"),
        legend.position = "right",
        legend.key.height = unit(0.6, "lines"))

trend_color <- "#000000" # bright red, prints as a strong dark gray

fig1_hex <- ggplot(qdat_plot, aes(x = mixture_qscore, y = phenoage_accel)) +
  geom_hex(bins = 35) +
  scale_fill_gradient(low = "grey92", high = "black", name = "Count") +
  facet_wrap(~ ckd_label, ncol = 2) +
  scale_x_continuous(limits = range(qdat_plot$mixture_qscore, na.rm = TRUE),
                     expand = expansion(mult = c(0.01, 0.01))) +
  labs(x = "Mixture quantile score",
       y = "PhenoAge acceleration (years)") +
  theme_facet_box
if (!is.null(pred_plot)) {
  fig1_hex <- fig1_hex +
    geom_ribbon(data = pred_plot,
                aes(x = mixture_qscore,
                    ymin = fit - 1.96 * se, ymax = fit + 1.96 * se,
                    group = ckd_label),
                fill = trend_color, alpha = 0.20, inherit.aes = FALSE) +
    geom_line(data = pred_plot,
              aes(x = mixture_qscore, y = fit, group = ckd_label),
              color = trend_color, linewidth = 1.3, inherit.aes = FALSE)
}
if (!is.null(ann_df)) {
```

```
  y_top <- max(qdat_plot$phenoage_accel, na.rm = TRUE)
  x_left <- min(qdat_plot$mixture_qscore, na.rm = TRUE)
  fig1_hex <- fig1_hex +
    geom_text(data = ann_df,
              aes(x = x_left, y = y_top, label = label),
              hjust = 0, vjust = 1.2, size = 3.2, color = trend_color,
              fontface = "bold", inherit.aes = FALSE)
}
fig1_hex
```




```
ggsave("figures/fig1_hex_facet_CKD.jpg", plot = fig1_hex,
       width = 9.5, height = 5, dpi = 600, device = "jpeg",
       quality = 95, bg = "white")

# Figure 2: forest plot of marginal mixture effects (design-based).
fig2 <- NULL
if (!is.null(fit_int)) {
  cf_names <- names(coef_pool(fit_int))
  int_name <- cf_names[str_detect(cf_names, "mixture_qscore:ckd")]
  if (length(int_name) == 1 && "mixture_qscore" %in% cf_names) {
    c_ckd0 <- setNames(rep(0, length(cf_names)), cf_names); c_ckd0["mixture_qscore"] <- 1
    c_ckd1 <- setNames(rep(0, length(cf_names)), cf_names); c_ckd1["mixture_qscore"] <- 1; c_ckd1[int_na
    ctr0 <- contrast_pool(fit_int, c_ckd0)
    ctr1 <- contrast_pool(fit_int, c_ckd1)

    mk_row <- function(ctr, grp) {
```

```r
      tibble(
        group = grp,
        estimate = ctr$estimate,
        conf.low = ctr$conf.low,
        conf.high = ctr$conf.high,
        p.value = ctr$p.value
      )
    }
    forest_df <- bind_rows(
      mk_row(ctr0, "No CKD"),
      mk_row(ctr1, "CKD")
    ) %>%
      mutate(group = factor(group, levels = c("No CKD", "CKD")))

    fig2 <- ggplot(forest_df, aes(x = estimate, y = group, color = group)) +
      geom_vline(xintercept = 0, linetype = "dashed", color = "gray40") +
      geom_errorbarh(aes(xmin = conf.low, xmax = conf.high), height = 0.18, linewidth = 0.8) +
      geom_point(size = 2.8) +
      geom_text(aes(label = ifelse(is.na(p.value), "",
                                   ifelse(p.value < 0.001, "p<0.001",
                                          paste0("p=", format(round(p.value, 3), nsmall = 3))))),
                nudge_y = 0.2, size = 3, color = "black", show.legend = FALSE) +
      scale_color_manual(values = c("No CKD" = "#1b9e77", "CKD" = "#d95f02")) +
      labs(
        title = "Marginal mixture effects by CKD status",
        subtitle = "Design-based contrasts from interaction model",
        x = "Beta (95% CI) for mixture score",
        y = NULL
      ) +
      theme_classic(base_size = 11) +
      theme(legend.position = "none")
    fig2
    ggsave("figures/fig2_forest_marginal_effects.jpg", plot = fig2, width = 7, height = 4, dpi = 300, d
  }
}

# Figure 3: MICE convergence diagnostics in MCMC-style plate
# (Trace | Autocorrelation | Kernel density) per variable, boxed panels, no overall title.
fig3 <- NULL
if (exists("mids_obj") && !is.null(mids_obj)) {
  miss_n <- vapply(mice_target_vars, function(v) sum(is.na(qdat_svy[[v]])), integer(1))
  top_vars <- names(sort(miss_n, decreasing = TRUE))[seq_len(min(4, length(miss_n)))]
  top_vars <- top_vars[miss_n[top_vars] > 0]

  if (length(top_vars) >= 1) {
    # ---- Trace: chainMean is [variable, iteration, chain] ----
    cm <- mids_obj$chainMean
    trace_df <- as.data.frame.table(cm, responseName = "value", stringsAsFactors = FALSE) %>%
      dplyr::rename(variable = Var1, iter_lab = Var2, chain_lab = Var3) %>%
      dplyr::mutate(
        iter  = as.integer(sub("[^0-9]+", "", iter_lab)),
        chain = as.integer(sub("[^0-9]+", "", chain_lab))
      ) %>%
```

```r
  dplyr::filter(variable %in% top_vars, !is.na(value))

# ---- ACF: per chain, averaged across chains ----
max_iter <- max(trace_df$iter, na.rm = TRUE)
lag_max  <- max(1L, min(8L, max_iter - 1L))
acf_ci   <- 1.96 / sqrt(max_iter) # white-noise 95% CI for ACF (Bartlett)
acf_df <- trace_df %>%
  dplyr::group_by(variable, chain) %>%
  dplyr::arrange(iter, .by_group = TRUE) %>%
  dplyr::summarise(
    lag = list(0:lag_max),
    acf = list(as.numeric(stats::acf(value, plot = FALSE, lag.max = lag_max)$acf)),
    .groups = "drop"
  ) %>%
  tidyr::unnest(c(lag, acf)) %>%
  dplyr::group_by(variable, lag) %>%
  dplyr::summarise(acf = mean(acf, na.rm = TRUE), .groups = "drop")

# ---- Density: observed (.imp == 0) vs imputed ----
long_full <- mice::complete(mids_obj, "long", include = TRUE) %>%
  dplyr::mutate(source = ifelse(.imp == 0, "Observed", "Imputed"))
var_pieces <- lapply(top_vars, function(v) {
  x <- long_full[[v]]
  val <- if (is.factor(x)) as.numeric(x) else suppressWarnings(as.numeric(x))
  data.frame(source = long_full$source, variable = v, value_num = val,
             stringsAsFactors = FALSE)
})
long_imp <- do.call(rbind, var_pieces) %>%
  dplyr::filter(!is.na(value_num)) %>%
  dplyr::mutate(source = factor(source, levels = c("Observed", "Imputed")))

# ---- ResearchGate-style theme: boxed, no grid, B&W ----
bw_boxed <- function(base_size = 9) {
  theme_bw(base_size = base_size) +
    theme(
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      panel.border = element_rect(color = "black", fill = NA, linewidth = 0.35),
      plot.background = element_rect(fill = "white", color = NA),
      plot.title = element_text(hjust = 0.5, face = "plain", size = base_size + 1),
      legend.position = "none"
    )
}

build_row <- function(var_name, col_titles) {
  tdf <- dplyr::filter(trace_df, variable == var_name)
  adf <- dplyr::filter(acf_df,   variable == var_name)
  ddf <- dplyr::filter(long_imp, variable == var_name)

  p_trace <- ggplot(tdf, aes(x = iter, y = value, group = chain)) +
    geom_line(alpha = 0.35, linewidth = 0.3, color = "black") +
    scale_x_continuous(expand = c(0.02, 0)) +
    labs(x = "Iteration", y = var_name,
```

```
            title = if (col_titles) "Traceplot" else NULL) +
        bw_boxed()

      p_acf <- ggplot(adf, aes(x = lag, y = acf)) +
        geom_hline(yintercept = 0, color = "black", linewidth = 0.35) +
        geom_hline(yintercept = c(-acf_ci, acf_ci), color = "gray55", linewidth = 0.25) +
        geom_segment(aes(xend = lag, yend = 0), linewidth = 0.35, color = "black") +
        geom_point(size = 1.6, color = "gray20") +
        scale_y_continuous(limits = c(-0.5, 1), expand = c(0, 0)) +
        labs(x = "Lag", y = "ACF",
            title = if (col_titles) "Autocorrelation" else NULL) +
        bw_boxed()

      # KDE: observed (solid) vs imputed (dashed); vertical lines at observed 2.5/97.5%
      q_obs <- stats::quantile(ddf$value_num[ddf$source == "Observed"],
                               probs = c(0.025, 0.975), na.rm = TRUE)
      p_kde <- ggplot(ddf, aes(x = value_num, linetype = source)) +
        geom_density(color = "black", linewidth = 0.45, na.rm = TRUE) +
        geom_vline(xintercept = q_obs, color = "gray45",
                   linewidth = 0.35, linetype = "solid") +
        scale_linetype_manual(values = c("Observed" = "solid", "Imputed" = "dashed")) +
        labs(x = "Value", y = "Density",
            title = if (col_titles) "Kernel density estimation" else NULL) +
        bw_boxed()

      p_trace + p_acf + p_kde + plot_layout(ncol = 3)
    }

    rows <- lapply(seq_along(top_vars), function(i) build_row(top_vars[i], col_titles = (i == 1L)))
    fig3 <- wrap_plots(rows, ncol = 1)

    fig3
    ggsave("figures/fig3_mice_convergence.jpg", plot = fig3,
           width = 8.2, height = 1.55 * length(top_vars) + 0.6,
           dpi = 300, device = "jpeg", quality = 95, bg = "white")
  }
}
```

Cautions and Limitations

- Cross-sectional NHANES data do not establish temporality.
- PhenoAge includes kidney-related biomarkers (creatinine, albumin), creating potential circularity with CKD definitions. Sensitivity analyses recompute PhenoAge excluding (a) creatinine only and (b) both creatinine and albumin; both are non-standard derivatives and interpreted as robustness checks.
- PhenoAge uses Levine (2018) coefficients with correct units: albumin g/L (NHANES g/dL × 10), creatinine µmol/L (mg/dL × 88.4), glucose mmol/L (mg/dL ÷ 18.018). Model B uses phenoage_accel (age-residualized) and does not adjust for age again. Mixture quantiles are computed within each NHANES cycle to avoid temporal confounding (PFAS trends across retained cycles). Survey design uses full PSU/strata when possible; if the design fails (e.g. singleton PSUs), a weights-only fallback is used and noted.
- `qgcomp` point estimates are presented descriptively (pooled sampling weights applied, but PSU/strata not incorporated for variance estimation). Statistical inference relies on the survey-weighted regression models (`svyglm`) above. Bootstrap variance estimation incorporating replicate weights was not imple-

mented because NHANES does not provide replicate weights for environmental subsamples across all cycles.

- LOD handling follows NHANES detection-flag conventions when indicator columns are available (`*LC` or `*L`): values flagged below LOD (indicator == 1) are replaced with value / sqrt(2). When no LOD indicator is available, values are left unchanged (no fallback NA imputation), which improves transparency but can reduce complete-case sample size.
- **Weight selection**: a weight audit table is printed and `config$weight_type = "auto"` applies the top-coverage eligible weight. For PFAS, auto-selection excludes full-sample interview/MEC weights (`WTINT2YR`, `WTMEC2YR`) and prioritizes environmental subsample weights (`WTS*2YR`), including the combined cross-cycle PFAS weight column.
- The primary PFAS mixture was defined as a fixed four-chemical set available across all retained cycles (PFOS, PFOA, PFNA, and PFHxS), with complete-case requirement on that set. PFDA was not consistently available across cycles and was therefore excluded from the primary mixture and evaluated only in supplementary analyses. Strict complete-case (qgcomp-compatible PFAS subset) modeling is reported as sensitivity. A sensitivity analysis using an available-exposure (adaptive) mixture can be added to assess robustness when using broader PFAS information.
- Exposure quantile cutpoints are defined within cycle using cycle-specific NHANES sampling weights.
- PhenoAgeAccel is computed as the residual from a survey-weighted regression of PhenoAge on chronological age (`svyglm`), ensuring the construct is population-representative. An unweighted residualization is included as sensitivity.
- CRP unit detection uses a two-part rule: (1) table-name provenance (HSCRP tables imply mg/L), and (2) a conservative distribution heuristic (median > 1.5 **and** p75 > 3). If either triggers, CRP is converted from mg/L to mg/dL (/ 10). Cycle-level diagnostic tables are printed. PhenoAge is recomputed after any conversions. Glucose is converted from mmol/L to mg/dL if median < 6.
- Extreme biomarker values are handled by plausibility filters (WBC > 50, glucose > 600, ALP > 500 set to missing). CRP > 1 mg/dL is retained in primary analyses and evaluated in sensitivity analyses excluding potential acute inflammatory states.
- Glucose is sourced from MEC biochemistry (`glu_mec`: BIOPRO/L40) for the primary analysis. The separate GLU table (fasting) is joined with renamed variables (`LBXGLU_GLU`, `LBXSGL_GLU`) to avoid duplicate columns; it is retained for diagnostics but not used in the primary model.
- Marginal effects in the interaction model (Table 3) use `svycontrast()` with design-based degrees of freedom and t-distribution p-values, maintaining full complex-survey consistency.
- Because complete-case requirements can collapse factor levels in the interaction model frame, Model B reports both intended and realized covariate sets; unsupported factor terms are dropped with explicit logging.
- **Multiple imputation (MICE).** When `config$use_mice = TRUE`, missing values on model covariates (smoking, alcohol, BMI, PIR, education, physical activity, diabetes, hypertension, CVD) are imputed within the Model B base sample (all participants with non-missing PhenoAge, CKD, design variables, and a complete fixed PFAS score). Outcome (PhenoAge / PhenoAgeAccel), exposures (PFAS, mixture quantile score), and survey design variables (PSU, strata, weights) are NEVER imputed. Imputation uses chained equations with `pmm` for continuous variables, `logreg` for binary factors, and `polyreg` for the 3-level Never/Former/Current factors. Model B and the survey-weighted sensitivity / single-pollutant analyses are then re-fit on each of the m imputed datasets and pooled with Rubin's rules via `mitools::MIcombine`; design-based marginal contrasts (`svycontrast`) are likewise computed per imputation and pooled. The complete-case Model B fit is retained and reported (in the Table 3 note) as a transparency check. **Table 2 (Model A overall mixture effect via `qgcomp`)** is also MI-pooled: `qgcomp.noboot` is fit on each imputed dataset and the overall psi (and its variance) are combined with Rubin's rules; a complete-case `qgcomp` row is reported alongside as a sensitivity check. `qgcomp` variance does not incorporate PSU/strata, so survey-design inference for the interaction question still relies on Model B (Table 3).

Reproducibility

```r
sessionInfo()
```

```
## R version 4.5.3 (2026-03-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] splines   grid      stats     graphics  grDevices utils     datasets
## [8] methods   base
##
## other attached packages:
##  [1] patchwork_1.3.2 mitools_2.4     mice_3.19.0     tibble_3.3.1
##  [5] forcats_1.0.1   qgcomp_2.18.10  knitr_1.51      broom_1.0.12
##  [9] ggplot2_4.0.2   survey_4.5      survival_3.8-6  Matrix_1.7-4
## [13] stringr_1.6.0   purrr_1.2.1     tidyr_1.3.2     dplyr_1.2.1
## [17] nhanesA_1.4.1
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1    rootSolve_1.8.2.4   farver_2.1.2
##  [4] S7_0.2.1            fastmap_1.2.0       rpart_4.1.24
##  [7] digest_0.6.39       lifecycle_1.0.5     magrittr_2.0.5
## [10] compiler_4.5.3      rlang_1.2.0         tools_4.5.3
## [13] yaml_2.3.12         labeling_0.4.3      curl_7.0.0
## [16] plyr_1.8.9          xml2_1.5.2          RColorBrewer_1.1-3
## [19] abind_1.4-8         withr_3.0.2         foreign_0.8-91
## [22] numDeriv_2016.8-1.1 nnet_7.3-20         jomo_2.7-6
## [25] scales_1.4.0        iterators_1.0.14    MASS_7.3-65
## [28] cli_3.6.6           rmarkdown_2.31      ragg_1.5.2
## [31] reformulas_0.4.4    generics_0.1.4      otel_0.2.0
## [34] httr_1.4.8          minqa_1.2.8         DBI_1.3.0
## [37] rvest_1.0.5         selectr_0.5-1       vctrs_0.7.2
## [40] boot_1.3-32         glmnet_4.1-10       sandwich_3.1-1
## [43] carData_3.0-6       car_3.1-5           mitml_0.4-5
## [46] Formula_1.2-5       systemfonts_1.3.2   foreach_1.5.2
## [49] hexbin_1.28.5       glue_1.8.0          pan_1.9
## [52] nloptr_2.2.1        codetools_0.2-20    stringi_1.8.7
## [55] gtable_0.3.6        shape_1.4.6.1       AER_1.2-16
## [58] lmtest_0.9-40       lme4_2.0-1          pillar_1.11.1
## [61] htmltools_0.5.9     pscl_1.5.9          R6_2.6.1
```

```
## [64] textshaping_1.0.5   Rdpack_2.6.6        evaluate_1.0.5
## [67] lattice_0.22-9      rbibutils_2.4.1     backports_1.5.1
## [70] arm_1.15-2          Rcpp_1.1.1          coda_0.19-4.1
## [73] gridExtra_2.3       nlme_3.1-168        xfun_0.57
## [76] zoo_1.8-15          pkgconfig_2.0.3
```